

DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence

DeepSeek-AI
research@deepseek.com

Abstract

We present a preview version of DeepSeek-V4 series, including two strong Mixture-of-Experts (MoE) language models — DeepSeek-V4-Pro with 1.6T parameters (49B activated) and DeepSeek-V4-Flash with 284B parameters (13B activated) — both supporting a context length of one million tokens. DeepSeek-V4 series incorporate several key upgrades in architecture and optimization: (1) a hybrid attention architecture that combines Compressed Sparse Attention (CSA) and Heavily Compressed Attention (HCA) to improve long-context efficiency; (2) Manifold-Constrained Hyper-Connections (*mHC*) that enhance conventional residual connections; (3) and the Muon optimizer for faster convergence and greater training stability. We pre-train both models on more than 32T diverse and high-quality tokens, followed by a comprehensive post-training pipeline that unlocks and further enhances their capabilities. DeepSeek-V4-Pro-Max, the maximum reasoning effort mode of DeepSeek-V4-Pro, redefines the state-of-the-art for open models, outperforming its predecessors in core tasks. Meanwhile, DeepSeek-V4 series are highly efficient in long-context scenarios. In the one-million-token context setting, DeepSeek-V4-Pro requires only 27% of single-token inference FLOPs and 10% of KV cache compared with DeepSeek-V3.2. This enables us to routinely support one-million-token contexts, thereby making long-horizon tasks and further test-time scaling more feasible. The model checkpoints are available at <https://huggingface.co/collections/deepseek-ai/deepseek-v4>.

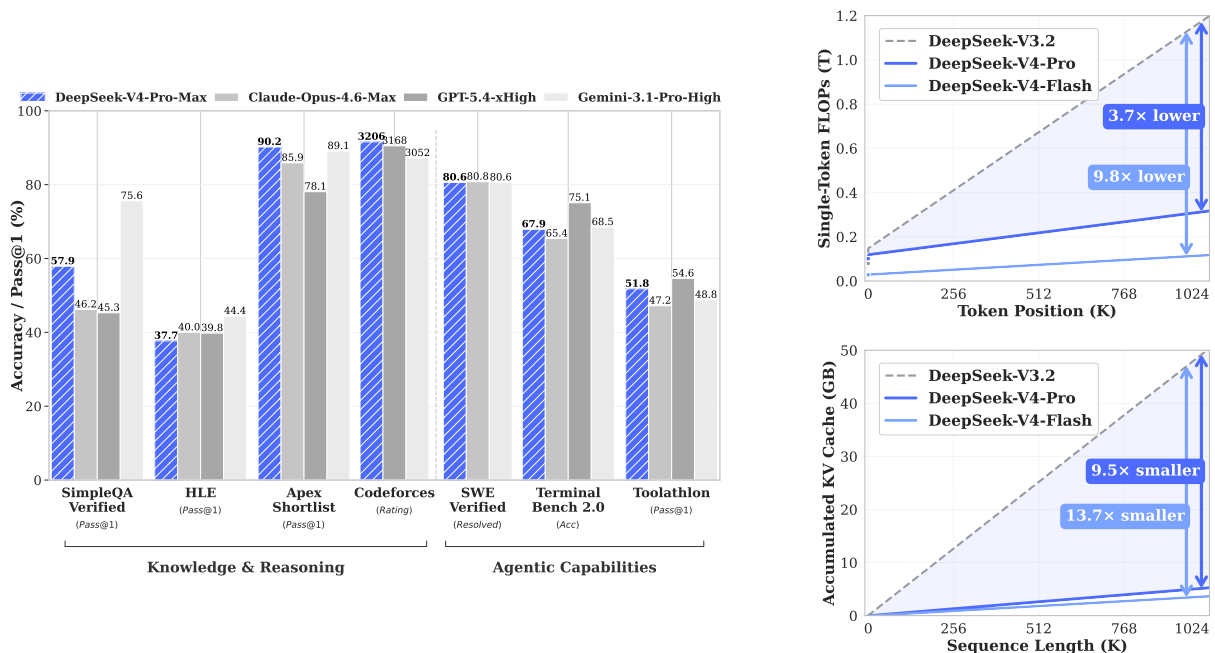


Figure 1 | **Left:** benchmark performance of DeepSeek-V4-Pro-Max and its counterparts. **Right:** inference FLOPs and KV cache size of DeepSeek-V4 series and DeepSeek-V3.2.

DeepSeek-V4: 迈向高效百万令牌上下

深度求索AI

research@deepseek.com

摘要

我们推出DeepSeek-V4系列的预览版本，包含两个强大的专家混合（MoE）语言模型——1.6万亿参数（激活490亿）的DeepSeek-V4-Pro与2840亿参数（激活130亿）的DeepSeek-V4-Flash，均支持百万token上下文长度。该系列在架构与优化层面实现多项关键升级：(1) 融合压缩稀疏注意力（CSA）与强压缩注意力（HCA）的混合注意力架构，提升长上下文效率；(2) 流形约束超连接（*mHC*）增强传统残差连接；(3) 采用 μ 子优化器加速收敛并提升训练稳定性。两个模型均在超过32万亿高质量多样化token上预训练，并通过全面后训练流程释放并强化其能力。其中DeepSeek-V4-Pro-Max作为Pro版的极致推理模式，重新定义了开源模型的技术标杆，在核心任务上超越前代。同时该系列在长上下文场景中表现高效：百万token上下文下，DeepSeek-V4-Pro的单token推理FLOPs仅需DeepSeek-V3.2的27%，KV缓存占用仅10%，使得常规支持百万token上下文成为可能，从而让长程任务与测试时扩展更易实现。模型检查点发布于<https://huggingface.co/collections/deepseek-ai/deepseek-v4>。

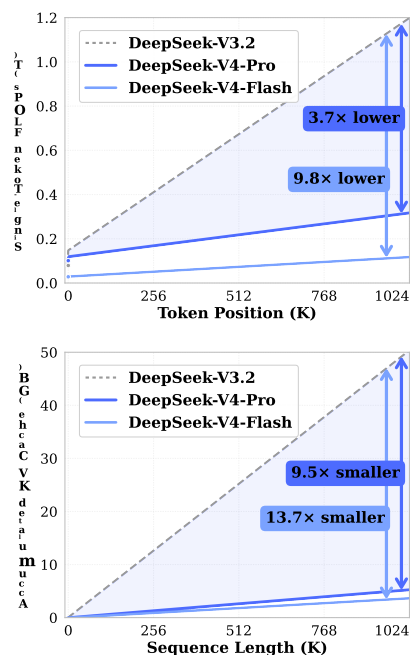
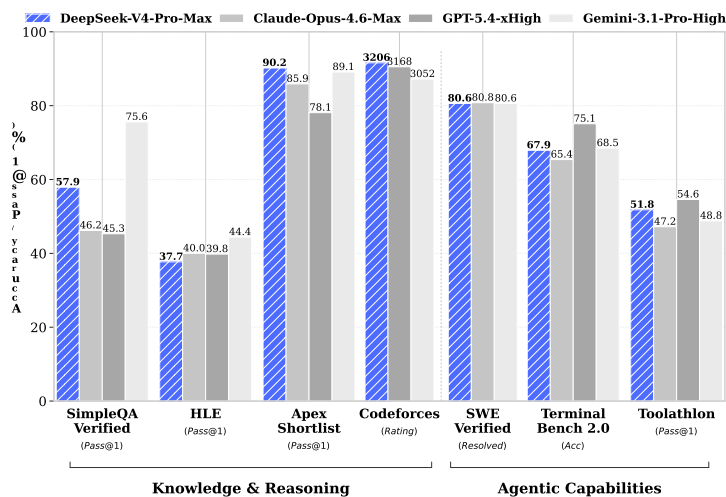


图1 | 左：DeepSeek-V4-Pro-Max与同类产品的基准性能对比。右：DeepSeek-V4系列与DeepSeek-V3.2的推理FLOPs及KV缓存大小。

Contents

1	Introduction	4
2	Architecture	6
2.1	Designs Inherited from DeepSeek-V3	7
2.2	Manifold-Constrained Hyper-Connections	7
2.3	Hybrid Attention with CSA and HCA	9
2.3.1	Compressed Sparse Attention	9
2.3.2	Heavily Compressed Attention	11
2.3.3	Other Details	12
2.3.4	Efficiency Discussion	13
2.4	Muon Optimizer	14
3	General Infrastructures	15
3.1	Fine-Grained Communication-Computation Overlap in Expert Parallelism	15
3.2	Flexible and Efficient Kernel Development with TileLang	16
3.3	High-Performance Batch-Invariant and Deterministic Kernel Libraries	18
3.4	Training Framework	19
3.4.1	Efficient Implementation of Muon	19
3.4.2	Cost-Effective and Memory-Efficient Implementation of <i>mHC</i>	20
3.4.3	Contextual Parallelism for Long-Context Attention	20
3.4.4	Extended Automatic Differentiation for Flexible Activation Checkpointing	21
3.5	Inference Framework	21
3.5.1	KV Cache Structure and Management	21
3.5.2	On-Disk KV Cache Storage	23
4	Pre-Training	24
4.1	Data Construction	24
4.2	Pre-Training Setups	24
4.2.1	Model Setups	24
4.2.2	Training Setups	25
4.2.3	Mitigating Training Instability	26
4.3	Evaluations	27
4.3.1	Evaluation Benchmarks	27
4.3.2	Evaluation Results	27

目录

1 引言	4
2 架构 6	
2.1 继承自DeepSeek-V3的设计	7
2.2 流形约束的超连接	7
2.3 CSA与HCA混合注意力机制	9
2.3.1 压缩稀疏注意力	9
2.3.2 高度压缩注意力	11
2.3.3 其他细节	12
2.3.4 效率讨论	13
2.4 μ 子优化器	14
3 大通用基础设施 15	
3.1 专家并行中的细粒度通信-计算重叠	15
3.2 使用TileLang实现灵活高效的内核开发	16
3.3 高性能批处理不变性与确定性内核库	18
3.4 训练框架	19
3.4.1 μ 子的高效实现	19
3.4.2 $\{v^*\}$ HC 的成本效益与内存高效实现	20
3.4.3 面向长上下文注意力的情境并行机制	20
3.4.4 灵活激活检查点的扩展自动微分技术	21
3.5 推理框架	21
3.5.1 KV缓存结构与管理	21
3.5.2 磁盘KV缓存存储方案	23
4 预训练 24	
4.1 数据构建	24
4.2 预训练设置	24
4.2.1 模型设置	24
4.2.2 训练设置	25
4.2.3 缓解训练不稳定性	26
4.3 评估	27
4.3.1 评估基准	27
4.3.2 评估结果	27

5	Post-Training	28
5.1	Post-Training Pipeline	28
5.1.1	Specialist Training	28
5.1.2	On-Policy Distillation	32
5.2	Post-Training Infrastructures	33
5.2.1	FP4 Quantization-Aware Training	33
5.2.2	Efficient Teacher Scheduling for Full-Vocabulary OPD	34
5.2.3	Preemptible and Fault-Tolerant Rollout Service	34
5.2.4	Scaling RL Framework for Million-Token Context	35
5.2.5	Sandbox Infrastructure for Agentic AI	35
5.3	Standard Benchmark Evaluation	36
5.3.1	Evaluation Setup	36
5.3.2	Evaluation Results	37
5.4	Performance on Real-World Tasks	41
5.4.1	Chinese Writing	41
5.4.2	Search	42
5.4.3	White-Collar Task	42
5.4.4	Code Agent	44
6	Conclusion, Limitations, and Future Directions	44
A	Author List and Acknowledgment	54
A.1	Author List	54
A.2	Acknowledgment	55
B	Evaluation Details	55

5 训练后	28
5.1 训练后流程	28
5.1.1 专家训练	32
5.1.2 同策略蒸馏	33
5.2 训练后基础设施	33
5.2.1 FP4量化感知训练	33
5.2.2 全词汇OPD的高效教师调度	34
5.2.3 可抢占式容错推出服务	35
5.2.4 百万级token上下文的RL框架扩展	35
5.2.5 智能体AI沙盒基础设施	35
5.3 标准基准评估	36
5.3.1 评估设置	36
5.3.2 评估结果	37
5.4 现实任务中的性能表现	41
5.4.1 中文写作	41
5.4.2 搜索	42
5.4.3 白领任务	42
5.4.4 代码代理	44
6 结论、局限性与未来方向	44
作者列表与致谢	54
A.1 作者列表	54
A.2 致谢	55
B 评估详情	55

1. Introduction

The emergence of reasoning models (DeepSeek-AI, 2025; OpenAI, 2024c) has established a new paradigm of test-time scaling, driving substantial performance gains for Large Language Models (LLMs). However, this scaling paradigm is fundamentally constrained by the quadratic computational complexity of the vanilla attention mechanism (Vaswani et al., 2017), which creates a prohibitive bottleneck for ultra-long contexts and reasoning processes. Concurrently, the emergence of long-horizon scenarios and tasks — from complex agentic workflows to massive cross-document analysis — has also made efficient support for ultra-long contexts critical for future progress. While recent open-source efforts (Bai et al., 2025a; DeepSeek-AI, 2024; MiniMax, 2025; Qwen, 2025) have advanced general capabilities, this core architectural inefficiency in handling ultra-long sequences remains a key impediment, limiting further gains from test-time scaling and hindering further exploration into long-horizon scenarios and tasks.

In order to break the efficiency barrier in ultra-long contexts, we develop the DeepSeek-V4 series, including the preview versions of DeepSeek-V4-Pro with 1.6T parameters (49B activated) and DeepSeek-V4-Flash with 284B parameters (13B activated). Through architectural innovations, DeepSeek-V4 series achieve a dramatic leap in computational efficiency for processing ultra-long sequences. This breakthrough enables efficient support for a context length of one million tokens, ushering in a new era of million-length contexts for next-generation LLMs. We believe our capability to efficiently handle ultra-long sequences unlocks the next frontier of test-time scaling, paves the way for deeper research into long-horizon tasks, and establishes a necessary foundation for exploring future paradigms like online learning.

Compared with the DeepSeek-V3 architecture (DeepSeek-AI, 2024), DeepSeek-V4 series retain the DeepSeekMoE framework (Dai et al., 2024) and Multi-Token Prediction (MTP) strategy, while introducing several key innovations in architecture and optimization. To enhance long-context efficiency, we design a hybrid attention mechanism combining Compressed Sparse Attention (CSA) and Heavily Compressed Attention (HCA). CSA compresses the KV caches along the sequence dimension and then performs DeepSeek Sparse Attention (DSA) (DeepSeek-AI, 2025), whereas HCA applies more aggressive compression to the KV caches but keeps dense attention. To strengthen modeling capability, we incorporate Manifold-Constrained Hyper-Connections (*mHC*) (Xie et al., 2026) that upgrade conventional residual connections. Additionally, we introduce the Muon (Jordan et al., 2024; Liu et al., 2025) optimizer to the training of DeepSeek-V4 series, leading to faster convergence and improved training stability.

To enable efficient training and inference for DeepSeek-V4 series as well as productive development, we introduce several infrastructure optimizations. First, we design and implement a single fused kernel for MoE modules that fully overlaps computation, communication, and memory access. Second, we employ TileLang (Wang et al., 2026), a Domain-Specific Language (DSL) to balance development productivity and runtime efficiency. Third, we provide efficient batch-invariant and deterministic kernel libraries to ensure bitwise reproducibility across training and inference. Fourth, for the training framework, we extend the autograd framework with tensor-level checkpointing for fine-grained recomputation control; and we enhance training efficiency with a hybrid ZeRO strategy for the Muon optimizer, cost-effective *mHC* implementations via recomputation and fused kernels, and two-stage contextual parallelism to manage compressed attention. Fifth, for the inference framework, we design a heterogeneous KV cache structure with on-disk storage strategies to enable efficient shared-prefix reuse. In addition, during the post-training stage, we incorporate FP4 quantization-aware training for MoE expert weights and the indexer QK path to reduce memory and computation.

1. 引言

推理模型的出现（DeepSeek-AI, 2025; OpenAI, 2024c）确立了测试时规模扩展的新范式，推动了大语言模型（LLMs）性能的显著提升。然而，这一扩展范式从根本上受限于原始注意力机制（Vaswani等人, 2017）的二次计算复杂度，为超长上下文和推理过程形成了难以逾越的瓶颈。与此同时，从复杂的智能体工作流到大规模跨文档分析等长视野场景与任务的涌现，也使得高效支持超长上下文成为未来发展的关键。尽管近期开源成果（Bai等人, 2025a; DeepSeek-AI, 2024; MiniMax, 2025; Qwen, 2025）提升了通用能力，但处理超长序列时的核心架构低效问题仍是主要障碍，既限制了测试时规模扩展的进一步收益，也阻碍了对长视野场景与任务的深入探索。

为了突破超长上下文的效率壁垒，我们开发了DeepSeek-V4系列，包括1.6万亿参数（激活490亿）的DeepSeek-V4-Pro预览版和2840亿参数（激活130亿）的DeepSeek-V4-Flash预览版。通过架构创新，DeepSeek-V4系列在处理超长序列时实现了计算效率的跨越式提升。这一突破性进展使得支持百万token级上下文长度成为可能，为下一代大语言模型开启了百万长度上下文的新纪元。我们相信，高效处理超长序列的能力将解锁测试时扩展的新疆界，为长周期任务的深入研究铺平道路，并为探索在线学习等未来范式奠定必要基础。

与DeepSeek-V3架构（DeepSeek-AI, 2024年）相比，DeepSeek-V4系列保留了DeepSeekMoE框架（Dai等人, 2024年）和多令牌预测（MTP）策略，同时在架构和优化方面引入了多项关键创新。为提升长上下文处理效率，我们设计了一种混合注意力机制，结合压缩稀疏注意力（CSA）与重度压缩注意力（HCA）。CSA沿序列维度压缩KV缓存后执行DeepSeek稀疏注意力（DSA）（DeepSeek-AI, 2025年），而HCA对KV缓存采用更激进的压缩但仍保持稠密注意力。为增强建模能力，我们融入了流形约束超连接（*mHC*）（Xie等人, 2026年）以升级传统残差连接。此外，我们在DeepSeek-V4系列训练中引入了Muon优化器（Jordan等人, 2024年; Liu等人, 2025年），实现了更快的收敛速度与更高的训练稳定性。

为实现DeepSeek-V4系列的高效训练与推理及生产力开发，我们引入多项基础设施优化。首先，我们设计并实现了MoE模块的单一融合内核，完全重叠计算、通信与内存访问。其次，我们采用领域专用语言TileLang（Wang等人, 2026）平衡开发效率与运行时性能。第三，我们提供高效的批不变确定性内核库，确保训练与推理过程的比特级可复现性。第四，在训练框架中扩展自动微分系统，通过张量级检查点实现细粒度重计算控制；通过混合ZeRO策略优化Muon优化器、基于重计算与融合内核的高性价比{v*}HC实现，以及两级上下文并行管理压缩注意力机制，全面提升训练效率。第五，在推理框架中设计异构KV缓存结构，结合磁盘存储策略实现共享前缀高效复用。此外，在后训练阶段引入MoE专家权重与索引器QK路径的FP4量化感知训练，显著降低内存与计算开销。

By employing hybrid CSA and HCA, along with precision optimizations on computation and storage, DeepSeek-V4 series achieve significantly lower inference FLOPs and a substantially reduced KV cache size compared with DeepSeek-V3.2, especially in long-context settings. The right part of Figure 1 demonstrates the estimated single-token inference FLOPs and accumulated KV cache size of DeepSeek-V3.2 and DeepSeek-V4 series. In the scenario of 1M-token context, even DeepSeek-V4-Pro, which has a larger number of activated parameters, attains only 27% of the single-token FLOPs (measured in equivalent FP8 FLOPs) and 10% of the KV cache size relative to DeepSeek-V3.2. Furthermore, DeepSeek-V4-Flash, with its smaller number of activated parameters, pushes efficiency even further: in the 1M-token context setting, it achieves only 10% of the single-token FLOPs and 7% of the KV cache size compared with DeepSeek-V3.2. Additionally, for DeepSeek-V4 series, the routed expert parameters utilize FP4 precision. While the peak FLOPs for $\text{FP4} \times \text{FP8}$ operations are currently the same as $\text{FP8} \times \text{FP8}$ on existing hardware, they can theoretically be implemented to be 1/3 more efficient on future hardware, which will further enhance the efficiency of DeepSeek-V4 series.

During pre-training, we train DeepSeek-V4-Flash on 32T tokens and DeepSeek-V4-Pro on 33T tokens, respectively. After pre-training, these two models can natively and efficiently support 1M-length contexts. In our internal evaluations, DeepSeek-V4-Flash-Base already surpasses DeepSeek-V3.2-Base across a majority of benchmarks with its more parameter-efficient design. DeepSeek-V4-Pro-Base further extends this advantage to set a new performance standard among DeepSeek foundation models, achieving comprehensive superiority across reasoning, coding, long-context, and world knowledge tasks.

The post-training pipeline of DeepSeek-V4 series features a two-stage paradigm: the independent cultivation of domain-specific experts, followed by unified model consolidation via on-policy distillation (Gu et al., 2024; Lu and Lab, 2025). Initially, for each target domain — such as mathematics, coding, agent, and instruction following — a separate expert model is trained independently. The base model first undergoes Supervised Fine-Tuning (SFT) on high-quality, domain-specific data to establish foundational capabilities. Subsequently, Reinforcement Learning (RL) is applied using Group Relative Policy Optimization (GRPO) (DeepSeek-AI, 2025), which further optimizes the model for domain-aligned behaviors guided by reward models tailored to specific success criteria. This phase yields a diverse set of specialized experts, each excelling in its respective field. Finally, to integrate these distinct proficiencies, a single unified model is trained through on-policy distillation, wherein the unified model acts as the student learning to optimize the reverse KL loss with teacher models.

Summary of Core Evaluation Results

- **Knowledge:** In assessments of broad world knowledge, DeepSeek-V4-Pro-Max, the maximum reasoning effort mode of DeepSeek-V4-Pro, significantly outperforms leading open-source models on the SimpleQA (OpenAI, 2024d) and Chinese-SimpleQA (He et al., 2024) benchmarks. Regarding educational knowledge — evaluated via MMLU-Pro (Wang et al., 2024b), HLE (Phan et al., 2025), and GPQA (Rein et al., 2023) — DeepSeek-V4-Pro-Max shows a marginal lead over its open-source counterparts. DeepSeek-V4-Pro-Max has significantly closed the gap with the leading proprietary model, Gemini-3.1-Pro, despite still trailing it in these knowledge-based evaluations.
- **Reasoning:** Through the expansion of reasoning tokens, DeepSeek-V4-Pro-Max demonstrates superior performance relative to GPT-5.2 and Gemini-3.0-Pro on standard reasoning benchmarks. Nevertheless, its performance falls marginally short of GPT-5.4 and Gemini-3.1-Pro, suggesting a developmental trajectory that trails state-of-the-art frontier models by approximately 3 to 6 months. Furthermore, DeepSeek-V4-Flash-Max achieves comparable

通过采用混合CSA与HCA技术，并在计算和存储上实施精准优化，DeepSeek-V4系列相较DeepSeek-V3.2显著降低了推理FLOPs并大幅缩减了KV缓存大小，尤其在长上下文场景中。图1右侧展示了DeepSeek-V3.2与DeepSeek-V4系列的单令牌推理FLOPs估算值及累积KV缓存大小。在100万令牌上下文的场景下，即便是激活参数更多的DeepSeek-V4-Pro，其单令牌FLOPs（以等效FP8 FLOPs衡量）仅为DeepSeek-V3.2的27%，KV缓存大小仅占10%。而激活参数更少的DeepSeek-V4-Flash则进一步提升了效率：在同等百万令牌上下文设置中，其单令牌FLOPs和KV缓存大小分别仅为DeepSeek-V3.2的10%与7%。此外，DeepSeek-V4系列的路由专家参数采用FP4精度。虽然当前硬件上FP4×FP8运算的峰值FLOPs与FP8×FP8相同，但理论上未来硬件可实现1/3倍的效率提升，这将进一步增强DeepSeek-V4系列的效能。

在预训练阶段，我们分别使用32T token训练DeepSeek-V4-Flash模型，以及33T token训练DeepSeek-V4-Pro模型。完成预训练后，这两个模型能原生高效支持100万长度的上下文处理。内部评估显示，凭借更高参数效率的设计，DeepSeek-V4-Flash-Base已在多数基准测试中超越DeepSeek-V3.2-Base。而DeepSeek-V4-Pro-Base进一步扩大优势，为DeepSeek基础模型树立了新的性能标杆，在推理、编程、长文本处理及世界知识任务中实现全面领先。

DeepSeek-V4系列的训练后流程采用两阶段范式：先独立培养领域专家模型，再通过在线策略蒸馏统一整合（Gu等，2024；Lu和Lab，2025）。首先针对数学、编程、智能体及指令遵循等目标领域，分别独立训练专家模型。基础模型先在高质量领域数据上进行监督微调（SFT）以建立基础能力，随后采用群体相对策略优化（GRPO）（DeepSeek-AI，2025）进行强化学习（RL），通过定制奖励模型引导的领域对齐行为进一步优化模型。此阶段产出一组各有所长的专业专家模型。最终通过在线策略蒸馏训练统一模型实现能力融合——该统一模型作为学生模型，以逆向KL散度为优化目标向教师模型学习。

核心评估结果总结

- 知识：在广泛世界知识的评估中，DeepSeek-V4-Pro的最强推理模式DeepSeek-V4-Pro-Max，在SimpleQA（OpenAI，2024d）和Chinese-SimpleQA（He等人，2024）基准测试上显著领先于主流开源模型。在教育知识方面——通过MMLU-Pro（Wang等人，2024b）、HLE（Phan等人，2025）和GPQA（Rein等人，2023）进行评估——DeepSeek-V4-Pro-Max对开源模型仅保持微弱优势。尽管在这些基于知识的评估中仍稍逊于领先的专有模型Gemini-3.1-Pro，DeepSeek-V4-Pro-Max已显著缩小了与其差距。
- 推理：通过扩展推理标记，DeepSeek-V4-Pro-Max在标准推理基准测试中展现出相对于GPT-5.2和Gemini-3.0-Pro的卓越性能。然而，其表现略逊于GPT-5.4和Gemini-3.1-Pro，这表明其发展轨迹落后于前沿尖端模型约3至6个月。此外，DeepSeek-V4-Flash-Max实现了与之相当的

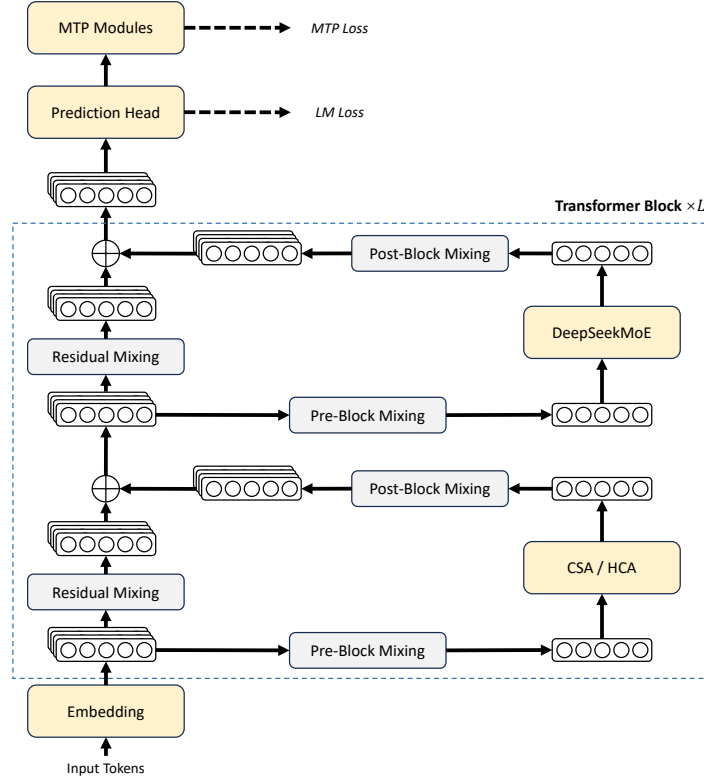


Figure 2 | Overall architecture of DeepSeek-V4 series. We use hybrid CSA (Compressed Sparse Attention) and HCA (Heavily Compressed Attention) for attention layers, DeepSeekMoE for feed-forward layers, and strengthen conventional residual connections with *m*HC.

performance to GPT-5.2 and Gemini-3.0-Pro, establishing itself as a highly cost-effective architecture for complex reasoning tasks.

- **Agent:** On public benchmarks, DeepSeek-V4-Pro-Max is on par with leading open-source models, such as Kimi-K2.6 and GLM-5.1, but slightly worse than frontier closed models. In our internal evaluation, DeepSeek-V4-Pro-Max outperforms Claude Sonnet 4.5 and approaches the level of Opus 4.5.
- **Long-Context:** DeepSeek-V4-Pro-Max delivers strong results on synthetic and real use cases with a 1-million-token context window, surpassing even Gemini-3.1-Pro on academic benchmarks.
- **DeepSeek-V4-Pro v.s. DeepSeek-V4-Flash:** DeepSeek-V4-Flash-Max exhibits lower performance in knowledge evaluations due to its smaller parameter scale. However, it achieves comparable results on reasoning tasks when allocated a larger thinking budget. In agent evaluations, while DeepSeek-V4-Flash-Max matches the performance of DeepSeek-V4-Pro-Max on several benchmarks, it still trails its larger counterpart on more complex, high-difficulty tasks.

2. Architecture

Overall, DeepSeek-V4 series retain the Transformer (Vaswani et al., 2017) architecture and Multi-Token Prediction (MTP) modules (DeepSeek-AI, 2024; Gloeckle et al., 2024), while introducing several key upgrades over DeepSeek-V3: (1) firstly, we introduce the Manifold-Constrained Hyper-Connections (*m*HC) (Xie et al., 2026) to strengthen conventional residual connections;

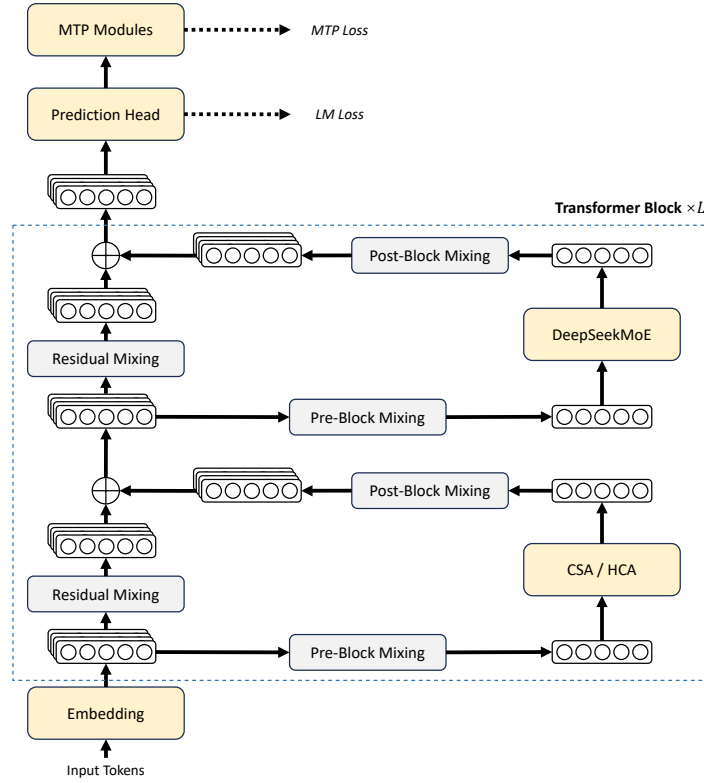


图2 | DeepSeek-V4系列的整体架构。我们在注意力层采用混合CSA（压缩稀疏注意力）与HCA（重度压缩注意力），前馈层采用DeepSeekMoE结构，并通过 m HC强化传统残差连接。

性能媲美GPT-5.2和Gemini-3.0-Pro，确立了其在复杂推理任务中高性价比架构的地位。

- 代理：在公开基准测试中，DeepSeek-V4-Pro-Max与领先的开源模型如Kimi-K2.6和GLM-5.1表现相当，但略逊于前沿的闭源模型。根据我们的内部评估，DeepSeek-V4-Pro-Max超越了Claude Sonnet 4.5，并接近Opus 4.5的水平。
- 长上下文能力：DeepSeek-V4-Pro-Max在100万token上下文窗口下，于合成及实际应用场景中表现卓越，学术基准测试成绩甚至超越Gemini-3.1-Pro。
- DeepSeek-V4-Pro 对比 DeepSeek-V4-Flash：DeepSeek-V4-Flash-Max 因其参数量较小，在知识评估中表现稍逊。但在分配更多思考资源时，其推理任务成绩可媲美前者。代理评估中，尽管 DeepSeek-V4-Flash-Max 在多项基准测试中与 DeepSeek-V4-Pro-Max 持平，面对更复杂的高难度任务时仍落后于这一更大规模的版本。

2. 架构

总体而言，DeepSeek-V4系列保留了Transformer架构（Vaswani等人，2017）与多令牌预测模块（DeepSeek-AI，2024；Gloeckle等人，2024），同时在DeepSeek-V3基础上进行了多项关键升级：（1）首先，我们引入了流形约束超连接（ m HC）（Xie等人，2026）以增强传统残差连接；

(2) secondly, we design a hybrid attention architecture, which greatly improves long-context efficiency through Compressed Sparse Attention and Heavily Compressed Attention. (3) thirdly, we employ Muon (Jordan et al., 2024; Liu et al., 2025) as the optimizer. For the Mixture-of-Experts (MoE) components, we still adopt the DeepSeekMoE (Dai et al., 2024) architecture, with only minor adjustments from DeepSeek-V3. The Multi-Token Prediction (MTP) (DeepSeek-AI, 2024; Gloeckle et al., 2024; Li et al., 2024; Qi et al., 2020) configuration remains identical to that of DeepSeek-V3. All other unspecified details follow the settings established in DeepSeek-V3 (DeepSeek-AI, 2024). Figure 2 illustrates the overall architecture of DeepSeek-V4, and the details are described below.

2.1. Designs Inherited from DeepSeek-V3

Mixture-of-Experts. As previous DeepSeek-series models (DeepSeek-AI, 2024; DeepSeek-AI, 2024), DeepSeek-V4 series also adopt the DeepSeekMoE paradigm (Dai et al., 2024) for Feed-Forward Networks (FFNs), which sets fine-grained routed experts and shared experts. Different from DeepSeek-V3, we change the activation function that computes the affinity scores from $\text{Sigmoid}(\cdot)$ into $\text{Sqrt}(\text{Softplus}(\cdot))$. For load balancing, we also employ the auxiliary-loss-free strategy (DeepSeek-AI, 2024; Wang et al., 2024a), augmented by a slight sequence-wise balance loss that prevents extreme imbalance within individual sequences. For DeepSeek-V4, we remove the constraint on the number of routing target nodes, and carefully redesign the parallelism strategy to maintain training efficiency. Furthermore, compared with DeepSeek-V3, we replace the dense FFN layers in the initial several Transformer blocks with MoE layers that employ Hash routing (Roller et al., 2021). The Hash routing strategy determines the target experts of each token according to a predefined hash function with regard to the input token ID.

Multi-Token Prediction. As DeepSeek-V3, DeepSeek-V4 series also set MTP modules and objectives. Given that the MTP strategy has been validated in DeepSeek-V3, we adopt the same strategy for DeepSeek-V4 series without modification.

2.2. Manifold-Constrained Hyper-Connections

As shown in Figure 2, DeepSeek-V4 series incorporate Manifold-Constrained Hyper-Connections ($m\text{HC}$) (Xie et al., 2026) to strengthen the conventional residual connections between adjacent Transformer blocks. Compared with naive Hyper-Connections (HC) (Zhu et al., 2025), the core idea of $m\text{HC}$ is to constrain the residual mapping onto a specific manifold, and thus enhance the stability of signal propagation across layers while preserving model expressivity. This subsection briefly introduces the standard HC and describes how we design $m\text{HC}$ for stable training.

Standard Hyper-Connections. The standard HC expands the width of the residual stream by a factor of n_{hc} . Specifically, the shape of the residual stream is expanded from $\mathbb{R}^d$ to $\mathbb{R}^{n_{\text{hc}} \times d}$, where d is the hidden size of the actual layer input. Let $X_l = [\mathbf{x}_{l,1}; \dots; \mathbf{x}_{l,n_{\text{hc}}}]^T \in \mathbb{R}^{n_{\text{hc}} \times d}$ be the residual state before the l -th layer. HC introduces three linear mappings: an input mapping $A_l \in \mathbb{R}^{1 \times n_{\text{hc}}}$, a residual transformation $B_l \in \mathbb{R}^{n_{\text{hc}} \times n_{\text{hc}}}$, and an output mapping $C_l \in \mathbb{R}^{n_{\text{hc}} \times 1}$. The update of the residual state is then formulated as:

$$X_{l+1} = B_l X_l + C_l \mathcal{F}_l(A_l X_l), \quad (1)$$

where $\mathcal{F}_l$ denotes the l -th layer (e.g., an MoE layer), whose input and output shapes are both $\mathbb{R}^d$. Note that the actual layer input $A_l X_l \in \mathbb{R}^d$ is also d -dimensional, so the expanded residual

(2) 其次，我们设计了一种混合注意力架构，通过压缩稀疏注意力（Compressed Sparse Attention）和重度压缩注意力（Heavily Compressed Attention）显著提升了长上下文处理效率。(3) 第三，我们采用Muon（Jordan等人，2024；Liu等人，2025）作为优化器。对于混合专家系统（MoE）组件，我们仍沿用DeepSeekMoE（Dai等人，2024）架构，仅对DeepSeek-V3进行了微小调整。多令牌预测（MTP）（DeepSeek-AI，2024；Gloeckle等人，2024；Li等人，2024；Qi等人，2020）配置与DeepSeek-V3保持完全一致。其他未明确说明的细节均遵循DeepSeek-V3（DeepSeek-AI，2024）既定设置。图2展示了DeepSeek-V4的整体架构，具体细节如下所述。

2.1. 继承自DeepSeek-V3的设计

专家混合（Mixture-of-Experts）。与先前DeepSeek系列模型（DeepSeek-AI，2024；DeepSeek-AI，2024）一致，DeepSeek-V4系列同样采用DeepSeekMoE范式（Dai等人，2024）构建前馈网络（FFNs），该范式设置了细粒度路由专家与共享专家。不同于DeepSeek-V3，我们将计算亲和度分数的激活函数由Sigmoid($\cdot$)改为Sqrt(Softplus($\cdot$))。在负载均衡方面，我们采用无辅助损失策略（DeepSeek-AI，2024；Wang等人，2024a），并辅以轻量级序列级平衡损失，防止单个序列内出现极端失衡。针对DeepSeek-V4，我们取消了对路由目标节点数量的限制，并精心重构并行策略以保持训练效率。此外，相较DeepSeek-V3，我们将前几个Transformer模块中的稠密FFN层替换为采用哈希路由（Roller等人，2021）的MoE层。该哈希路由策略根据预定义的输入token ID哈希函数，确定每个token的目标专家。

多令牌预测。与DeepSeek-V3一样，DeepSeek-V4系列也设置了MTP模块和目标。鉴于MTP策略已在DeepSeek-V3中得到验证，我们在DeepSeek-V4系列中采用相同的策略，不作修改。

2.2. 流形约束的超连接

如图2所示，DeepSeek-V4系列采用了流形约束超连接（ m HC）（Xie等人，2026）以增强相邻Transformer模块间的传统残差连接。相较于朴素超连接（HC）（Zhu等人，2025）， m HC的核心思想在于将残差映射约束至特定流形上，从而在保持模型表达能力的同时增强跨层信号传播的稳定性。本小节简要介绍标准HC，并阐述我们如何设计 m HC以实现稳定训练。

标准超连接。标准HC将残差流的宽度扩展了 n_{hc} 倍。具体来说，残差流的形状从 $\mathbb{R}^d$ 扩展到 $\mathbb{R}^{n_{hc} \times d}$ ，其中 d 为实际层输入的隐藏大小。设 $X_l = [x_{l,1}; \dots; x_{l,n_{hc}}]^T \in \mathbb{R}^{n_{hc} \times d}$ 为第 l 层前的残差状态。HC引入了三个线性映射：输入映射 $A_l \in \mathbb{R}^{1 \times n_{hc}}$ 、残差变换 $B_l \in \mathbb{R}^{n_{hc} \times n_{hc}}$ 和输出映射 $C_l \in \mathbb{R}^{n_{hc} \times 1}$ 。残差状态的更新公式如下：

$$X_{l+1} = B_l X_l + C_l \mathcal{F}_l(A_l X_l), \quad (1)$$

其中 $\mathcal{F}_l$ 表示第 l 层（例如一个混合专家层），其输入和输出形状均为 $\mathbb{R}^d$ 。需注意实际层输入 $A_l X_l \in \mathbb{R}^d$ 也是 d 维的，因此扩展残差

width does not influence the design of the inner layers. HC decouples the residual width from the actual hidden size, offering a complementary scaling axis with minimal computational overhead, as n_{hc} is typically much smaller than the hidden size d . However, even though HC has demonstrated potential in improving model performance, we find that the training will frequently exhibit numerical instability when stacking multiple layers, which hinders the scaling of HC.

Manifold-Constrained Residual Mapping. The core innovation of $m\text{HC}$ is to constrain the residual mapping matrix B_l to the manifold of doubly stochastic matrices (the Birkhoff polytope) $\mathcal{M}$, and thus enhance the stability of signal propagation across layers:

$$B_l \in \mathcal{M} := \{M \in \mathbb{R}^{n \times n} \mid M\mathbf{1}_n = \mathbf{1}_n, \mathbf{1}_n^T M = \mathbf{1}_n^T, M \geq 0\}. \quad (2)$$

This constraint ensures that the spectral norm of the mapping matrix $\|B_l\|_2$ is bounded by 1, so the residual transformation is non-expansive, which increases the numerical stability during both the forward pass and backpropagation. Besides, the set $\mathcal{M}$ is closed under multiplication, which guarantees stability in the scenarios of deep stacks of $m\text{HC}$. In addition, the input transformation A_l and output transformation C_l are also constrained to be non-negative and bounded via a Sigmoid function to avoid the risk of signal cancellation.

Dynamic Parameterization. The parameters of three linear mappings are dynamically generated, which are decomposed into a dynamic (input-dependent) component and a static (input-independent) component. Given the input $X_l \in \mathbb{R}^{n_{\text{hc}} \times d}$, it is first flattened and normalized: $\hat{X}_l = \text{RMSNorm}(\text{vec}(X_l)) \in \mathbb{R}^{1 \times n_{\text{hc}}d}$. Then, we follow the conventional HC to generate the unconstrained raw parameters $\tilde{A}_l \in \mathbb{R}^{1 \times n_{\text{hc}}}$, $\tilde{B}_l \in \mathbb{R}^{n_{\text{hc}} \times n_{\text{hc}}}$, and $\tilde{C}_l \in \mathbb{R}^{n_{\text{hc}} \times 1}$:

$$\tilde{A}_l = \alpha_l^{\text{pre}} \cdot (\hat{X}_l W_l^{\text{pre}}) + S_l^{\text{pre}}, \quad (3)$$

$$\tilde{B}_l = \alpha_l^{\text{res}} \cdot \text{Mat}(\hat{X}_l W_l^{\text{res}}) + S_l^{\text{res}}, \quad (4)$$

$$\tilde{C}_l = \alpha_l^{\text{post}} \cdot (\hat{X}_l W_l^{\text{post}})^T + S_l^{\text{post}}, \quad (5)$$

where $W_l^{\text{pre}}, W_l^{\text{post}} \in \mathbb{R}^{n_{\text{hc}}d \times n_{\text{hc}}}$ and $W_l^{\text{res}} \in \mathbb{R}^{n_{\text{hc}}d \times n_{\text{hc}}^2}$ are learnable parameters for generating the dynamic components; $\text{Mat}(\cdot)$ reshapes a vector of size $1 \times n_{\text{hc}}^2$ into a matrix of size $n_{\text{hc}} \times n_{\text{hc}}$; $S_l^{\text{pre}} \in \mathbb{R}^{1 \times n_{\text{hc}}}$, $S_l^{\text{post}} \in \mathbb{R}^{n_{\text{hc}} \times 1}$, and $S_l^{\text{res}} \in \mathbb{R}^{n_{\text{hc}} \times n_{\text{hc}}}$ are learnable static biases; and $\alpha_l^{\text{pre}}, \alpha_l^{\text{res}}, \alpha_l^{\text{post}} \in \mathbb{R}$ are learnable gating factors initialized to small values.

Applying Parameter Constraints. After obtaining the unconstrained raw parameters $\tilde{A}_l, \tilde{B}_l, \tilde{C}_l$, we then apply constraints described earlier to them to enhance the numerical stability. To be specific, for the input and output mappings, we employ a Sigmoid function $\sigma(\cdot)$ to ensure their non-negativity and boundedness:

$$A_l = \sigma(\tilde{A}_l), \quad (6)$$

$$C_l = 2\sigma(\tilde{C}_l). \quad (7)$$

As for the residual mapping $\tilde{B}_l$, we project it onto the manifold of doubly stochastic matrices $\mathcal{M}$. This is achieved by the Sinkhorn-Knopp algorithm, which first applies an exponential function to $\tilde{B}_l$ to ensure positivity, getting $M^{(0)} = \exp(\tilde{B}_l)$, and then iteratively performs column and row normalization:

$$M^{(t)} = \mathcal{T}_r(\mathcal{T}_c(M^{(t-1)})), \quad (8)$$

where $\mathcal{T}_r$ and $\mathcal{T}_c$ denote row and column normalization, respectively. This iteration converges to a constrained doubly stochastic matrix $B_l = M^{(t_{\text{max}})}$. We choose $t_{\text{max}} = 20$ as a practical value.

宽度并不影响内层的设计。HC将残差宽度与实际隐藏大小解耦，提供了一个补充的扩展维度，且计算开销极低，因为 n_{hc} 通常远小于隐藏大小 d 。然而，尽管HC在提升模型性能方面已展现出潜力，但我们发现当堆叠多层时，训练过程常会表现出数值不稳定性，这阻碍了HC的扩展。

流形约束残差映射。 $m\text{HC}$ 的核心创新在于将残差映射矩阵 B_l 约束在双随机矩阵的流形（Birkhoff多胞形） $\mathcal{M}$ 上，从而增强信号在层间传播的稳定性：

$$B_l \in \mathcal{M} := \{M \in \mathbb{R}^{n \times n} \mid M\mathbf{1}_n = \mathbf{1}_n, \mathbf{1}_n^T M = \mathbf{1}_n^T, M \geq 0\}. \quad (2)$$

该约束确保了映射矩阵 $\|B_l\|_2$ 的谱范数被限定在1以内，因此残差变换是非扩张性的，从而提升了前向传播和反向传播过程中的数值稳定性。此外，集合 $\mathcal{M}$ 在乘法运算下是封闭的，这保证了在深度堆叠 $m\text{HC}$ 场景下的稳定性。另外，输入变换 A_l 和输出变换 C_l 也通过Sigmoid函数被约束为非负且有界的，以避免信号抵消的风险。

动态参数化。三个线性映射的参数被动态生成，这些参数被分解为一个动态（输入相关）组件和一个静态（输入无关）组件。给定输入 $X_l \in \mathbb{R}^{n_{\text{hc}} \times d}$ ，首先将其展平并归一化： $\hat{X}_l = \text{RMSNorm}(\text{vec}(X_l)) \in \mathbb{R}^{1 \times n_{\text{hc}}d}$ 。随后，我们遵循传统的HC方法来生成无约束的原始参数 $\tilde{A}_l \in \mathbb{R}^{1 \times n_{\text{hc}}}$ 、 $\tilde{B}_l \in \mathbb{R}^{n_{\text{hc}} \times n_{\text{hc}}}$ 和 $\tilde{C}_l \in \mathbb{R}^{n_{\text{hc}} \times 1}$ ：

$$\tilde{A}_l = \alpha_l^{\text{pre}} \cdot (\hat{X}_l W_l^{\text{pre}}) + S_l^{\text{pre}}, \quad (3)$$

$$\tilde{B}_l = \alpha_l^{\text{res}} \cdot \text{Mat}(\hat{X}_l W_l^{\text{res}}) + S_l^{\text{res}}, \quad (4)$$

$$\tilde{C}_l = \alpha_l^{\text{post}} \cdot (\hat{X}_l W_l^{\text{post}})^T + S_l^{\text{post}}, \quad (5)$$

其中 $W_l^{\text{pre}}, W_l^{\text{post}} \in \mathbb{R}^{n_{\text{hc}}d \times n_{\text{hc}}}$ 和 $W_l^{\text{res}} \in \mathbb{R}^{n_{\text{hc}}d \times n_{\text{hc}}^2}$ 是用于生成动态组件的可学习参数； $\text{Mat}(\cdot)$ 将大小为 $1 \times n_{\text{hc}}^2$ 的向量重塑为大小为 $n_{\text{hc}} \times n_{\text{hc}}$ 的矩阵； $S_l^{\text{pre}} \in \mathbb{R}^{1 \times n_{\text{hc}}}$ 、 $S_l^{\text{post}} \in \mathbb{R}^{n_{\text{hc}} \times 1}$ 和 $S_l^{\text{res}} \in \mathbb{R}^{n_{\text{hc}} \times n_{\text{hc}}}$ 是可学习的静态偏置；而 $\alpha_l^{\text{pre}}, \alpha_l^{\text{res}}$ 和 $\alpha_l^{\text{post}} \in \mathbb{R}$ 是初始化为较小值的可学习门控因子。

应用参数约束。在获得无约束的原始参数 $\tilde{A}_l, \tilde{B}_l, \tilde{C}_l$ 后，我们对它们应用前述约束以增强数值稳定性。具体来说，对于输入和输出映射，我们采用Sigmoid函数 $\sigma(\cdot)$ 来确保它们的非负性和有界性：

$$A_l = \sigma(\tilde{A}_l), \quad (6)$$

$$C_l = 2\sigma(\tilde{C}_l). \quad (7)$$

关于残差映射 $\tilde{B}_l$ ，我们将其投影到双随机矩阵 $\mathcal{M}$ 的流形上。这一过程通过Sinkhorn-Knopp算法实现，该算法首先对 $\tilde{B}_l$ 应用指数函数以确保正值性，得到 $M^{(0)} = \exp(\tilde{B}_l)$ ，随后迭代执行行与列归一化：

$$M^{(t)} = \mathcal{T}_r(\mathcal{T}_c(M^{(t-1)})), \quad (8)$$

其中 $\mathcal{T}_r$ 和 $\mathcal{T}_c$ 分别表示行归一化和列归一化。该迭代过程会收敛至一个受约束的双随机矩阵 $B_l = M^{(t_{\text{max}})}$ 。我们选择 $t_{\text{max}} = 20$ 作为实际应用值。

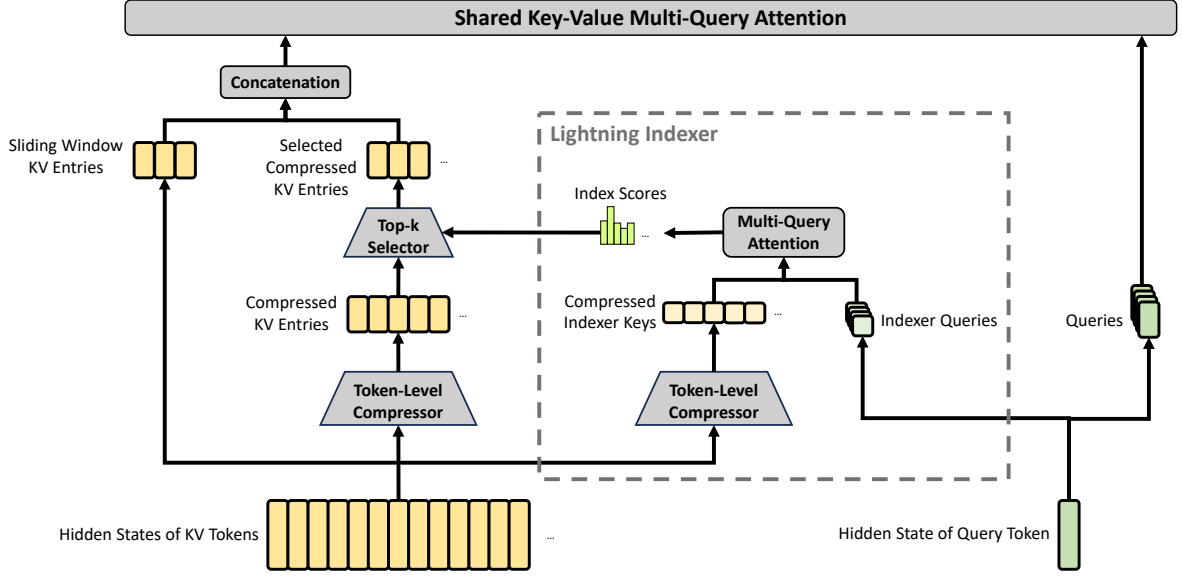


Figure 3 | Core architectures of CSA. It compresses the number of KV entries to $\frac{1}{m}$ times, and then applies DeepSeek Sparse Attention for further acceleration. Additionally, a small set of sliding window KV entries is combined with the selected compressed KV entries to enhance local fine-grained dependencies.

2.3. Hybrid Attention with CSA and HCA

As the context length reaches extreme scales, the attention mechanism emerges as the dominant computational bottleneck in a model. For DeepSeek-V4, we design two efficient attention architectures — Compressed Sparse Attention (CSA) and Heavily Compressed Attention (HCA) — and employ their interleaved hybrid configuration, which substantially reduces the computational cost of attention in long-text scenarios. CSA integrates both compression and sparse attention strategies: it first compresses the Key-Value (KV) cache of every m tokens into one entry, and then applies DeepSeek Sparse Attention (DSA) (DeepSeek-AI, 2025) where each query token attends to only k compressed KV entries. HCA aims for extreme compression by consolidating the KV cache of every m' ($\gg m$) tokens into a single entry. The hybrid architecture of CSA and HCA remarkably improves the long-context efficiency of DeepSeek-V4 series, making one-million-token context feasible in practice. This subsection describes the core techniques of our hybrid attention architecture, and we also provide an open-source implementation¹ to specify more details unambiguously.

2.3.1. Compressed Sparse Attention

The core architecture of CSA is illustrated in Figure 3, which first compresses the KV cache of each m tokens into one entry, and then applies DeepSeek Sparse Attention for further acceleration.

Compressed Key-Value Entries. Let $H \in \mathbb{R}^{n \times d}$ be a sequence of input hidden states, where n is the sequence length and d is the hidden size. CSA first computes two series of KV entries $C^a, C^b \in \mathbb{R}^{n \times c}$ and their corresponding compression weights $Z^a, Z^b \in \mathbb{R}^{n \times c}$, where c is the head

¹<https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/tree/main/inference>

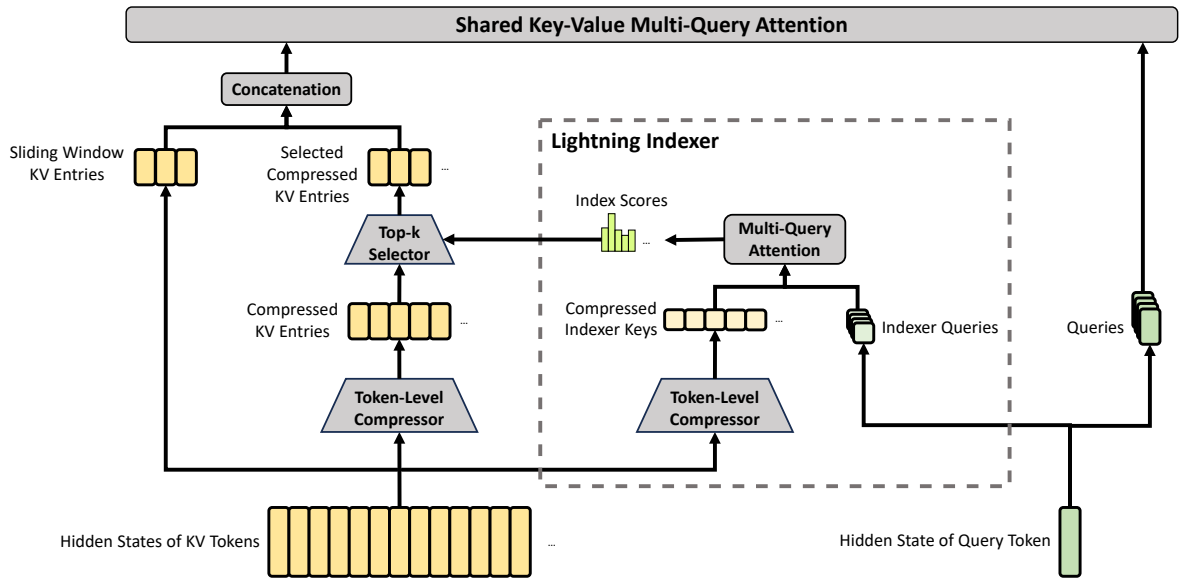


图3 | CSA的核心架构。它将KV条目数量压缩至 $\frac{1}{m}$ 倍，随后应用DeepSeek稀疏注意力机制实现进一步加速。此外，通过将少量滑动窗口KV条目与精选压缩KV条目相结合，增强了局部细粒度依赖关系。

2.3. 结合CSA与HCA的混合注意力机制

随着上下文长度达到极端规模，注意力机制成为模型中的主要计算瓶颈。对于DeepSeek-V4，我们设计了两种高效注意力架构——压缩稀疏注意力（CSA）和重度压缩注意力（HCA）——并采用它们的交错混合配置，这大幅降低了长文本场景下注意力的计算成本。CSA融合了压缩与稀疏注意力策略：首先将每 m 个token的键值（KV）缓存压缩为一个条目，然后应用DeepSeek稀疏注意力（DSA）（DeepSeek-AI, 2025），其中每个查询token仅关注 k 个压缩后的KV条目。HCA则追求极致压缩，将每 m' ($\gg m$)个token的KV缓存合并为单一条目。CSA与HCA的混合架构显著提升了DeepSeek-V4系列的长上下文效率，使百万级token的上下文处理在实践中成为可能。本小节阐述了我们混合注意力架构的核心技术，并提供了开源实现¹以明确更多细节。

2.3.1. Compressed Sparse Attention

CSA的核心架构如图3所示，其首先将每个 m 令牌的KV缓存压缩为一个条目，随后应用DeepSeek稀疏注意力机制以实现进一步加速。

压缩键值条目。设 $H \in \mathbb{R}^{n \times d}$ 为输入隐藏状态序列，其中 n 为序列长度， d 为隐藏大小。CSA首先计算两组KV条目 $C^a, C^b \in \mathbb{R}^{n \times c}$ 及其对应的压缩权重 $Z^a, Z^b \in \mathbb{R}^{n \times c}$ ，其中 c 表示头

¹<https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/tree/main/inference>

dimension:

$$C^a = H \cdot W^{aKV}, \quad C^b = H \cdot W^{bKV}, \quad (9)$$

$$Z^a = H \cdot W^{aZ}, \quad Z^b = H \cdot W^{bZ}, \quad (10)$$

where $W^{aKV}, W^{bKV}, W^{aZ}, W^{bZ} \in \mathbb{R}^{d \times c}$ are trainable parameters. Next, each m KV entries in C^a and C^b will be compressed into one entry according to their compression weights and learnable positional biases $B^a, B^b \in \mathbb{R}^{m \times c}$, producing $C^{\text{Comp}} \in \mathbb{R}^{\frac{n}{m} \times c}$. Each compressed entry $C_i^{\text{Comp}} \in \mathbb{R}^c$ is computed by

$$[S_{mi:m(i+1)-1}^a; S_{m(i-1):mi-1}^b] = \text{Softmax}_{\text{row}}([Z_{mi:m(i+1)-1}^a + B^a; Z_{m(i-1):mi-1}^b + B^b]), \quad (11)$$

$$C_i^{\text{Comp}} = \sum_{j=mi}^{m(i+1)-1} S_j^a \odot C_j^a + \sum_{j=m(i-1)}^{mi-1} S_j^b \odot C_j^b, \quad (12)$$

where $\odot$ denotes the Hadamard product; $\text{Softmax}_{\text{row}}(\cdot)$ denotes the softmax operation along the row dimension, which performs normalization across the total of $2m$ elements from both Z^a and Z^b . When $i = 0$, $Z_{m(i-1):mi-1}^b$ is padded with negative infinity and $C_{m(i-1):mi-1}^b$ is padded with zeros. Note that each C_i^{Comp} is derived from $2m$ KV entries, but the indexes of C^b used for C_i^{Comp} and the indexes of C^a used for C_{i-1}^{Comp} are overlapped. Therefore, CSA in fact compresses the sequence length to $\frac{1}{m}$ times.

Lightning Indexer for Sparse Selection. After obtaining the compressed KV entries C^{Comp} , CSA applies the DSA strategy to select top-k compressed KV entries for core attention. First, CSA performs the same compression operation used for C^{Comp} to get compressed indexer keys $K^{\text{IComp}} \in \mathbb{R}^{\frac{n}{m} \times c^I}$, where c^I is the indexer head dimension. Then, for a query token t , we produce the indexer queries $\{\mathbf{q}_{t,1}^I; \mathbf{q}_{t,2}^I; \dots; \mathbf{q}_{t,n_h^I}^I\}$ in a low-rank manner:

$$\mathbf{c}_t^Q = \mathbf{h}_t \cdot W^{DQ}, \quad (13)$$

$$[\mathbf{q}_{t,1}^I; \mathbf{q}_{t,2}^I; \dots; \mathbf{q}_{t,n_h^I}^I] = \mathbf{q}_t^I = \mathbf{c}_t^Q \cdot W^{IUQ}, \quad (14)$$

where $\mathbf{h}_t \in \mathbb{R}^d$ is the input hidden state of the query token t ; $\mathbf{c}_t^Q \in \mathbb{R}^{d_c}$ is the compressed latent vector for queries; d_c denotes the query compression dimension; n_h^I denotes the number of indexer query heads; $W^{DQ} \in \mathbb{R}^{d \times d_c}$ and $W^{IUQ} \in \mathbb{R}^{d_c \times c^I n_h^I}$ are the down-projection and up-projection matrices for indexer queries, respectively. Next, the index score $I_{t,s} \in \mathbb{R}$ between the query token t and a preceding compressed block s ($s < \text{Floor}(\frac{t}{m})$) is computed by

$$[w_{t,1}^I; w_{t,2}^I; \dots; w_{t,n_h^I}^I] = \mathbf{w}_t^I = \mathbf{h}_t \cdot W^w, \quad (15)$$

$$I_{t,s} = \sum_{h=1}^{n_h^I} w_{t,h}^I \cdot \text{ReLU}(\mathbf{q}_{t,h}^I \cdot K_s^{\text{IComp}}), \quad (16)$$

where $W^w \in \mathbb{R}^{d \times n_h^I}$ is a learnable matrix; $w_{t,h}^I \in \mathbb{R}$ is the weight of the h -th indexer head. For a query token t , given its index scores $I_{t,:}$, we employ a top-k selector to selectively retain a subset of compressed KV entries C_t^{SprsComp} for subsequent core attention:

$$C_t^{\text{SprsComp}} = \{C_s^{\text{Comp}} \mid I_{t,s} \in \text{Top-k}(I_{t,:})\}. \quad (17)$$

维度：

$$C^a = H \cdot W^{aKV}, \quad C^b = H \cdot W^{bKV}, \quad (9)$$

$$Z^a = H \cdot W^{aZ}, \quad Z^b = H \cdot W^{bZ}, \quad (10)$$

其中 W^{aKV} 、 W^{bKV} 、 W^{aZ} 、 $W^{bZ} \in \mathbb{R}^{d \times c}$ 是可训练参数。接着， C^a 和 C^b 中的每个 m 个键值条目将根据其压缩权重及可学习的位置偏置 B^a 、 $B^b \in \mathbb{R}^{m \times c}$ 被压缩为一个条目，生成 $C^{\text{Comp}} \in \mathbb{R}^{\frac{n}{m} \times c}$ 。每个压缩后的条目 $C_i^{\text{Comp}} \in \mathbb{R}^c$ 通过以下方式计算得出：

$$[S_{mi:m(i+1)-1}^a; S_{m(i-1):mi-1}^b] = \text{Softmax}_{\text{row}}([Z_{mi:m(i+1)-1}^a + B^a; Z_{m(i-1):mi-1}^b + B^b]), \quad (11)$$

$$C_i^{\text{Comp}} = \sum_{j=mi}^{m(i+1)-1} S_j^a \odot C_j^a + \sum_{j=m(i-1)}^{mi-1} S_j^b \odot C_j^b, \quad (12)$$

其中 $\odot$ 表示Hadamard积； $\text{Softmax}_{\text{row}}(\cdot)$ 表示沿行维度的softmax运算，该运算对来自 Z^a 和 Z^b 的总共 $2m$ 个元素进行归一化。当 $i=0$ 时， $Z_{m(i-1):mi-1}^b$ 用负无穷填充， $C_{m(i-1):mi-1}^b$ 用零填充。需要注意的是，每个 C_i^{Comp} 由 $2m$ 个KV条目派生而来，但用于 C_i^{Comp} 的 C^b 索引和用于 C_{i-1}^{Comp} 的 C^a 索引存在重叠。因此，CSA实际上将序列长度压缩至 $\frac{1}{m}$ 倍。

稀疏选择的闪电索引器。在获得压缩后的键值条目 C^{Comp} 后，CSA应用DSA策略选取前 k 个压缩键值条目进行核心注意力计算。首先，CSA采用与 C^{Comp} 相同的压缩操作，得到压缩索引器键 $K^{\text{IComp}} \in \mathbb{R}^{\frac{n}{m} \times c^I}$ ，其中 c^I 为索引器头维度。接着，针对查询令牌 t ，我们生成索引器查询 $\{q_{t,1}^I; q_{t,2}^I; \dots; q_{t,n_h^I}^I\}$ 以低秩方式：

$$c_t^Q = \mathbf{h}_t \cdot W^{DQ}, \quad (13)$$

$$[q_{t,1}^I; q_{t,2}^I; \dots; q_{t,n_h^I}^I] = \mathbf{q}_t^I = c_t^Q \cdot W^{IUQ}, \quad (14)$$

其中， $\mathbf{h}_t \in \mathbb{R}^d$ 是查询标记 t 的输入隐藏状态； $c_t^Q \in \mathbb{R}^{d_c}$ 是查询的压缩潜在向量； d_c 表示查询压缩维度； n_h^I 表示索引器查询头的数量； $W^{DQ} \in \mathbb{R}^{d \times d_c}$ 和 $W^{IUQ} \in \mathbb{R}^{d_c \times c^I n_h^I}$ 分别为索引器查询的下投影和上投影矩阵。接下来，查询标记 t 与前一个压缩块 s ($s < \text{Floor}(\frac{t}{m})$)之间的索引分数 $I_{t,s} \in \mathbb{R}$ 通过以下方式计算：

$$[w_{t,1}^I; w_{t,2}^I; \dots; w_{t,n_h^I}^I] = \mathbf{w}_t^I = \mathbf{h}_t \cdot W^w, \quad (15)$$

$$I_{t,s} = \sum_{h=1}^{n_h^I} w_{t,h}^I \cdot \text{ReLU}(\mathbf{q}_{t,h}^I \cdot K_s^{\text{IComp}}), \quad (16)$$

其中 $W^w \in \mathbb{R}^{d \times n_h^I}$ 是一个可学习的矩阵； $w_{t,h}^I \in \mathbb{R}$ 是第 h 个索引器头的权重。对于查询标记 t ，给定其索引分数 $I_{t,:}$ ，我们采用top-k选择器来选择性保留一部分压缩后的KV条目 C_t^{SprsComp} ，用于后续的核心注意力计算：

$$C_t^{\text{SprsComp}} = \left\{ C_s^{\text{Comp}} \mid I_{t,s} \in \text{Top-k}(I_{t,:}) \right\}. \quad (17)$$

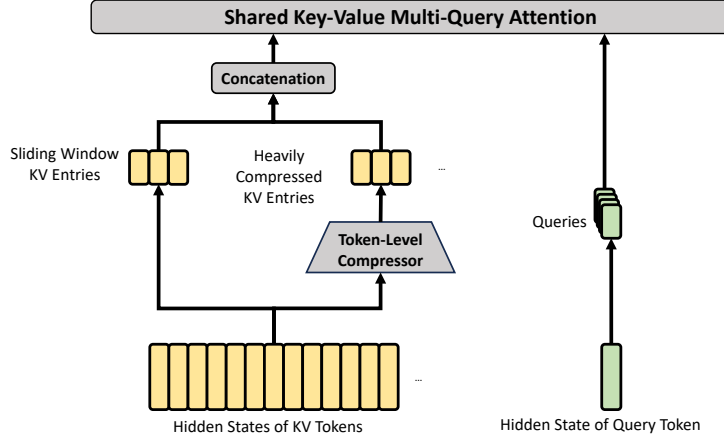


Figure 4 | Core architectures of HCA. It performs heavier compression, where the KV entries of $m' (\gg m)$ tokens will be consolidated into one. Also, we additionally introduce a small set of sliding window KV entries to enhance local fine-grained dependencies.

Shared Key-Value MQA. After selecting the sparse KV entries, CSA then performs core attention in a Multi-Query Attention (MQA) (Shazeer, 2019) manner, where each compressed KV entry in C_t^{SprsComp} serves as both attention key and value. To be specific, for a query token t , we first produce attention queries $\{\mathbf{q}_{t,1}; \mathbf{q}_{t,2}; \dots; \mathbf{q}_{t,n_h}\}$ from the compressed latent vector $\mathbf{c}_t^Q$:

$$[\mathbf{q}_{t,1}; \mathbf{q}_{t,2}; \dots; \mathbf{q}_{t,n_h}] = \mathbf{q}_t = \mathbf{c}_t^Q \cdot W^{UQ}, \quad (18)$$

where n_h denotes the number of query heads; $W^{UQ} \in \mathbb{R}^{d_c \times cn_h}$ is the up-projection matrices for queries. Note that the latent query vector $\mathbf{c}_t^Q$ is shared with that used for the indexer queries. Next, we perform MQA on $\{\mathbf{q}_{t,i}\}$ and C_t^{SprsComp} :

$$\mathbf{o}_{t,i} = \text{CoreAttn}(\text{query}=\mathbf{q}_{t,i}, \text{key}=C_t^{\text{SprsComp}}, \text{value}=C_t^{\text{SprsComp}}), \quad (19)$$

where $\mathbf{o}_{t,i} \in \mathbb{R}^c$ is the core attention output of the i -th head at the t -th token; $\text{CoreAttn}(\cdot)$ denotes the core attention operation.

Grouped Output Projection. In the configuration of DeepSeek-V4, cn_h is quite large. Therefore, directly projecting the outputs of the core attention operation $[\mathbf{o}_{t,1}; \mathbf{o}_{t,2}; \dots; \mathbf{o}_{t,n_h}] = \mathbf{o}_t \in \mathbb{R}^{cn_h}$ to a d -dimensional hidden state will impose a substantial computational burden. To mitigate this cost, we design a grouped output projection strategy. To be specific, we first split n_h outputs into g groups, and then for each group of output $\mathbf{o}_{t,i}^G \in \mathbb{R}^{c \frac{n_h}{g}}$, we project it to a d_g -dimensional intermediate output $\mathbf{o}_{t,i}^{G'} \in \mathbb{R}^{d_g}$, where $d_g < c \frac{n_h}{g}$. Finally, we project the intermediate output $[\mathbf{o}_{t,1}^{G'}; \mathbf{o}_{t,2}^{G'}; \dots; \mathbf{o}_{t,g}^{G'}] \in \mathbb{R}^{d_g g}$ to the final attention output $\hat{\mathbf{o}}_t \in \mathbb{R}^d$.

2.3.2. Heavily Compressed Attention

The core architecture of HCA is illustrated in Figure 4, which compresses the KV cache in a heavier manner, but does not employ sparse attention.

Compressed Key-Value Entries. By and large, the compression strategy of HCA is similar to that of CSA, but employs a larger compression rate $m' (\gg m)$ and does not perform overlapped

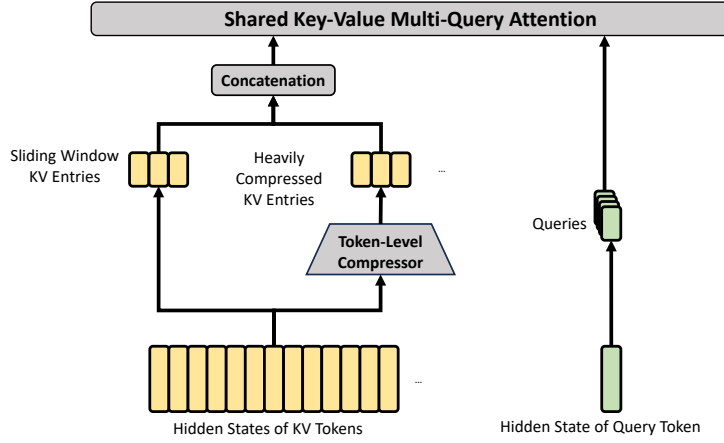


图4 | HCA的核心架构。它执行更重的压缩，其中 m' ($\gg m$)个令牌的KV条目将被合并为一个。此外，我们还额外引入了一小部分滑动窗口KV条目，以增强局部细粒度依赖关系。

共享键值多查询注意力（MQA）。在选定稀疏键值条目后，CSA采用多查询注意力机制（MQA）（Shazeer, 2019）执行核心注意力计算，其中压缩键值集合 C_t^{SprsComp} 中的每个条目同时作为注意力键和值。具体而言，对于查询标记 t ，我们首先从压缩隐向量 c_t^Q 生成注意力查询序列 $\{q_{t,1}; q_{t,2}; \dots; q_{t,n_h}\}$ ：

$$[q_{t,1}; q_{t,2}; \dots; q_{t,n_h}] = \mathbf{q}_t = \mathbf{c}_t^Q \cdot \mathbf{W}^{UQ}, \quad (18)$$

其中 n_h 表示查询头的数量； $\mathbf{W}^{UQ} \in \mathbb{R}^{d_c \times cn_h}$ 是查询的上投影矩阵。请注意，潜在查询向量 $\mathbf{c}_t^Q$ 与索引器查询中使用的向量是共享的。接着，我们在 $\{q_{t,i}\}$ 和 C_t^{SprsComp} 上执行MQA操作：

$$\mathbf{o}_{t,i} = \text{CoreAttn}(\text{query}=\mathbf{q}_{t,i}, \text{key}=\mathbf{C}_t^{\text{SprsComp}}, \text{value}=\mathbf{C}_t^{\text{SprsComp}}), \quad (19)$$

其中 $\mathbf{o}_{t,i} \in \mathbb{R}^c$ 是第 i 个头在第 t 个标记处的核心注意力输出； $\text{CoreAttn}(\cdot)$ 表示核心注意力操作。

分组输出投影。在DeepSeek-V4的配置中， cn_h 非常大。因此，直接将核心注意力操作的输出 $[\mathbf{o}_{t,1}; \mathbf{o}_{t,2}; \dots; \mathbf{o}_{t,n_h}] = \mathbf{o}_t \in \mathbb{R}^{cn_h}$ 投影到 d 维隐藏状态会带来巨大的计算负担。为降低这一成本，我们设计了一种分组输出投影策略。具体而言，我们首先将 n_h 个输出划分为 g 个组，然后对每组输出 $\mathbf{o}_{t,i}^G \in \mathbb{R}^{c \frac{n_h}{g}}$ ，将其投影至 d_g 维中间输出 $\mathbf{o}_{t,i}^{G'} \in \mathbb{R}^{d_g}$ ，其中 $d_g < c \frac{n_h}{g}$ 。最后，我们将中间输出 $[\mathbf{o}_{t,1}^{G'}; \mathbf{o}_{t,2}^{G'}; \dots; \mathbf{o}_{t,g}^{G'}] \in \mathbb{R}^{d_g g}$ 投影至最终的注意力输出 $\hat{\mathbf{o}}_t \in \mathbb{R}^d$ 。

2.3.2. Heavily Compressed Attention

HCA的核心架构如图4所示，其以更重的力度压缩KV缓存，但并未采用稀疏注意力机制。

压缩键值条目。总体而言，HCA的压缩策略与CSA类似，但采用了更大的压缩率 m' ($\gg m$)且不执行重叠

compression. Let $H \in \mathbb{R}^{n \times d}$ be a sequence of input hidden states, HCA first computes the original KV entries $C \in \mathbb{R}^{n \times c}$ and their corresponding compression weights $Z \in \mathbb{R}^{n \times c}$:

$$C = H \cdot W^{KV}, \quad (20)$$

$$Z = H \cdot W^Z, \quad (21)$$

where $W^{KV}, W^Z \in \mathbb{R}^{d \times c}$ are trainable parameters. Next, each m' KV entries in C will be compressed into one according to the compression weights and learnable positional biases $B \in \mathbb{R}^{m' \times c}$, producing $C^{\text{Comp}} \in \mathbb{R}^{\frac{n}{m'} \times c}$. Each compressed entry $C_i^{\text{Comp}} \in \mathbb{R}^c$ is computed by

$$S_{m'i:m'(i+1)-1} = \text{Softmax}_{\text{row}}(Z_{m'i:m'(i+1)-1} + B), \quad (22)$$

$$C_i^{\text{Comp}} = \sum_{j=m'i}^{m'(i+1)-1} S_j \odot C_j. \quad (23)$$

Through this compression operation, HCA compresses the sequence length to $\frac{1}{m'}$ times.

Shared Key-Value MQA and Grouped Output Projection. HCA also employs the shared KV MQA and grouped output projection strategies as CSA does. After the KV compression, for a query token t , HCA first produces attention queries $\{\mathbf{q}_{t,1}; \mathbf{q}_{t,2}; \dots; \mathbf{q}_{t,n_h}\}$ in a low-rank manner:

$$\mathbf{c}_t^Q = \mathbf{h}_t \cdot W^{DQ}, \quad (24)$$

$$[\mathbf{q}_{t,1}; \mathbf{q}_{t,2}; \dots; \mathbf{q}_{t,n_h}] = \mathbf{c}_t^Q \cdot W^{UQ}, \quad (25)$$

where $\mathbf{h}_t \in \mathbb{R}^d$ is the input hidden state of the query token t ; n_h denotes the number of query heads; $W^{DQ} \in \mathbb{R}^{d \times d_c}$ and $W^{UQ} \in \mathbb{R}^{d_c \times c n_h}$ are the down-projection and up-projection matrices for queries, respectively. Next, we perform MQA on $\{\mathbf{q}_{t,i}\}$ and C^{Comp} :

$$\mathbf{o}_{t,i} = \text{CoreAttn}(\text{query}=\mathbf{q}_{t,i}, \text{key}=C^{\text{Comp}}, \text{value}=C^{\text{Comp}}), \quad (26)$$

where $\mathbf{o}_{t,i} \in \mathbb{R}^c$ is the core attention output of the i -th head at the t -th token. Next, as CSA does, HCA splits n_h outputs into g groups, and for each group of output $\mathbf{o}_{t,i}^G \in \mathbb{R}^{c \frac{n_h}{g}}$, HCA projects it to a d_g -dimensional intermediate output $\mathbf{o}_{t,i}^{G'} \in \mathbb{R}^{d_g}$, where $d_g < c \frac{n_h}{g}$. Finally, HCA projects the intermediate output $[\mathbf{o}_{t,1}^{G'}; \mathbf{o}_{t,2}^{G'}; \dots; \mathbf{o}_{t,g}^{G'}] \in \mathbb{R}^{d_g g}$ to the final attention output $\hat{\mathbf{o}}_t \in \mathbb{R}^d$.

2.3.3. Other Details

In addition to the core architectures of CSA and HCA described above, our hybrid attention incorporates several other techniques. For writing clarity, we omit these additional techniques from the above introduction and will briefly describe them in this subsection. Also, this subsection focuses only on the core ideas of them and may omit some tiny details for simplicity. We encourage the readers to refer to our open-source implementation for unambiguous details.

Query and Key-Value Entry Normalization. For both CSA and HCA, we perform an additional RMSNorm operation on each head of the queries and the only head of the compressed KV entries, just before the core attention operation. This normalization avoids exploding attention logits and may improve training stability.

压缩。令 $H \in \mathbb{R}^{n \times d}$ 为输入隐藏状态序列，HCA首先计算原始KV条目 $C \in \mathbb{R}^{n \times c}$ 及其对应的压缩权重 $Z \in \mathbb{R}^{n \times c}$:

$$C = H \cdot W^{KV}, \quad (20)$$

$$Z = H \cdot W^Z, \quad (21)$$

其中 W^{KV} 、 $W^Z \in \mathbb{R}^{d \times c}$ 是可训练参数。接下来， C 中的每个 m' 键值对将根据压缩权重和可学习的位置偏置 $B \in \mathbb{R}^{m' \times c}$ 被压缩为一个条目，生成 $C^{\text{Comp}} \in \mathbb{R}^{\frac{n}{m'} \times c}$ 。每个压缩后的条目 $C_i^{\text{Comp}} \in \mathbb{R}^c$ 的计算方式为

$$S_{m':m'(i+1)-1} = \text{Softmax}_{\text{row}}(Z_{m':m'(i+1)-1} + B), \quad (22)$$

$$C_i^{\text{Comp}} = \sum_{j=m'i}^{m'(i+1)-1} S_j \odot C_j. \quad (23)$$

通过这一压缩操作，HCA将序列长度压缩至 $\frac{1}{m'}$ 倍。

共享键值多头注意力与分组输出投影。HCA同样采用了与CSA相同的共享KV多头注意力及分组输出投影策略。经过KV压缩后，对于查询标记 t ，HCA首先以低秩方式生成注意力查询 $\{q_{t,1}; q_{t,2}; \dots; q_{t,n_h}\}$:

$$\mathbf{c}_t^Q = \mathbf{h}_t \cdot W^{DQ}, \quad (24)$$

$$[q_{t,1}; q_{t,2}; \dots; q_{t,n_h}] = \mathbf{q}_t = \mathbf{c}_t^Q \cdot W^{UQ}, \quad (25)$$

其中 $\mathbf{h}_t \in \mathbb{R}^d$ 是查询令牌 t 的输入隐藏状态； n_h 表示查询头的数量； $W^{DQ} \in \mathbb{R}^{d \times d_c}$ 和 $W^{UQ} \in \mathbb{R}^{d_c \times cn_h}$ 分别是查询的下投影和上投影矩阵。接下来，我们对 $\{q_{t,i}\}$ 和 C^{Comp} 执行MQA:

$$\mathbf{o}_{t,i} = \text{CoreAttn}(\text{query}=\mathbf{q}_{t,i}, \text{key}=C^{\text{Comp}}, \text{value}=C^{\text{Comp}}), \quad (26)$$

其中 $\mathbf{o}_{t,i} \in \mathbb{R}^c$ 是第 i 个头在第 t 个令牌处的核心注意力输出。接着，与CSA相同，HCA将 n_h 个输出划分为 g 组，对于每组输出 $\mathbf{o}_{t,i}^G \in \mathbb{R}^{c \frac{n_h}{g}}$ ，HCA将其投影为一个 d_g 维的中间输出 $\mathbf{o}_{t,i}^{G'} \in \mathbb{R}^{d_g}$ ，其中 $d_g < c \frac{n_h}{g}$ 。最后，HCA将中间输出 $[\mathbf{o}_{t,1}^{G'}; \mathbf{o}_{t,2}^{G'}; \dots; \mathbf{o}_{t,g}^{G'}] \in \mathbb{R}^{d_g g}$ 投影为最终的注意力输出 $\hat{\mathbf{o}}_t \in \mathbb{R}^d$ 。

2.3.3. Other Details

除了上述CSA和HCA的核心架构外，我们的混合注意力机制还融合了其他多项技术。为保持行文简洁，上文未提及这些附加技术，本节将对其进行简要说明。同时，为简化表述，本小节仅聚焦于其核心思想，可能省略部分细枝末节。建议读者参考我们的开源实现以获取确切的细节信息。

查询与键值条目归一化。对于CSA和HCA，我们在核心注意力操作之前，对每个查询头及压缩KV条目的唯一头执行额外的RMSNORM操作。这种归一化处理避免了注意力对数爆炸，并可能提升训练稳定性。

Partial Rotary Positional Embedding. For both CSA and HCA, we partially employ the Rotary Positional Embedding (RoPE) (Su et al., 2024) to the attention queries, KV entries, and the core attention outputs. To be specific, for each query vector and KV entry vector used in CSA and HCA, we apply RoPE to its last 64 dimensions. Since the KV entries serve as both attention keys and values, the naive core attention outputs $\{\mathbf{o}_{t,i}\}$ will carry absolute position embeddings, derived from the weighted sum of KV entries. As a countermeasure, we also apply RoPE with position $-i$ on the last 64 dimensions of each $\mathbf{o}_{t,i}$. In this way, the output of the core attention will also carry relative position embeddings — the contribution of each KV entry to the core attention outputs will also be related to the distance between the query and the KV entry.

Additional Branch of Sliding Window Attention. In order to strictly preserve causality in CSA and HCA, each query attends to only preceding compressed KV blocks. Consequently, a query cannot access information from other tokens within its own compressed block. Meanwhile, recent tokens usually possess greater relevance to the query token in language modeling. For these reasons, we introduce a supplementary attention branch to both CSA and HCA in a sliding window manner, for better modeling of local dependencies. To be specific, for each query token, we additionally produce n_{win} uncompressed KV entries corresponding to the recent n_{win} tokens. In the core attention of CSA and HCA, these KV entries in the sliding window will be used along with the compressed KV entries.

Attention Sink. In the core attention of CSA and HCA, we employ the trick of attention sink (OpenAI, 2025; Xiao et al., 2024). To be specific, we set a series of learnable sink logits $\{z'_1, z'_2, \dots, z'_{n_h}\}$. For the h -th attention head, $\text{Exp}(z'_h)$ will be added to the denominator of the attention score:

$$s_{h,i,j} = \frac{\text{Exp}(z_{h,i,j})}{\sum_k \text{Exp}(z_{h,i,k}) + \text{Exp}(z'_h)}, \quad (27)$$

where $s_{h,i,j}, z_{h,i,j} \in \mathbb{R}$ denote the attention score and attention logit of the h -th attention head between the i -th query token and the j -th preceding token or compressed block. This technique allows each query head to adjust its total attention scores to be not equal to 1, and even to be near 0.

2.3.4. Efficiency Discussion

Due to the employment of hybrid CSA and HCA, together with low-precision computation and storage, the attention module of DeepSeek-V4 series achieves remarkable efficiency in both attention FLOPs and KV cache size, especially in long-context scenarios. First, we adopt a mixed storage format for KV entries: BF16 precision is used for the rotary positional embedding (RoPE) dimensions, while FP8 precision is applied to the remaining dimensions. This hybrid representation reduces the KV cache size by nearly half compared with pure BF16 storage. Second, attention computation within the lightning indexer is performed in FP4 precision, which accelerates the attention operation under extremely long contexts. Third, relative to DeepSeek-V3.2, a smaller attention top-k is chosen in DeepSeek-V4 series, thereby improving model efficiency on short- and medium-length texts. Finally, and most importantly, compressed attention and hybrid attention techniques substantially reduce both the KV cache size and the computational FLOPs.

Taking BF16 GQA8 (Ainslie et al., 2023) with a head dimension of 128 as the baseline — one of the common configurations of LLM attention — the KV cache size of DeepSeek-V4 series can be dramatically reduced to approximately 2% times of that baseline in the 1M-context setting.

部分旋转位置嵌入。对于CSA和HCA，我们部分采用了旋转位置嵌入（RoPE）（Su等人，2024）来处理注意力查询、KV条目及核心注意力输出。具体而言，针对CSA和HCA中使用的每个查询向量和KV条目向量，我们对其最后64个维度应用RoPE。由于KV条目同时充当注意力键和值，原始核心注意力输出 $\{o_{t,i}\}$ 会携带绝对位置嵌入——这些嵌入源自KV条目的加权求和。作为应对措施，我们还在每个 $o_{t,i}$ 的最后64个维度上应用了位置 $-i$ 的RoPE。通过这种方式，核心注意力的输出也将携带相对位置嵌入——每个KV条目对核心注意力输出的贡献还将与查询和KV条目之间的距离相关。

滑动窗口注意力的附加分支。为了在CSA和HCA中严格保持因果性，每个查询仅关注先前的压缩KV块。因此，查询无法访问其自身压缩块内的其他令牌信息。同时，在语言建模中，近期令牌通常与查询令牌具有更高的相关性。基于这些原因，我们以滑动窗口的方式为CSA和HCA引入了一个补充注意分支，以更好地建模局部依赖关系。具体而言，对于每个查询令牌，我们会额外生成 n_{win} 个未压缩的KV条目，对应最近的 n_{win} 个令牌。在CSA和HCA的核心注意力机制中，这些滑动窗口内的KV条目将与压缩后的KV条目一同使用。

注意力汇点。在CSA和HCA的核心注意力机制中，我们采用了注意力汇点技巧（OpenAI, 2025; Xiao et al., 2024）。具体而言，我们设置了一系列可学习的汇点对数 $\{z'_1, z'_2, \dots, z'_{n_h}\}$ 。对于第 h 个注意力头， $\text{Exp}(z'_h)$ 将被添加到注意力分数的分母中：

$$s_{h,i,j} = \frac{\text{Exp}(z_{h,i,j})}{\sum_k \text{Exp}(z_{h,i,k}) + \text{Exp}(z'_h)}, \quad (27)$$

其中 $s_{h,i,j}, z_{h,i,j} \in \mathbb{R}$ 表示第 h 个注意力头在第 i 个查询标记与第 j 个前驱标记或压缩块之间的注意力分数与注意力对数。该技术使每个查询头能调整其总注意力分数，使其不必等于1，甚至可以接近0。

2.3.4. Efficiency Discussion

由于采用了混合CSA和HCA技术，结合低精度计算与存储，DeepSeek-V4系列的注意力模块在注意力FLOPs和KV缓存大小方面均实现了显著效率提升，尤其在长上下文场景中。首先，我们对KV条目采用混合存储格式：旋转位置编码（RoPE）维度使用BF16精度，其余维度则采用FP8精度。相较于纯BF16存储，这种混合表示使KV缓存大小减少近半。其次，闪电索引器内部的注意力计算以FP4精度执行，从而在超长上下文下加速注意力运算。第三，相对于DeepSeek-V3.2，V4系列选用更小的注意力top-k值，从而提升模型在短中文本文上的效率。最后且最关键的是，压缩注意力与混合注意力技术大幅降低了KV缓存体积与计算FLOPs。

以BF16 GQA8（Ainslie等人，2023）作为基准——这是LLM注意力机制的常见配置之一，其头维度为128——在100万上下文长度的设定下，DeepSeek-V4系列的KV缓存大小可大幅缩减至该基准的约2%倍。

Algorithm 1 Muon Optimizer for DeepSeek-V4

Require: Learning rate η , momentum μ , weight decay λ , update rescaling factor γ

```
1: for each training step  $t$  do
2:   for each logically independent weight  $W \in \mathbb{R}^{n \times m}$  do
3:      $G_t = \nabla_W \mathcal{L}_t(W_{t-1})$  ▷ Compute gradients
4:      $M_t = \mu M_{t-1} + G_t$  ▷ Accumulate momentum buffer
5:      $O'_t = \text{HybridNewtonSchulz}(\mu M_t + G_t)$  ▷ Nesterov trick and hybrid Newton-Schulz
6:      $O_t = O'_t \cdot \sqrt{\max(n, m)} \cdot \gamma$  ▷ Rescale the update RMS
7:      $W_t = W_{t-1} \cdot (1 - \eta\lambda) - \eta O_t$  ▷ Perform weight decay and update
8:   end for
9: end for
```

Moreover, even when compared with DeepSeek-V3.2 (DeepSeek-AI, 2025) — already an efficient baseline — DeepSeek-V4 series still exhibits substantial advantages in efficiency. A comparison of their inference FLOPs and KV cache size is provided in the right part of Figure 1.

2.4. Muon Optimizer

We employ the Muon (Jordan et al., 2024; Liu et al., 2025) optimizer for the majority of modules in DeepSeek-V4 series due to its faster convergence and improved training stability. The full algorithm of our Muon optimization is summarized in Algorithm 1.

Basic Configurations. We maintain the AdamW (Loshchilov and Hutter, 2017) optimizer for the embedding module, the prediction head module, the static biases and gating factors of m HC modules, and the weights of all RMSNorm modules. All other modules are updated with Muon. Following Liu et al. (2025), we also apply weight decay to Muon parameters, use the Nesterov (Jordan et al., 2024; Nesterov, 1983) trick, and rescale the Root Mean Square (RMS) of the update matrix for reutilization of our AdamW hyper-parameters. Different from them, we use hybrid Newton-Schulz iterations for orthogonalization.

Hybrid Newton-Schulz Iterations. For a given matrix M , let its Singular Value Decomposition (SVD) be $M = U\Sigma V^T$. The Newton-Schulz iterations aim to approximately orthogonalize M to be UV^T . Usually, M will be first normalized as $M_0 = M / \|M\|_F$ to ensure its maximum singular value does not exceed 1. Then, each Newton-Schulz iteration performs the following operation:

$$M_k = aM_{k-1} + b(M_{k-1}M_{k-1}^T)M_{k-1} + c(M_{k-1}M_{k-1}^T)^2M_{k-1}. \quad (28)$$

Our hybrid Newton-Schulz performs 10 iterations over two distinct stages. During the first 8 steps, we use coefficients $(a, b, c) = (3.4445, -4.7750, 2.0315)$ to drive rapid convergence, bringing the singular values close to 1. In the final 2 steps, we switch to coefficients $(a, b, c) = (2, -1.5, 0.5)$, which stabilize the singular values precisely at 1.

Avoiding Exploding Attention Logits. The attention architecture of DeepSeek-V4 series allows us to directly apply RMSNorm on the attention queries and KV entries, which effectively prevents attention logits from exploding. Consequently, we do not employ the QK-Clip technique (Liu et al., 2025) in our Muon optimizer.

Algorithm 1 Muon Optimizer for DeepSeek-V4

Require: Learning rate η , momentum μ , weight decay λ , update rescaling factor γ

```
1: for each training step  $t$  do
2:   for each logically independent weight  $W \in \mathbb{R}^{n \times m}$  do
3:      $G_t = \nabla_W \mathcal{L}_t(W_{t-1})$  ▷ Compute gradients
4:      $M_t = \mu M_{t-1} + G_t$  ▷ Accumulate momentum buffer
5:      $O'_t = \text{HybridNewtonSchulz}(\mu M_t + G_t)$  ▷ Nesterov trick and hybrid Newton-Schulz
6:      $O_t = O'_t \cdot \sqrt{\max(n, m)} \cdot \gamma$  ▷ Rescale the update RMS
7:      $W_t = W_{t-1} \cdot (1 - \eta\lambda) - \eta O_t$  ▷ Perform weight decay and update
8:   end for
9: end for
```

此外，即便与已经相当高效的基线模型DeepSeek-V3.2（DeepSeek-AI, 2025）相比，DeepSeek-V4系列在效率上仍展现出显著优势。图1右侧部分提供了两者推理FLOPs及KV缓存大小的对比情况。

2.4. μ 子优化器

我们采用Muon（Jordan等人，2024；Liu等人，2025）优化器来处理DeepSeek-V4系列中的大多数模块，因其更快的收敛速度和提升的训练稳定性。完整的Muon优化算法总结于算法1中。

基础配置。我们为嵌入模块、预测头模块、 m HC模块的静态偏置与门控因子，以及所有RMSNorm模块的权重保持使用AdamW（Loshchilov和Hutter, 2017）优化器。其余模块均采用Muon进行更新。遵循Liu等人（2025）的方法，我们同样对Muon参数施加权重衰减，运用Nesterov（Jordan等人，2024；Nesterov, 1983）技巧，并重新调整更新矩阵的均方根（RMS）以实现AdamW超参数的复用。与他们的方法不同，我们采用混合Newton-Schulz迭代进行正交化处理。

混合牛顿-舒尔茨迭代。对于给定矩阵 M ，设其奇异值分解(SVD)为 $M = U\Sigma V^T$ 。牛顿-舒尔茨迭代旨在将 M 近似正交化为 UV^T 。通常， M 会首先被归一化为 $M_0 = M/\|M\|_F$ ，以确保其最大奇异值不超过1。随后，每次牛顿-舒尔茨迭代执行以下操作：

$$M_k = aM_{k-1} + b(M_{k-1}M_{k-1}^T)M_{k-1} + c(M_{k-1}M_{k-1}^T)^2M_{k-1}. \quad (28)$$

我们的混合牛顿-舒尔茨方法在两个不同阶段共执行10次迭代。在前8步中，我们采用系数 $(a, b, c) = (3.4445, -4.7750, 2.0315)$ 来驱动快速收敛，使奇异值逼近1。在最后2步中，我们切换至系数 $(a, b, c) = (2, -1.5, 0.5)$ ，将奇异值精确稳定在1。

避免注意力对数爆炸。DeepSeek-V4系列的注意力架构使我们能够在注意力查询和键值条目上应用RMSNorm，有效防止注意力对数爆炸。因此，我们在Muon优化器中未采用QK-Clip技术（Liu等人，2025）。

3. General Infrastructures

3.1. Fine-Grained Communication-Computation Overlap in Expert Parallelism

Mixture-of-Experts (MoE) can be accelerated via Expert Parallelism (EP). However, EP requires complex inter-node communication and imposes substantial demands on interconnect bandwidth and latency. To alleviate the communication bottleneck in EP and achieve higher end-to-end performance under lower interconnection bandwidth requirements, we propose a fine-grained EP scheme that fuses communication and computation into a single pipelined kernel for communication-computation overlapping.

Communication Latency Can Be Hidden. The key insight of our EP scheme is that the communication latency can be effectively hidden beneath computation in MoE layers. As shown in Figure 5, in DeepSeek-V4 series, each MoE layer can be decomposed mainly into four stages: two communication-bound stages, Dispatch and Combine, and two computation-bound stages, Linear-1 and Linear-2. Our profiling reveals that within a single MoE layer, the total time of communication is less than that of the computation. Therefore, after fusing communication and computation into a unified pipeline, computation remains the dominant bottleneck, implying that the system can tolerate lower interconnect bandwidth without degrading end-to-end performance.

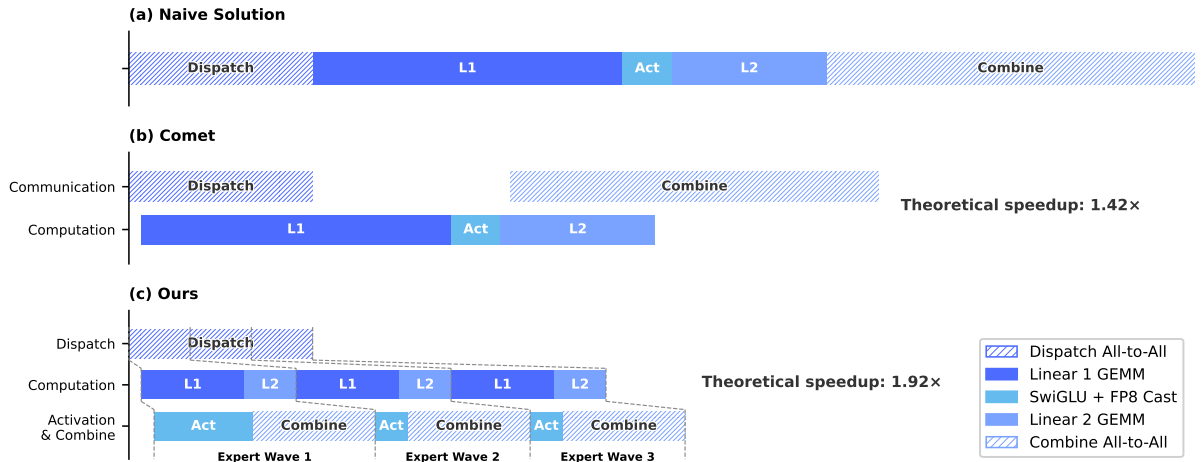


Figure 5 | Illustration of our EP scheme with related works. Comet (Zhang et al., 2025b) overlaps Dispatch with Linear-1, and Linear-2 with Combine, separately. Our EP scheme achieves a finer-grained overlapping by splitting and scheduling experts into waves. The theoretical speedup is evaluated in the configuration of the DeepSeek-V4-Flash architecture.

Fine-Grained EP Scheme. To further lower the interconnect bandwidth requirement and amplify the benefits of overlapping, we introduce a finer-grained expert partitioning scheme. Inspired by many related works (Aimuyo et al., 2025; Zhang et al., 2025b), we split and schedule the experts into waves. Each wave consists of a small portion of experts. As soon as all experts within the wave have completed their communication, computation can commence immediately without waiting for other experts. In steady state, computation of current wave, token transfer for the next wave, and result sending of completed experts all proceed concurrently, as demonstrated in Figure 5. This forms a fine-grained pipeline among experts, keeping both computation and communication continuous throughout the wave. The wave-based scheduling speeds up the

3. 通用基础设施

3.1. 专家并行中的细粒度通信-计算重叠

专家混合模型（MoE）可通过专家并行（EP）实现加速。然而，EP需要复杂的节点间通信，并对互连带宽和延迟提出了较高要求。为缓解EP中的通信瓶颈，在更低互连带宽需求下实现更高的端到端性能，我们提出了一种细粒度EP方案，该方案将通信与计算融合至单一流水线内核中，实现通信-计算重叠。

通信延迟可被隐藏。我们EP方案的核心洞见在于，MoE层中的通信延迟可被有效隐藏在计算之下。如图5所示，在DeepSeek-V4系列中，每个MoE层主要可分解为四个阶段：两个通信受限阶段（Dispatch与Combine）和两个计算受限阶段（Linear-1与Linear-2）。我们的性能分析表明，在单个MoE层内，通信总时长小于计算时长。因此，将通信与计算融合为单一流水线后，计算仍是主要瓶颈，这意味着系统可容忍更低的互连带宽而不影响端到端性能。

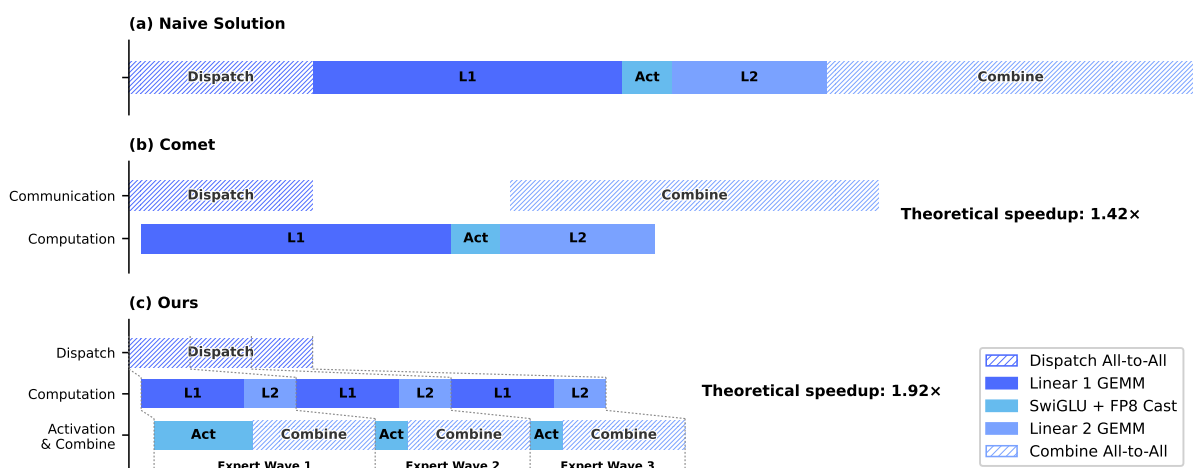


图5 | 我们EP方案与相关工作的对比示意图。Comet（Zhang等人，2025b）分别将Dispatch与Linear-1、Linear-2与Combine重叠执行。我们的EP方案通过将专家分组为波浪式调度，实现了更细粒度的重叠。理论加速效果基于DeepSeek-V4-Flash架构配置进行评估。

细粒度专家并行方案。为了进一步降低互连带宽需求并放大重叠计算的效益，我们引入了一种更细粒度的专家划分策略。受多项相关研究启发（Aimuyo等人，2025；Zhang等人，2025b），我们将专家分组为多个波次进行调度。每个波次包含少量专家子集，当波次内所有专家完成通信后，计算可立即启动而无需等待其他专家。如图5所示，在稳定状态下，当前波次的计算、下一波次的令牌传输以及已完成专家的结果发送三者并行推进，由此在专家间形成细粒度流水线，使计算与通信在整个波次中持续运转。基于波次的调度机制加速了...

performance on extreme cases such as Reinforcement Learning (RL) rollout, which usually encounters long-tail small batches.

Performance and Open-Sourced Mega-Kernel. We validated the fine-grained EP scheme on both NVIDIA GPUs and HUAWEI Ascend NPUs platforms. Compared against strong non-fused baselines, it achieves $1.50 \sim 1.73\times$ speedup for general inference workloads, and up to $1.96\times$ for latency-sensitive scenarios such as RL rollouts and high-speed agent serving. We have open-sourced the CUDA-based mega-kernel implementation named **MegaMoE²** as a component of DeepGEMM.

Observations and Proposals. We share observations and lessons from kernel development and offer some proposals to hardware vendors, in the hope of aiding efficient hardware design and achieving better software-hardware co-design:

- **Computation-Communication Ratio.** Full communication-computation overlap hinges on the computation-communication ratio, rather than the bandwidth solely. Denoting peak compute throughput as C and interconnect bandwidth as B , communication can be fully hidden when $C/B \leq V_{\text{comp}}/V_{\text{comm}}$, where V_{comp} denotes the computation volume and V_{comm} refers to the communication volume. For DeepSeek-V4-Pro, where each token-expert pair requires $6hd$ FLOPs (SwiGLU gate, up, and down projections) but only $3h$ bytes of communication (FP8 Dispatch + BF16 Combine), this simplifies to:

$$\frac{C}{B} \leq 2d = 6144 \text{ FLOPs/Byte.}$$

That is, each GBps of interconnect bandwidth suffices to hide the communication for 6.1 TFLOP/s of compute. Once bandwidth meets this threshold, it ceases to be the bottleneck, and devoting additional silicon area to further bandwidth brings diminishing returns. We encourage future hardware designs to target such balance points rather than scale bandwidth unconditionally.

- **Power Budget.** Extreme kernel fusion drives compute, memory, and network to high load simultaneously, making power throttling a key performance limiter. We suggest that future hardware designs provide sufficient power headroom for such fully concurrent workloads.
- **Communication Primitives.** In the dispatch stage, we adopt a pull-based approach where each GPU actively reads activations from remote GPUs, avoiding the high notification latency that fine-grained push entails. Future hardware with lower-latency cross-GPU signaling would make push viable and enable more natural communication patterns.
- **Activation Function.** We propose replacing SwiGLU with a low-cost element-wise activation that involves no exponential or division operations. This directly reduces the overhead of post-GEMM processing, preventing the GEMM pipeline from being stalled by activation function computation, thereby enhancing overall computational throughput and resource utilization.

3.2. Flexible and Efficient Kernel Development with TileLang

In practice, our elaborate model architecture would have resulted in hundreds of fine-grained Torch ATen operators. We adopt TileLang (Wang et al., 2026) to develop a set of fused kernels

²<https://github.com/deepseek-ai/DeepGEMM/pull/304>

在极端情况下的性能表现，如强化学习（RL）展开过程中通常会遇到的长尾小批量情况。

性能与开源Mega-Kernel。我们在NVIDIA GPU和华为昇腾NPU平台上验证了细粒度EP方案。与未融合的强劲基线相比，该方案在通用推理工作负载上实现了1.50~至1.73×的加速比，在强化学习推演和高速智能体服务等延迟敏感场景中，加速比最高可达1.96×。我们已将基于CUDA的超级内核实现MegaMoE²作为DeepGEMM的组成部分开源发布。

观察与建议。我们分享内核开发中的观察与经验教训，并向硬件供应商提出若干建议，以期助力高效硬件设计并实现更优的软硬件协同设计：

- 计算-通信比率。实现完全的计算-通信重叠不仅取决于带宽，更关键的是计算与通信的比率。将峰值计算吞吐量记为 C ，互连带宽记为 B ，当满足 $C/B \leq V_{\text{comp}}/V_{\text{comm}}$ 时通信可被完全隐藏，其中 V_{comp} 表示计算量， V_{comm} 指通信量。对于DeepSeek-V4-Pro而言，每个token-专家对需执行 $6hd$ 次浮点运算（SwiGLU门控、上行及下行投影），但仅产生 $3h$ 字节的通信量（FP8分发+ BF16合并），该条件简化为：

$$\frac{C}{B} \leq 2d = 6144 \text{ FLOPs/Byte.}$$

也就是说，每GBps的互连带宽足以掩盖6.1 TFLOP/s计算量带来的通信开销。一旦带宽达到这一阈值，它就不再是瓶颈，而将额外的芯片面积用于进一步提升带宽则会带来收益递减。我们鼓励未来的硬件设计瞄准此类平衡点，而非无条件地扩展带宽。

- 功耗预算。极端内核融合使得计算、内存和网络同时处于高负载状态，导致功耗调节成为关键的性能限制因素。我们建议未来的硬件设计为此类完全并发的的工作负载提供充足的功率余量。
- 通信原语。在调度阶段，我们采用基于拉取的方法，每个GPU主动从远程GPU读取激活值，避免了细粒度推送所伴随的高通知延迟。未来具备更低延迟跨GPU信号传输的硬件将使推送方案变得可行，并支持更自然的通信模式。
- 激活函数。我们提出用一种低成本逐元素激活函数替代SwiGLU，该函数不涉及指数或除法运算。这直接降低了GEMM后处理的开销，避免了激活函数计算阻塞GEMM流水线，从而提升整体计算吞吐量与资源利用率。

3.2. 使用TileLang进行灵活高效的核开发

在实践中，我们精心设计的模型架构本会产生数百个细粒度的Torch ATen算子。为此，我们采用TileLang（Wang等人，2026）开发了一套融合内核

²<https://github.com/deepseek-ai/DeepGEMM/pull/304>

to replace the vast majority of them, delivering optimal performance with minimal effort. It also allows us to quickly prototype operators like attention variants during validation. These kernels play critical roles in model architecture development, large-scale training, and ultimately production deployment of inference services. As a Domain-Specific Language (DSL), TileLang balances development productivity with runtime efficiency, enabling rapid development while supporting deep, iterative optimizations within the same codebase. Additionally, we collaborate closely with the TileLang community to foster a more agile, efficient, and stable kernel development workflow.

Reducing Invocation Overhead with Host Codegen. As accelerators continue to grow in performance, CPU-side orchestration overhead becomes increasingly prominent. For small, highly optimized kernels, such fixed host overhead can easily cap utilization and throughput. A common source of this overhead is that host-side logic, such as runtime contract checks, is typically written in Python for flexibility and thus incurs a fixed per-invocation cost.

We mitigate this overhead with *Host Codegen*, which moves most host-side logic into generated host code. Specifically, we first co-generate the device kernel and a lightweight host launcher at the IR (Intermediate Representation) level, embedding the necessary metadata—such as data types, rank/shape constraints, and stride/layout assumptions—parsed from the language frontend. The launcher is then lowered to the host source code built on top of the TVM-FFI (Chen et al., 2018) framework, whose compact calling convention and zero-copy tensor interop together minimize host-side overhead. At runtime, this generated host code performs validation and argument marshaling, shifting all per-invocation checks out of the Python execution path. Our measurements show that CPU-side validation overhead drops from tens or hundreds of microseconds to less than one microsecond per invocation.

SMT-Solver-Assisted Formal Integer Analysis. TileLang kernels involve complex tensor index arithmetic that requires strong formal integer analysis. During compilation passes such as layout inference, memory hazard detection, and bound analysis, the compiler must verify whether integer expressions satisfy specific properties to enable the corresponding optimizations. Therefore, stronger formal analysis capabilities can unlock more advanced and complex optimization opportunities.

To this end, we integrate the Z3 SMT solver (De Moura and Bjørner, 2008) into TileLang’s algebraic system, providing formal analysis capability for most integer expressions in tensor programs. We strike a balance between computational overhead and formal expressiveness by translating TileLang’s integer expressions into Z3’s quantifier-free non-linear integer arithmetic (QF_NIA). Based on Integer Linear Programming (ILP) solvers, QF_NIA seamlessly resolves standard linear integer expressions common in kernels. Furthermore, its inherent non-linear reasoning capacity effectively addresses advanced challenges like vectorization over variable tensor shapes. Under reasonable resource limits, Z3 elevates overall optimization performance while restricting compilation time overhead to just a few seconds. The impact is substantial across multiple passes, including vectorization, barrier insertion, and code simplification.

Numerical Precision and Bitwise Reproducibility. In production settings, numerical correctness and reproducibility are as critical as raw throughput. We therefore prioritize accuracy by default: fast-math optimizations are disabled at the compiler level, and precision-affecting approximations are provided only as explicit, opt-in frontend operators (e.g., `T.__exp`, `T.__log`, and `T.__sin`). Conversely, when strict IEEE-754 semantics are required, TileLang provides

以最小的努力替换其中的绝大多数，提供最优性能。它还使我们能够在验证过程中快速原型化注意力变体等算子。这些内核在模型架构开发、大规模训练以及最终的推理服务生产部署中扮演着关键角色。作为一种领域特定语言（DSL），TileLang在开发效率与运行时性能之间取得了平衡，既能支持快速开发，又能在同一代码库内实现深度迭代优化。此外，我们与TileLang社区紧密合作，以培育更加敏捷、高效且稳定的内核开发工作流程。

降低主机代码生成带来的调用开销。随着加速器性能持续提升，CPU端的协调开销日益凸显。对于经过高度优化的小型内核而言，这种固定的主机开销极易限制利用率和吞吐量。该开销的常见根源在于：出于灵活性考虑，主机端逻辑（如运行时合约检查）通常采用Python编写，从而导致每次调用都会产生固定成本。

我们通过*Host Codegen*减轻了这一开销，它将大部分主机端逻辑移入生成的宿主代码中。具体而言，我们首先在IR（中间表示）层级协同生成设备内核与轻量级主机启动器，并嵌入从语言前端解析的必要元数据——如数据类型、维度/形状约束以及步幅/布局假设。随后，该启动器被降级为基于TVM-FFI框架（Chen等人，2018）构建的宿主源代码，其紧凑的调用约定和零拷贝张量互操作性共同将主机端开销降至最低。运行时，这段生成的宿主代码执行验证和参数编组，将所有每次调用的检查移出Python执行路径。我们的测量显示，CPU端验证开销从每次调用数十或数百微秒降至不足一微秒。

SMT求解器辅助的形式化整数分析。TileLang内核涉及复杂的张量索引运算，这需要强大的形式化整数分析能力。在布局推断、内存风险检测以及边界分析等编译过程中，编译器必须验证整数表达式是否满足特定属性，以实现相应的优化。因此，更强大的形式化分析能力能够开启更先进、更复杂的优化机会。

为此，我们将Z3 SMT求解器（De Moura与Bjørner，2008）集成到TileLang的代数系统中，为张量程序中的大多数整数表达式提供形式化分析能力。通过将TileLang的整数表达式转换为Z3的无量词非线性整数算术（QF_NIA），我们在计算开销与形式表达能力之间取得了平衡。基于整数线性规划（ILP）求解器，QF_NIA无缝处理了内核中常见的标准线性整数表达式。此外，其固有的非线性推理能力有效应对了诸如可变张量形状向量化等高级挑战。在合理资源限制下，Z3提升了整体优化性能，同时将编译时间开销控制在数秒内。这一改进在向量化、屏障插入及代码简化等多个环节均产生了显著影响。

数值精度与位级可复现性。在生产环境中，数值正确性和可复现性与原始吞吐量同等重要。因此，我们默认优先保证准确性：编译器层面禁用快速数学优化，且仅通过显式的前端操作符（如`T.__exp`、`T.__log`和`T.__sin`）提供影响精度的近似计算。反之，当需要严格的IEEE-754语义时，TileLang可确保

IEEE-compliant intrinsics with explicit rounding modes (e.g., `T.ieee_fsqrt`, `T.ieee_fdiv`, and `T.ieee_add`), enabling developers to precisely specify numerical behavior.

We also target bitwise reproducibility for validating kernels against hand-written CUDA baselines. We align TileLang’s algebraic simplification and lowering rules with mainstream CUDA toolchains (e.g., NVCC) to avoid transformations that introduce unintended bit-level differences. Layout annotations (e.g., `T.annotate_layout`) further allow users to pin down layout-dependent lowering decisions, keeping evaluation and accumulation order consistent with the reference CUDA implementation and thus enabling bit-identical outputs when desired.

Our evaluation shows that these accuracy- and reproducibility-oriented design choices do not sacrifice performance: under conservative defaults, TileLang kernels remain competitive, while exposing knobs to selectively relax numerical constraints for higher speed.

3.3. High-Performance Batch-Invariant and Deterministic Kernel Libraries

To enable efficient training and inference, we develop a comprehensive set of high-performance computational kernels. Beyond basic functionalities and maximizing hardware utilization, another pivotal design goal is to ensure training reproducibility and bitwise alignment among pre-training, post-training, and inference pipelines. Therefore, we implement end-to-end, bitwise batch-invariant, and deterministic kernels with minimal performance overhead. These kernels are helpful for debugging, stability analysis, and consistent post-training behavior.

Batch Invariance. Batch invariance ensures that the output of any given token remains bitwise identical, regardless of its position within a batch. To implement batch invariance, the primary challenges are listed as follows:

- **Attention.** To achieve batch invariance, we cannot use the split-KV method (Dao et al., 2023), which distributes the attention computation for a single sequence across multiple Stream Multiprocessors (SMs) to balance the load of SMs. However, abandoning this technique will lead to severe wave-quantization problems³, which can adversely affect GPU utilization. To address this, we develop a dual-kernel strategy for batch-invariant decoding. The first kernel computes the attention output for an entire sequence within a single SM, ensuring high throughput for fully occupied waves. The second kernel, to minimize the latency of the final partially-filled wave and thus alleviate wave-quantization, uses multiple SMs for a single sequence. For the bitwise identity of these two kernels, we carefully design the calculation path of the second kernel to ensure its accumulation order is the same as that of the first kernel. Additionally, the second kernel utilizes distributed shared memory⁴ within thread-block clusters, enabling high-speed data exchange across SMs. This dual-kernel method effectively confines the overhead of batch-invariant decoding to be negligible.
- **Matrix Multiplication.** Traditional cuBLAS library (NVIDIA Corporation, 2024) cannot achieve batch invariance. Therefore, we replace it end-to-end with DeepGEMM (Zhao et al., 2025). Furthermore, for very small batch sizes, conventional implementation usually employs split-k (Osama et al., 2023) techniques to improve performance. Unfortunately, split-k techniques cannot guarantee batch invariance, a pivotal feature in DeepSeek-V4.

³<https://docs.nvidia.com/deeplearning/performance/dl-performance-matrix-multiplication/index.html#wave-quant>

⁴<https://docs.nvidia.com/cuda/cuda-programming-guide/02-basics/writing-cuda-kernels.html#distributed-shared-memory>

符合IEEE标准的显式舍入模式内置函数（例如`T.ieee_fsqrt`、`T.ieee_fdiv`和`T.ieee_add`），使开发者能够精确指定数值行为。

我们还致力于实现比特级可复现性，以验证内核与手写CUDA基线的对比。我们将TileLang的代数简化和降阶规则与主流CUDA工具链（如NVCC）对齐，避免引入非预期的比特级差异的转换。布局注解（例如`T.annotate_layout`）进一步允许用户锁定依赖于布局的降阶决策，保持求值和累加顺序与参考CUDA实现一致，从而在需要时实现比特级相同的输出。

我们的评估表明，这些以精度和可复现性为导向的设计选择并未牺牲性能：在保守默认设置下，TileLang内核仍保持竞争力，同时提供可调节选项以选择性放宽数值约束来换取更高速度。

3.3. 高性能批处理不变性与确定性内核库

为实现高效的训练与推理，我们开发了一套全面的高性能计算核心。除基础功能与硬件利用率最大化外，另一关键设计目标是确保训练可复现性，以及预训练、后训练与推理流程间的比特级对齐。为此，我们实现了端到端、比特级批次不变且确定性的计算核心，其性能开销极低。这些核心有助于调试、稳定性分析及保持后训练行为的一致性。

批次不变性。批次不变性确保任何给定标记的输出保持位级一致，无论其在批次中的位置如何。为实现批次不变性，主要挑战列举如下：

- 注意。为实现批次不变性，我们无法采用分割键值方法（Dao等人，2023），该方法通过将单序列的注意力计算分配到多个流式多处理器（SM）上来平衡SM负载。但放弃该技术将引发严重的波形量化问题³，进而影响GPU利用率。为此，我们开发了双内核批次不变解码策略：首个内核在单个SM内完成整条序列的注意力输出计算，确保全满载波形的高吞吐量；次内核则采用多SM协同处理单序列的方式，最大限度降低末段非满载波形的延迟，从而缓解波形量化效应。为保证两内核运算结果的比特级一致性，我们精心设计了次内核的计算路径，使其累加顺序与首内核严格一致。此外，次内核利用线程块集群内的分布式共享内存⁴，实现了跨SM的高速数据交换。该双内核方案将批次不变解码的开销控制在可忽略范围内。
- 矩阵乘法。传统的cuBLAS库（NVIDIA Corporation, 2024）无法实现批次不变性。因此，我们采用DeepGEMM（Zhao等人，2025）进行端到端替换。此外，对于极小的批次规模，常规实现通常采用split-k（Osama等人，2023）技术来提升性能。遗憾的是，split-k技术无法保证批次不变性——这正是DeepSeek-V4的核心特性。

³<https://docs.nvidia.com/deeplearning/performance/dl-performance-matrix-multiplication/index.html#wave-quant>

⁴<https://docs.nvidia.com/cuda/cuda-programming-guide/02-basics/writing-cuda-kernels.html#distributed-shared-memory>

Therefore, we abandon split-k in most scenarios, which, however, may cause performance degradation. To address this, we introduce a set of optimizations that enable our implementation of matrix multiplication to match or even surpass the performance of standard split-k in most major scenarios.

Determinism. Deterministic training is highly beneficial for debugging hardware or software issues. Moreover, when training exhibits anomalies such as loss spikes, determinism enables researchers to more easily pinpoint numerical causes and further refine the model design. Non-determinism in training typically stems from non-deterministic accumulation order, often due to the use of atomic addition instructions. This issue primarily occurs during the backward pass, notably at the following parts:

- **Attention Backward.** In conventional implementations of backward propagation for sparse attention, we use `atomicAdd` to accumulate gradients for the KV tokens. This introduces non-determinism due to the non-associativity of floating-point addition. To address this problem, we allocate separate accumulation buffers for each SM, followed by a global deterministic summation across all buffers.
- **MoE Backward.** When multiple SMs from different ranks concurrently write data to the same buffer on a receiving rank, negotiating writing positions also introduces non-determinism. To resolve this, we design a token order pre-processing mechanism within each single rank, combined with buffer isolation across multiple ranks. This strategy ensures determinism of both the send results of expert parallelism and the accumulation order in the MoE backward pass.
- **Matrix Multiplication in *mHC*.** *mHC* involves a matrix multiplication with an output dimension of only 24. For very small batch sizes, we are compelled to use the split-k (Osama et al., 2023) algorithm, whose naive implementation will cause non-determinism. To overcome this, we output each split part separately and perform a deterministic reduction in a subsequent kernel, thereby preserving both performance and determinism.

3.4. Training Framework

Our training framework is built upon the scalable and efficient infrastructure developed for DeepSeek-V3 (DeepSeek-AI, 2024). In training DeepSeek-V4, we inherit this robust foundation while introducing several key innovations to accommodate its novel architectural components — specifically the Muon optimizer, *mHC*, and the hybrid attention mechanism — while maintaining high training efficiency and stability.

3.4.1. Efficient Implementation of Muon

The Muon optimizer requires the full gradient matrix to compute parameter updates, which presents a challenge when combined with the Zero Redundancy Optimizer (ZeRO) (Rajbhandari et al., 2020). Traditional ZeRO is designed for element-wise optimizers like AdamW, where a single parameter matrix can be partitioned and updated across multiple ranks. To address this conflict, we design a hybrid strategy of ZeRO bucket assignment for Muon.

For dense parameters, we limit the maximum size of ZeRO parallelism and employ a knapsack algorithm to assign parameter matrices to these ranks, ensuring each rank manages a roughly balanced load. The bucket on each rank is padded to match the size of the largest bucket across ranks, facilitating efficient reduce-scatter operations. This padding typically incurs less than 10% memory overhead in our setup, where each rank manages no more than five parameter

因此，在多数情况下我们弃用了split-k方法，但这可能导致性能下降。为解决该问题，我们引入了一系列优化措施，使得我们的矩阵乘法实现在大多数主要场景下能够匹配甚至超越标准split-k的性能表现。

决定论。确定性训练对于调试硬件或软件问题极为有益。此外，当训练中出现诸如损失峰值等异常情况时，确定性使研究人员能够更轻松地定位数值原因，并进一步优化模型设计。训练中的非确定性通常源于非确定性的累积顺序，这往往是由于使用了原子加法指令所致。该问题主要出现在反向传播过程中，特别是在以下部分：

- 注意力反向传播。在稀疏注意力反向传播的传统实现中，我们使用atomicAdd为KV令牌累积梯度。由于浮点数加法的非结合性，这会引入不确定性。为解决这一问题，我们为每个流式多处理器(SM)分配独立的累加缓冲区，随后在所有缓冲区之间进行全局确定性求和。
- MoE反向传播。当来自不同等级的多个SM同时向接收等级上的同一缓冲区写入数据时，协商写入位置也会引入非确定性。为解决这一问题，我们在每个单独等级内设计了令牌顺序预处理机制，并结合跨多等级的缓冲区隔离策略。此方法确保了专家并行性的发送结果与MoE反向传播中累积顺序的确定性。
- *mHC*中的矩阵乘法。*mHC*涉及输出维度仅为24的矩阵乘法。对于极小的批次大小，我们不得不采用split-k (Osama等人, 2023年提出) 算法，其原始实现会引发非确定性。为此，我们分别输出每个分割部分，并在后续核函数中进行确定性归约，从而兼顾性能与确定性。

3.4. 训练框架

我们的训练框架建立在为DeepSeek-V3 (DeepSeek-AI, 2024年) 开发的可扩展高效基础设施之上。在训练DeepSeek-V4时，我们继承了这一坚实基础，同时引入了几项关键创新以适应其新颖架构组件——特别是Muon优化器、*mHC*和混合注意力机制——同时保持高训练效率与稳定性。

3.4.1. Efficient Implementation of Muon

Muon优化器需要完整的梯度矩阵来计算参数更新，这在与零冗余优化器 (ZeRO) (Rajbhandari等人, 2020) 结合时带来了挑战。传统的ZeRO专为像AdamW这样的逐元素优化器设计，其中单个参数矩阵可以跨多个rank进行分区和更新。为解决这一冲突，我们为Muon设计了一种ZeRO桶分配的混合策略。

对于密集参数，我们限制ZeRO并行的最大规模，并采用背包算法将这些参数矩阵分配给各计算秩，确保每个秩管理的负载大致均衡。每个秩上的存储桶会进行填充，以匹配跨秩最大存储桶的尺寸，从而促进高效的reduce-scatter操作。在我们的配置中，此类填充通常导致不到10%的内存开销，且每个秩管理的参数不超过五个

matrices. When the overall size of data parallelism exceeds the limit for ZeRO, we compute the Muon update redundantly across the extra data-parallel groups, trading computation for reduced total bucket memory.

For MoE parameters, we optimize each expert independently. We first flatten all down projection matrices in SwiGLU (Shazeer, 2020) of all experts across all layers, followed by flattened up projection matrices and gate matrices. Then, we pad the flattened vector to ensure we can evenly distribute this vector across all ranks without splitting any logically independent matrix. Given the large number of experts, we do not impose a limit of ZeRO parallelism for MoE parameters, and the padding overhead is also negligible.

Additionally, on each rank, consecutive parameters of identical shape will be automatically merged, enabling batched execution of the Newton-Schulz iterations for better hardware utilization. Furthermore, we observe that the Newton-Schulz iterations in Muon remain stable when computed with BF16 matrix multiplications. Leveraging this, we further quantize, in a stochastic rounding manner, the MoE gradients to be synchronized across data-parallel ranks to the BF16 precision, halving the communication volume. To avoid accumulation errors introduced by low-precision adders, we replace conventional tree- or ring-based reduce-scatter collectives with a two-phase approach. First, an all-to-all operation exchanges local gradients across ranks, and then each rank performs a local sum in FP32. This design maintains numerical robustness.

3.4.2. Cost-Effective and Memory-Efficient Implementation of *mHC*

The introduction of *mHC* increases both activation memory consumption and communication volume between pipeline stages, compared with conventional residual connections. To mitigate these costs, we implement several optimization strategies.

Firstly, we carefully design and implement fused kernels of *mHC* for both training and inference. Secondly, we introduce a recomputation strategy that selectively checkpoints intermediate tensors. Specifically, we recompute most hidden states between layers and all normalized layer inputs, while avoiding recomputation of compute-intensive operations. This achieves a balance between memory saving and computational overhead. Thirdly, we adjust the DualPipe 1F1B overlapping scheme to accommodate the increased pipeline communication and enable concurrent execution of some operations in *mHC*.

Collectively, these optimizations constrain the wall-time overhead of *mHC* to only 6.7% of the overlapped 1F1B pipeline stage. More details of the engineering optimization can be found in the dedicated *mHC* paper (Xie et al., 2026).

3.4.3. Contextual Parallelism for Long-Context Attention

Conventional Context Parallelism (CP) partitions the sequence dimension, with each rank maintaining contiguous s tokens. This introduces two challenges to our compressed attention mechanisms (i.e., CSA and HCA). On the one hand, training samples are packed from multiple sequences, and each sequence is compressed independently by a factor of m (or m'), with any trailing tokens fewer than m being discarded. Consequently, the compressed KV lengths are typically less than $\frac{s}{m}$ and vary across ranks. On the other hand, the compression requires m consecutive KV entries, which may straddle the boundary between two neighboring CP ranks.

To address these challenges, we design a two-stage communication approach. In the first stage, each rank i sends its last m uncompressed KV entries to rank $i + 1$. Then, rank $i + 1$ compresses some of these received entries together with its local s uncompressed KV entries,

矩阵。当数据并行的总体规模超出ZeRO的限制时，我们会在额外的数据并行组间冗余计算Muon更新，以计算换取减少的总存储桶内存。

对于MoE参数，我们独立优化每个专家。首先将所有层中所有专家的SwiGLU（Shazeer, 2020）下投影矩阵展平，随后展平上投影矩阵与门控矩阵。接着对展平后的向量进行填充，确保该向量能均匀分布至所有计算单元，且不分割任何逻辑独立的矩阵。鉴于专家数量庞大，我们对MoE参数不设ZeRO并行限制，填充开销亦可忽略不计。

此外，在每个计算节点上，形状相同的连续参数会被自动合并，从而实现对牛顿-舒尔茨迭代的批量执行，以提升硬件利用率。我们还观察到，Muon中的牛顿-舒尔茨迭代在使用BF16精度矩阵乘法时仍能保持稳定。基于这一特性，我们进一步以随机舍入方式将跨数据并行节点同步的MoE梯度量化为BF16精度，使通信量减半。为避免低精度加法器引入的累积误差，我们采用两阶段方法替代传统的树状或环状reduce-scatter集合通信：首先通过全对全操作跨节点交换本地梯度，随后各节点在FP32精度下执行本地求和。这一设计确保了数值鲁棒性。

3.4.2. Cost-Effective and Memory-Efficient Implementation of mHC

引入mHC相比传统的残差连接，既增加了激活内存消耗，也提高了流水线阶段间的通信量。为缓解这些开销，我们实施了多项优化策略。

首先，我们精心设计并实现了mHC在训练与推理阶段的融合内核。其次，我们引入了一种选择性检查点中间张量的重计算策略：通过重计算大部分层间隐藏状态及所有归一化层输入，同时规避计算密集型操作的重算，实现了内存节省与计算开销的平衡。第三，我们调整DualPipe 1F1B重叠调度方案，以适应增加的流水线通信需求，并支持mHC中部分操作的并发执行。

总的来说，这些优化措施将mHC的挂钟时间开销限制在重叠1F1B流水线阶段的仅6.7%。更多工程优化的细节可在专门的mHC论文（Xie等人，2026年）中找到。

3.4.3. Contextual Parallelism for Long-Context Attention

传统的上下文并行（CP）方法沿序列维度进行划分，每个计算节点维护连续的 s 个令牌。这为我们的压缩注意力机制（即CSA和HCA）带来了两个挑战。一方面，训练样本由多个序列打包而成，每个序列会独立地以 m （或 m' ）的系数进行压缩，不足 m 的尾部令牌将被舍弃。因此，压缩后的键值长度通常小于 $\frac{s}{m}$ ，且在不同节点间存在差异。另一方面，压缩过程需要 m 个连续的键值条目，这些条目可能跨越两个相邻CP节点的分界区域。

为应对这些挑战，我们设计了一种两阶段通信方法。在第一阶段，每个排名 i 会将其最后的 m 未压缩KV条目发送至排名 $i + 1$ 。接着，排名 $i + 1$ 会压缩部分接收到的条目以及其本地的 s 未压缩KV条目，

producing a fixed length of $\frac{s}{m} + 1$ compressed entries, in which exist some padding entries. In the second stage, an all-gather operation across all CP ranks collects the locally compressed KV entries. Then, a fused select-and-pad operator reorganizes them into the full set of compressed KV entries with a total length of $\text{cp_size} \cdot \frac{s}{m}$. Any padding entries are placed at the tail. For HCA and the indexer in CSA, the visible range of compressed KV entries for each query token can be precomputed by rules. For the sparse attention in CSA, the top- k selector explicitly specifies the indices of visible compressed KV entries for each query.

3.4.4. *Extended Automatic Differentiation for Flexible Activation Checkpointing*

Conventional activation checkpointing implementations operate at the granularity of an entire module, deciding whether to retain or recompute its output activations during the backward pass. This coarse granularity often leads to suboptimal trade-offs between recomputation cost and activation memory footprint. An alternative approach is to manually implement the forward and backward logic of an entire layer, explicitly managing tensor checkpointing states. While enabling fine-grained control, this method loses the convenience of the automatic differentiation framework, substantially increasing development complexity.

To achieve fine-grained control without sacrificing programming efficiency, we implement a tensor-level activation checkpointing mechanism with automatic differentiation support. With this mechanism, developers only need to implement the forward pass and selectively annotate individual tensors for automatic checkpointing and recomputation. Our framework leverages TorchFX (Reed et al., 2022) to trace the full computation graph. For each annotated tensor, it performs a backward traversal to identify the minimal subgraph required for its recomputation. We define these minimal subgraphs as recomputation graphs and insert them into the backward logic just before the corresponding gradient computation.

Compared with the manual implementation, this design introduces no additional overhead during training. Recomputation in this framework is implemented by directly freeing the GPU memory of the annotated tensor and reusing the storage pointer from the recomputed tensor, without any GPU memory copy. Furthermore, since graph tracing executes the model concretely, we can track the underlying storage pointer of each tensor, which enables automatic deduplication of recomputation for tensors that share storage (e.g., the input and output of a reshape operation). This relieves developers from reasoning about low-level memory details when annotating recomputation.

3.5. Inference Framework

Our inference framework largely inherits from that of DeepSeek-V3, with some differences in KV Cache management.

3.5.1. *KV Cache Structure and Management*

To efficiently manage the heterogeneous KV caches arising from the hybrid attention mechanism in DeepSeek-V4, we design a customized KV cache layout. The layout is illustrated in Figure 6, and we will elaborate on it in detail as follows.

Heterogeneous KV Entries in DeepSeek-V4. The hybrid attention mechanism in DeepSeek-V4 series introduces multiple types of KV entries with different Key-Value (KV) cache sizes and update rules. The lightning indexer for sparse selection introduces additional dimensions

生成固定长度的 $\frac{s}{m} + 1$ 个压缩条目，其中包含若干填充条目。在第二阶段，所有CP秩间执行全收集操作，汇聚本地压缩后的KV条目。随后，一个融合的选择与填充算子将这些条目重组为完整的一组压缩KV条目，总长度为 $cp_size \cdot \frac{s}{m}$ 。所有填充条目均被置于末尾。对于HCA及CSA中的索引器，每个查询令牌可见的压缩KV条目范围可通过规则预先计算。而对于CSA中的稀疏注意力机制，前k选择器会显式指定每个查询对应的可见压缩KV条目索引。

3.4.4. Extended Automatic Differentiation for Flexible Activation Checkpointing

传统的激活检查点实现以整个模块为粒度进行操作，决定在反向传播过程中是保留还是重新计算其输出激活。这种粗粒度往往导致重新计算成本和激活内存占用之间的次优权衡。另一种方法是手动实现整个层的前向和反向逻辑，显式管理张量检查点状态。虽然这种方法能实现细粒度控制，但失去了自动微分框架的便利性，大幅增加了开发复杂度。

为了实现细粒度控制的同时不牺牲编程效率，我们实现了一套支持自动微分的张量级激活检查点机制。借助该机制，开发者只需实现前向传播过程，并选择性地为单个张量添加注解以启用自动检查点与重计算。我们的框架利用TorchFX（Reed等人，2022）来追踪完整计算图。对于每个被注解的张量，系统会执行反向遍历以确定其重计算所需的最小子图。我们将这些最小子图定义为重计算图，并将它们插入到反向传播逻辑中，紧邻对应的梯度计算步骤之前。

与手动实现相比，该设计在训练过程中不会引入额外开销。该框架通过直接释放标注张量的GPU内存并复用重计算张量的存储指针来实现重计算，无需任何GPU内存拷贝。此外，由于图追踪会具体执行模型，我们可以追踪每个张量的底层存储指针，从而自动对共享存储的张量（例如reshape操作的输入和输出）进行重计算去重。这使得开发者在标注重计算时无需考虑底层内存细节。

3.5. 推理框架

我们的推理框架在很大程度上继承了DeepSeek-V3的设计，但在KV Cache管理方面存在一些差异。

3.5.1. KV Cache Structure and Management

为了高效管理DeepSeek-V4混合注意力机制产生的异构KV缓存，我们设计了一种定制化的KV缓存布局。该布局如图6所示，接下来我们将详细阐述其具体实现。

DeepSeek-V4中的异构KV条目。DeepSeek-V4系列采用的混合注意力机制引入了多种类型的KV条目，这些条目具有不同的键值对（KV）缓存大小及更新规则。面向稀疏选择的闪电索引器还额外引入了若干维度

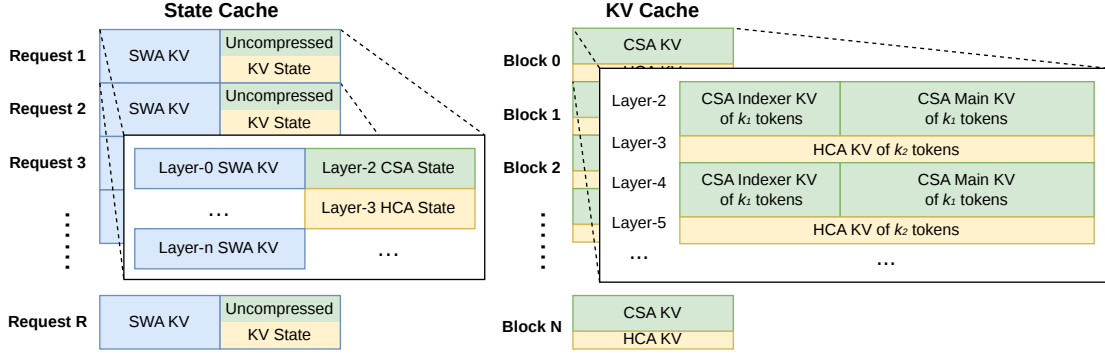


Figure 6 | Illustration of the KV cache Layout for DeepSeek-V4. The KV cache is organized into two primary components: a classical KV cache for CSA/HCA, and a state cache for SWA and unready-for-compression tokens in CSA/HCA. In the state cache, each request is assigned a fixed-size cache block. Within this block, the SWA segment stores the KV entries corresponding to the most recent n_{win} tokens, while the CSA/HCA segment stores uncompressed tail states that are not yet ready for compression. In the classical KV cache, we allocate multiple blocks per request. Each cache block covers $\text{lcm}(m, m')$ original tokens, producing $k_1 = \frac{\text{lcm}(m, m')}{m}$ CSA compressed tokens and $k_2 = \frac{\text{lcm}(m, m')}{m'}$ HCA compressed tokens.

into the KV cache that possess embedding sizes distinct from those in the primary attention. The compression techniques employed in CSA and HCA reduce the sequence length by factors of $\frac{1}{m}$ and $\frac{1}{m'}$, respectively, thereby decreasing the overall KV cache size. As a result, KV cache sizes vary across different layers. Furthermore, Sliding Window Attention (SWA) layers also operate with distinct KV cache sizes, as well as separate cache hit and eviction policies. In the compression branch, one KV entry is generated for every m tokens. When the number of remaining tokens is insufficient for compression, all pending tokens and their associated hidden states must be retained in a buffer until the compression operation can be executed. These buffered tokens represent a sequence state determined by positional context and are also managed within the KV cache framework.

Challenges in Managing Hybrid Attention KV Cache. The hybrid attention mechanism violates fundamental assumptions behind PagedAttention and its variants. Although recent hybrid KV cache managing algorithms (e.g., Jenga (Zhang et al., 2025a), Hymba (Dong et al., 2025)) target general hybrid attention models or specific structures, two principal obstacles prevent consolidating KV caches across all layers under the PagedAttention framework:

- Diverse cache policies, such as those used in Sliding Window Attention.
- Constraints imposed by high-performance attention kernels, including alignment requirements.

For efficient KV cache management of DeepSeek-V4, we design corresponding strategies to overcome these two challenges.

State Cache for SWA and Uncompressed Tail Tokens. To address the first obstacle, we adopt an alternative cache management mechanism. Since SWA is designed to enhance performance under a limited KV cache size, it is reasonable to treat it, along with the uncompressed tail tokens from the compression branch, as a state-space model. The corresponding KV cache can thus be

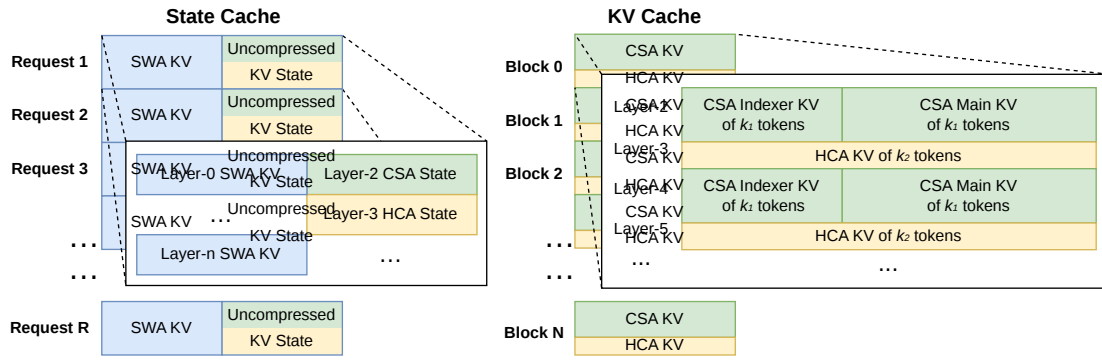


图6 | 展示了DeepSeek-V4的KV缓存布局示意图。KV缓存主要由两大组件构成：用于CSA/HCA的传统KV缓存，以及为SWA和CSA/HCA中尚未准备好压缩的令牌所设的状态缓存。在状态缓存中，每个请求被分配一个固定大小的缓存块。该缓存块内，SWA段存储与最近 n_{win} 个令牌对应的KV条目，而CSA/HCA段则存放尚未准备好压缩的尾部未压缩状态。传统KV缓存中，我们为每个请求分配多个缓存块。每个缓存块覆盖 $\text{lcm}(m, m')$ 个原始令牌，生成 $k_1 = \frac{\text{lcm}(m, m')}{m}$ 个CSA压缩令牌和 $k_2 = \frac{\text{lcm}(m, m')}{m'}$ 个HCA压缩令牌。

进入KV缓存的嵌入尺寸与主注意力机制中的不同。CSA和HCA采用的压缩技术分别以 $\frac{1}{m}$ 和 $\frac{1}{m'}$ 的倍数缩减序列长度，从而降低整体KV缓存大小。因此，不同层的KV缓存大小各异。此外，滑动窗口注意力（SWA）层也使用不同的KV缓存尺寸，并采用独立的缓存命中与淘汰策略。在压缩分支中，每 m 个令牌生成一个KV条目。当剩余令牌数量不足以完成压缩时，所有待处理的令牌及其关联的隐藏状态必须暂存于缓冲区，直至压缩操作可执行。这些缓冲令牌代表由位置上下文决定的序列状态，同样在KV缓存框架内进行管理。

混合注意力键值缓存管理的挑战。混合注意力机制违背了PagedAttention及其变体背后的基本假设。尽管近期的混合键值缓存管理算法（如Jenga（Zhang等，2025a）、Hymba（Dong等，2025））针对通用混合注意力模型或特定结构进行了优化，但在PagedAttention框架下整合所有层的键值缓存仍面临两大主要障碍：

- 多样化的缓存策略，例如滑动窗口注意力（Sliding Window Attention）中采用的那些。
- 高性能注意力内核施加的约束，包括对齐要求。

为了高效管理DeepSeek-V4的KV缓存，我们设计了相应策略来应对这两大挑战。

SWA与未压缩尾部令牌的状态缓存。针对第一个障碍，我们采用了另一种缓存管理机制。由于SWA旨在有限KV缓存大小下提升性能，将其与压缩分支中的未压缩尾部令牌一同视为状态空间模型是合理的。相应的KV缓存因此可被

regarded as a sequence-specific state that depends solely on the current position. Accordingly, we pre-allocate a fixed- and limited-size pool of state caches, and dynamically assign it to each sequence.

Sparse Attention Kernel Co-Design. Regarding the second obstacle, conventional high-performance attention kernels typically assume a fixed number B of tokens per block to optimize performance, corresponding to $B \cdot m$ original tokens in CSA and $B \cdot m'$ in HCA. Through employing a high-performance sparse-attention kernel, different layers can accommodate variable tokens per block without performance degradation. Achieving this requires co-designing the KV cache layout and the sparse attention kernel. For instance, padding blocks to align with cache lines can improve performance. Thus, for CSA with compression ratio m and HCA with ratio m' , the number of original tokens per block can be any multiple of $\text{lcm}(m, m')$, the least common multiple of these two compression ratios.

3.5.2. On-Disk KV Cache Storage

When serving DeepSeek-V4, we leverage an on-disk KV cache storage mechanism to eliminate repeated prefilling for shared-prefix requests. For the compressed KV entries in CSA/HCA and the uncompressed KV entries in Sliding Window Attention (SWA), we design separate solutions for storage management.

For CSA and HCA, we simply store all of the compressed KV entries to the disk. When a request hits a stored prefix, we read and reuse the compressed KV entries corresponding to the prefix, until the last complete compression block. Specially, for prefix tokens in the tail incomplete block, we still need to recompute them to restore the uncompressed KV entries, as uncompressed KV entries in CSA and HCA are not stored.

For the SWA KV entries, since they are not compressed and exist in every layer, their volume is approximately 8 times larger than the compressed CSA and HCA KV entries. To handle these large SWA KV entries efficiently, we propose and implement three distinct strategies for managing on-disk SWA KV entries, each offering a different trade-off between storage overhead and computational redundancy:

- **Full SWA Caching.** This strategy stores the complete SWA KV entries for all tokens, ensuring computational zero-redundancy. Under this strategy, the SWA KV entries of the hitting prefix can be reconstructed by just reading the on-disk cache of the last n_{win} tokens within that prefix. Despite computational zero-redundancy, this strategy is inefficient for modern SSD-based storage systems — only a small subset of the stored SWA KV cache will be accessed for each hitting request, which leads to an unbalanced write-intensive access pattern.
- **Periodic Checkpointing.** This strategy checkpoints SWA KV entries of the last n_{win} tokens within every p tokens, where p is a tunable parameter. For a hitting prefix, we load the most recent checkpointed state, and then recompute the remaining tail tokens. Through tuning p , this strategy enables an on-demand trade-off between storage and computation.
- **Zero SWA Caching.** This strategy does not store any SWA KV entries. For a hitting prefix, we need to perform more recomputation to restore the SWA KV entries. To be specific, in each attention layer, the SWA KV entry of each token depends on the SWA KV entries of only the most recent n_{win} tokens from the previous layer. Therefore, leveraging cached CSA and HCA KV entries, recomputing the last $n_{\text{win}} \cdot L$ tokens is enough to restore the last n_{win} SWA KV entries for an L -layer model.

被视为一种仅依赖于当前位置的序列特定状态。因此，我们预先分配一个固定且有限大小的状态缓存池，并动态地将其分配给每个序列。

稀疏注意力内核协同设计。针对第二个障碍，传统高性能注意力内核通常假设每个块有固定数量 B 的令牌以优化性能，对应CSA中的 $B \cdot m$ 个原始令牌和HCA中的 $B \cdot m'$ 个。通过采用高性能稀疏注意力内核，不同层可以适应每块可变数量的令牌，而不会降低性能。实现这一点需要协同设计KV缓存布局与稀疏注意力内核。例如，通过填充块以对齐缓存线可以提升性能。因此，对于压缩比为 m 的CSA和压缩比为 m' 的HCA，每块的原始令牌数量可以是这两个压缩比的最小公倍数 (m, m') 的任意倍数。

3.5.2. On-Disk KV Cache Storage

在部署DeepSeek-V4时，我们采用基于磁盘的KV缓存存储机制，以消除共享前缀请求的重复预填充操作。针对CSA/HCA中的压缩KV条目与滑动窗口注意力(SWA)中的未压缩KV条目，我们分别设计了独立的存储管理解决方案。

对于CSA和HCA，我们仅将所有压缩后的KV条目存储至磁盘。当请求命中已存储的前缀时，我们会读取并复用与该前缀对应的压缩KV条目，直至最后一个完整的压缩块。特别地，对于尾部不完整块中的前缀标记，由于CSA和HCA中未存储未压缩的KV条目，我们仍需重新计算这些标记以恢复未压缩的KV条目。

对于SWA键值条目，由于它们未经压缩且存在于每一层中，其体积大约是压缩后的CSA和HCA键值条目的8倍。为了高效处理这些庞大的SWA键值条目，我们提出并实施了三种不同的磁盘存储管理策略，每种策略在存储开销与计算冗余之间提供了不同的权衡：

- 全量SWA缓存。此策略存储所有token的完整SWA键值条目，确保计算零冗余。在该策略下，命中前缀的SWA键值条目仅需读取该前缀内最后 n_{win} 个token的磁盘缓存即可重构。尽管实现了计算零冗余，该策略对基于现代SSD的存储系统效率低下——每个命中请求仅会访问存储的SWA键值缓存中的一小部分，从而导致写密集型的非均衡访问模式。
- 周期性检查点策略。该策略每隔 p 个令牌，就对最后 n_{win} 个令牌的SWA KV条目进行一次检查点保存，其中 p 为可调参数。当遇到匹配前缀时，我们加载最近一次检查点保存的状态，然后重新计算剩余的尾部令牌。通过调整 p ，此策略实现了存储与计算之间的按需权衡。
- 零SWA缓存。该策略不存储任何SWA键值条目。对于命中的前缀，我们需要执行更多的重新计算来恢复SWA键值条目。具体来说，在每一个注意力层中，每个令牌的SWA键值条目仅依赖于前一层中最新的 n_{win} 个令牌的SWA键值条目。因此，利用缓存的CSA和HCA键值条目，重新计算最后的 $n_{\text{win}} \cdot L$ 个令牌足以为一个 L 层模型恢复最后的 n_{win} 个SWA键值条目。

Depending on specific deployment scenarios, we select the most suitable strategy to achieve the desired trade-off between storage and computation.

4. Pre-Training

4.1. Data Construction

On top of the pre-training data of DeepSeek-V3, we endeavor to construct a more diverse and higher-quality training corpus with longer effective contexts. We continually refine our data construction pipelines. For web-sourced data, we implement filtering strategies to remove batched auto-generated and templated content, thereby mitigating the risk of model collapse (Zhu et al., 2024). Mathematical and programming corpora still remain core components of our training data, and we further enhance the coding capabilities of DeepSeek-V4 series by incorporating agentic data during the mid-training phase. For multilingual data, we build a larger corpus for DeepSeek-V4, improving its capture of long-tail knowledge across different cultures. For DeepSeek-V4, we place a particular emphasis on long-document data curation, prioritizing scientific papers, technical reports, and other materials that reflect unique academic values. Combining all the above, our pre-training corpus comprises more than 32T tokens, containing mathematical contents, codes, web pages, long documents, and other high-quality categories.

For pre-training data, we largely follow the same pre-processing strategies of DeepSeek-V3. For tokenization, on top of the DeepSeek-V3 tokenizer, we introduce a few special tokens for context construction, and still remain the vocabulary size to be 128K. We also inherit the token-splitting (DeepSeek-AI, 2024) and Fill-in-Middle (FIM) (DeepSeek-AI, 2024) strategies from DeepSeek-V3. Inspired by Ding et al. (2024), we pack documents from different sources into appropriate sequences to minimize sample truncation. Different from DeepSeek-V3, we employ sample-level attention masking during pre-training.

4.2. Pre-Training Setups

4.2.1. Model Setups

DeepSeek-V4-Flash. We set the number of Transformer layers to 43 and the hidden dimension d to 4096. For the first two layers, we use pure sliding window attention. For the subsequent layers, CSA and HCA are used in an interleaved manner. For CSA, we set the compression rate m to 4, the number of indexer query heads n_h^I to 64, the indexer head dimension c^I to 128, and the number of KV entries selected for sparse attention (i.e., attention top-k) to 512. For HCA, we set the compression rate m' to 128. For both CSA and HCA, we set the number of query heads n_h to 64, the head dimension c to 512, and the query compression dimension d_c to 1024. The number of output projection groups g is set to 8, and the dimension of each intermediate attention output d_g is set to 1024. For the additional branch of sliding window attention, the window size n_{win} is set to 128. We employ MoE layers in all Transformer blocks, but use the Hash routing strategy for the first 3 MoE layers. Each MoE layer consists of 1 shared expert and 256 routed experts, where the intermediate hidden dimension of each expert is 2048. Among the routed experts, 6 experts will be activated for each token. The multi-token prediction depth is set to 1. As for $m\text{HC}$, the expansion factor n_{hc} is set to 4, and the number of Sinkhorn-Knopp iterations t_{max} is set to 20. Under this configuration, DeepSeek-V4-Flash comprises 284B total parameters, of which 13B are activated for each token.

根据具体部署场景，我们选择最合适的策略，以实现存储与计算之间的理想权衡。

4. 预训练

4.1. 数据构建

在DeepSeek-V3预训练数据的基础上，我们致力于构建更加多样化、更高质量且具有更长有效上下文的训练语料库。我们持续优化数据构建流程：针对网络来源数据，实施过滤策略以去除批量自动生成和模板化内容，从而降低模型崩溃的风险（Zhu et al., 2024）。数学与编程语料仍是我们训练数据的核心组成部分，通过在训练中期引入代理数据，进一步强化了DeepSeek-V4系列的代码能力。对于多语言数据，我们为DeepSeek-V4构建了更大规模的语料库，提升其对不同文化长尾知识的捕捉能力。针对DeepSeek-V4，我们特别注重长文档数据的整理，优先选取科学论文、技术报告等体现独特学术价值的材料。综合上述所有要素，我们的预训练语料库规模超过3 2T token，涵盖数学内容、代码、网页、长文档及其他高质量类别。

对于预训练数据，我们主要遵循DeepSeek-V3的相同预处理策略。在分词方面，基于DeepSeek-V3的分词器，我们引入了若干特殊标记用于上下文构建，同时保持128K的词汇量不变。我们还继承了DeepSeek-V3的令牌分割(DeepSeek-AI, 2024)和中间填充(FIM)(DeepSeek-AI, 2024)策略。受Ding等人(2024)的启发，我们将不同来源的文档打包成适当序列以最小化样本截断。与DeepSeek-V3不同之处在于，我们在预训练阶段采用了样本级注意力掩码机制。

4.2. 预训练设置

4.2.1. Model Setups

DeepSeek-V4-Flash。我们将Transformer层数设置为43，隐藏维度 d 设为4096。前两层采用纯滑动窗口注意力机制，后续层则交替使用CSA与HCA。对于CSA，压缩率 m 设为4，索引器查询头数 n_h^i 设为64，索引器头维度 c^i 设为128，稀疏注意力选取的KV条目数（即注意力top-k）设为512。对于HCA，压缩率 m' 设为128。CSA与HCA的查询头数 n_h 均设为64，头维度 c 设为512，查询压缩维度 d_c 设为1024。输出投影组数 g 设为8，各中间注意力输出维度 d_g 设为1024。滑动窗口注意力附加分支的窗口大小 n_{win} 设为128。所有Transformer块均采用MoE层，但前3个MoE层使用哈希路由策略。每个MoE层包含1个共享专家和256个路由专家，各专家中间隐藏维度为2048。路由专家中每个token会激活6个专家。多token预测深度设为1。对于 mHC ，扩展因子 n_{hc} 设为4，Sinkhorn-Knopp迭代次数 t_{max} 设为20。该配置下DeepSeek-V4-Flash总参数量达2840亿，每个token激活参数量为130亿。

DeepSeek-V4-Pro. We set the number of Transformer layers to 61 and the hidden dimension d to 7168. For the first two layers, we use HCA. For the subsequent layers, CSA and HCA are used in an interleaved manner. For CSA, we set the compression rate m to 4, the number of indexer query heads n_h^I to 64, the indexer head dimension c^I to 128, and the number of KV entries selected for sparse attention (i.e., attention top-k) to 1024. For HCA, we set the compression rate m' to 128. For both CSA and HCA, we set the number of query heads n_h to 128, the head dimension c to 512, and the query compression dimension d_c to 1536. The number of output projection groups g is set to 16, and the dimension of each intermediate attention output d_g is set to 1024. For the additional branch of sliding window attention, the window size n_{win} is set to 128. We employ MoE layers in all Transformer blocks, but use the Hash routing strategy for the first 3 MoE layers. Each MoE layer consists of 1 shared expert and 384 routed experts, where the intermediate hidden dimension of each expert is 3072. Among the routed experts, 6 experts will be activated for each token. The multi-token prediction depth is set to 1. As for $m\text{HC}$, the expansion factor n_{hc} is set to 4, and the number of Sinkhorn-Knopp iterations t_{max} is set to 20. Under this configuration, DeepSeek-V4-Pro comprises 1.6T total parameters, of which 49B are activated for each token.

4.2.2. Training Setups

DeepSeek-V4-Flash. We employ the Muon optimizer (Jordan et al., 2024; Liu et al., 2025) for the majority of parameters, but use the AdamW optimizer (Loshchilov and Hutter, 2017) for the embedding module, the prediction head module, and the weights of all RMSNorm modules. For AdamW, we set its hyper-parameters to $\beta_1 = 0.9$, $\beta_2 = 0.95$, $\epsilon = 10^{-20}$, and $\text{weight_decay} = 0.1$. For Muon, we set the momentum to 0.95 and the weight decay to 0.1, and rescale the RMS of each update matrix to 0.18 for reutilization of the AdamW learning rate. We train DeepSeek-V4-Flash on 32T tokens, and as in DeepSeek-V3, we also employ a batch size scheduling strategy that increases the batch size (in tokens) from a small size to 75.5M and then keeps it at 75.5M during most of the training. The learning rate is linearly warmed up in the first 2000 steps, maintained at 2.7×10^{-4} for most of the training. Near the end of the training, we finally decay the learning rate to 2.7×10^{-5} following a cosine schedule. The training starts with a sequence length of 4K, and we gradually extend the training sequence length to 16K, 64K, and 1M. As for the setups of sparse attention, we first warmup the model with dense attention for the first 1T tokens, and introduce sparse attention at the sequence length of 64K and keep sparse attention during the rest of the training. When introducing attention sparsity, we first set a short stage to warm up the lightning indexer in CSA, and then train the model with sparse attention for most of the training. For auxiliary-loss-free load balancing, we set the bias update speed to 0.001. For the balance loss, we set its loss weight to 0.0001 to avoid extreme imbalance within single sequences. The MTP loss weight is set to 0.3 for most of the training, and to 0.1 upon the start of learning rate decay.

DeepSeek-V4-Pro. Except for specific values of hyper-parameters, the training setup of DeepSeek-V4-Pro is largely consistent with that of DeepSeek-V4-Flash. We employ the Muon optimizer for the majority of parameters, but use the AdamW optimizer for the embedding module, the prediction head module, and the weights of all RMSNorm modules. The hyper-parameters of AdamW and Muon are the same as those of DeepSeek-V4-Flash. We train DeepSeek-V4-Pro on 33T tokens, and also employ a batch size scheduling strategy, with the maximum batch size being 94.4M tokens. The learning rate scheduling strategy is largely the same as that of DeepSeek-V4-Flash, but the peak learning rate is set to 2.0×10^{-4} and the end learning rate is set to 2.0×10^{-5} . The training also starts with a sequence length of 4K, and the length is gradually

DeepSeek-V4-Pro。我们将Transformer层数设置为61，隐藏维度 d 设为7168。前两层采用HCA结构，后续层则以交替方式使用CSA与HCA。针对CSA，设置压缩率 m 为4、索引器查询头数 n_h^I 为64、索引器头维度 c^I 为128，并为稀疏注意力（即注意力top-k）选定1024个KV条目。HCA的压缩率 m' 设为128。CSA与HCA均配置查询头数 n_h 为128、头维度 c 为512、查询压缩维度 d_c 为1536。输出投影组数 g 设为16，各中间注意力输出维度 d_g 设为1024。滑动窗口注意力分支的窗口大小 n_{win} 设置为128。所有Transformer块均采用MoE层，但前3个MoE层使用哈希路由策略。每个MoE层包含1个共享专家和384个路由专家，各专家中间隐藏维度为3072，其中每个令牌会激活6个路由专家。多令牌预测深度设为1。对于 mHC ，扩展因子 n_{hc} 设为4，Sinkhorn-Knopp迭代次数 t_{max} 设为20。此配置下，DeepSeek-V4-Pro总参数量达1.6T，每个令牌激活参数为49B。

4.2.2. Training Setups

DeepSeek-V4-Flash。我们对多数参数采用Muon优化器（Jordan等人，2024；Liu等人，2025），但对嵌入模块、预测头模块及所有RMSNorm模块的权重使用AdamW优化器（Loshchilov与Hutter，2017）。AdamW的超参数设置为 $\beta_1 = 0.9$ 、 $\beta_2 = 0.95$ 、 $\epsilon = 10^{-20}$ ，权重衰减=0.1。Muon的动量设为0.95，权重衰减为0.1，并将每次更新矩阵的RMS重新缩放至0.18以实现AdamW学习率的复用。我们在32T token上训练DeepSeek-V4-Flash，并如DeepSeek-V3般采用批次大小调度策略：初始较小批次逐步增至75.5M token后保持该规模。学习率在前2000步线性预热，主体训练阶段维持在 2.7×10^{-4} ；训练末期按余弦调度最终衰减至 2.7×10^{-5} 。训练序列长度从4K开始，逐步扩展至16K、64K及1M。稀疏注意力设置方面，前1T token使用稠密注意力预热模型，64K序列长度时引入稀疏注意力并持续至训练结束。启用注意力稀疏性时，先通过短阶段预热CSA中的闪电索引器，之后主要训练均采用稀疏注意力。无辅助损失的负载均衡中，偏置更新速度设为0.001；平衡损失的权重设为0.0001以避免单序列内极端失衡。MTP损失权重主体训练阶段为0.3，学习率衰减开始时调整为0.1。

DeepSeek-V4-Pro。除特定超参数值外，DeepSeek-V4-Pro的训练设置与DeepSeek-V4-Flash基本一致。我们对大部分参数采用Muon优化器，但对嵌入模块、预测头模块及所有RMSNorm模块的权重使用AdamW优化器。AdamW与Muon的超参数设置与DeepSeek-V4-Flash相同。我们在33T token上训练DeepSeek-V4-Pro，并采用批量大小调度策略，最大批量大小为94.4M token。学习率调度策略与DeepSeek-V4-Flash大体相同，但峰值学习率设为 2.0×10^{-4} ，最终学习率设为 2.0×10^{-5} 。训练初始序列长度为4K，随后逐步

extended to 16K, 64K, and 1M. Compared with DeepSeek-V4-Flash, DeepSeek-V4-Pro starts with a longer stage of dense attention, and the strategy of introducing sparse attention is the same as DeepSeek-V4-Flash, following a two-stage training method. For auxiliary-loss-free load balancing, we set the bias update speed to 0.001. For the balance loss, we set its loss weight to 0.0001 to avoid extreme imbalance within single sequences. The MTP loss weight is set to 0.3 for most of the training, and to 0.1 upon the start of learning rate decay.

4.2.3. Mitigating Training Instability

Training trillion-parameter MoE models presents significant stability challenges, and DeepSeek-V4 series are no exception. We encountered notable instability challenges during training. While simple rollbacks could temporarily restore the training state, they proved inadequate as a long-term solution because they do not prevent the recurrence of loss spikes. Empirically, we identified that the occurrence of spikes is consistently tied to outliers in the MoE layers, and the routing mechanism itself appears to exacerbate the emergence of these outliers. Therefore, we sought to tackle this issue from two dimensions: breaking the vicious cycle induced by routing, and directly suppressing anomalous values. Fortunately, we discovered two practical techniques that effectively maintain training stability. Although a comprehensive theoretical understanding of their underlying mechanisms remains an open question for now, we are sharing them openly to foster further exploration by the community.

Anticipatory Routing. We found that decoupling the synchronous updates of the backbone network and the routing network significantly improves training stability. Consequently, at step t , we use the current network parameters θ_t for feature computation, but the routing indices are computed and applied using the historical network parameters $\theta_{t-\Delta t}$. In practice, to circumvent the overhead of loading model parameters twice, we fetch the data for step t in advance at step $t - \Delta t$. We "anticipatorily" compute and cache the routing indices to be used later at step t , which is why we name this approach Anticipatory Routing. We also heavily optimized this at the infrastructure level. First, given that pre-computing the routing indices only requires a single forward pass over the data, we carefully orchestrated the pipeline execution and the overlapping of computation with Expert Parallelism (EP) communication, successfully bounding the additional wall-clock time overhead of Anticipatory Routing to approximately 20%. Second, we introduced an automatic detection mechanism that triggers a short rollback and activates Anticipatory Routing exclusively when a loss spike occurs; after operating in this mode for a certain period, the system reverts to standard training. Ultimately, this dynamic application allows us to avert loss spikes with negligible overall additional training overhead, all without compromising model performance.

SwiGLU Clamping. In previous literature (Bello et al., 2017; Riviere et al., 2024), clamping has been explicitly utilized to constrain numerical ranges, thereby enhancing training stability. In our actual training runs, we empirically found that applying SwiGLU clamping (OpenAI, 2025) effectively eliminates outliers and substantially aids in stabilizing the training process, without compromising performance. Throughout the training of both DeepSeek-V4-Flash and DeepSeek-V4-Pro, we clamped the linear component of SwiGLU to the range of $[-10, 10]$, while capping the upper bound of the gate component at 10.

扩展至16K、64K和1M。与DeepSeek-V4-Flash相比，DeepSeek-V4-Pro初始阶段采用更长时间的密集注意力机制，稀疏注意力的引入策略与DeepSeek-V4-Flash相同，均遵循两阶段训练方法。在无辅助损失的负载均衡方面，我们将偏置更新速度设为0.001；对于平衡损失，其损失权重设为0.0001以避免单序列内出现极端失衡。MTP损失权重在训练大部分时段设为0.3，在开始学习率衰减时调整为0.1。

4.2.3. Mitigating Training Instability

训练万亿参数的混合专家（MoE）模型面临着显著的稳定性挑战，DeepSeek-V4系列也不例外。在训练过程中，我们遇到了明显的不稳定性难题。虽然简单的回滚操作能暂时恢复训练状态，但这并非长久之计，因为它无法阻止损失突刺的再次发生。经验表明，突刺现象始终与MoE层中的异常值相关联，而路由机制本身似乎会加剧这些异常值的出现。因此，我们尝试从两个维度解决这一问题：打破路由引发的恶性循环，以及直接抑制异常数值。幸运的是，我们发现了两种实用技术能有效维持训练稳定性。尽管目前对其背后机制的完整理论理解仍是未解之谜，但我们选择公开分享这些发现，以促进社区进一步探索。

前瞻性路由。我们发现，将主干网络和路由网络的同步更新解耦能显著提升训练稳定性。因此，在步骤 t 时，我们使用当前网络参数 θ_t 进行特征计算，但路由索引则是基于历史网络参数 $\theta_{t-\Delta t}$ 计算并应用的。实践中，为避免重复加载模型参数的开销，我们会在步骤 $t - \Delta t$ 预先获取步骤 t 所需数据。通过“前瞻性”计算并缓存后续步骤 t 将用到的路由索引，我们将该方法命名为前瞻性路由。在基础设施层面，我们还对此进行了深度优化：首先，鉴于预计算路由索引仅需单次数据前向传播，我们精心编排了流水线执行与专家并行（EP）通信的计算重叠，成功将前瞻性路由的额外实时开销控制在约20%；其次，我们引入了自动检测机制，在损失突增时触发短暂回滚并启用前瞻性路由，该模式运行一段时间后系统会恢复标准训练。最终，这种动态应用方式让我们能以可忽略的整体额外训练开销规避损失突增，且完全不损害模型性能。

SwiGLU 钳位。在先前的研究中（Bello等人，2017；Riviere等人，2024），钳位被明确用于约束数值范围，从而提升训练稳定性。在我们的实际训练过程中，经验发现应用SwiGLU钳位（OpenAI，2025）能有效消除异常值，并在不牺牲性能的前提下显著帮助稳定训练过程。在DeepSeek-V4-Flash和DeepSeek-V4-Pro的训练中，我们将SwiGLU的线性分量钳位在 $[-10, 10]$ 范围内，同时将门控分量的上限限制在10。

4.3. Evaluations

4.3.1. Evaluation Benchmarks

For the evaluation of the base models, we consider benchmarks spanning four key dimensions: world knowledge, language understanding and reasoning, coding and mathematics, and long-context processing.

World knowledge benchmarks include AGIEval (Zhong et al., 2023), C-Eval (Huang et al., 2023), CMMLU (Li et al., 2023) MMLU (Hendrycks et al., 2020), MMLU-Redux (Gema et al., 2024), MMLU-Pro (Wang et al., 2024b), MMMLU (OpenAI, 2024a), MultiLoKo (Hupkes and Bogoychev, 2025), Simple-QA verified (Haas et al., 2025), SuperGPQA (Du et al., 2025), FACTS Parametric (Cheng et al., 2025), and TriviaQA (Joshi et al., 2017).

Language understanding and reasoning benchmarks include BigBench Hard (BBH) (Suzgun et al., 2022), DROP (Dua et al., 2019), HellaSwag (Zellers et al., 2019), CLUEWSC (Xu et al., 2020), and WinoGrande (Sakaguchi et al., 2019).

Coding and mathematical benchmarks include BigCodeBench (Zhuo et al., 2025), HumanEval (Chen et al., 2021), GSM8K (Cobbe et al., 2021), MATH (Hendrycks et al., 2021), MGSM (Shi et al., 2023), and CMATH (Wei et al., 2023).

Long context benchmarks include LongBench-V2 (Bai et al., 2025b).

4.3.2. Evaluation Results

In Table 1, we provide a detailed comparison of the base models for DeepSeek-V3.2, DeepSeek-V4-Flash, and DeepSeek-V4-Pro, all evaluated under a unified internal framework with strictly consistent settings.

Comparing DeepSeek-V4-Flash-Base with DeepSeek-V3.2-Base reveals a compelling efficiency story. Despite utilizing a substantially smaller number of both activated and total parameters, DeepSeek-V4-Flash-Base outperforms DeepSeek-V3.2-Base across a wide array of benchmarks. This advantage is especially evident in world knowledge tasks and challenging long-context scenarios. These results underscore that architectural improvements, refined data quality, and training optimizations in DeepSeek-V4-Flash-Base yield superior performance even with a more compact parameter budget, effectively surpassing the larger DeepSeek-V3.2-Base on the majority of evaluations.

Furthermore, DeepSeek-V4-Pro-Base demonstrates a further, decisive leap in capability, establishing near-universal dominance over both DeepSeek-V3.2-Base and DeepSeek-V4-Flash-Base. With improvements across almost all categories, DeepSeek-V4-Pro-Base reaches new performance highs among DeepSeek base models on the most demanding benchmarks. On knowledge-intensive evaluations, it delivers dramatic gains, while also substantially advancing long-context understanding. On most reasoning and code benchmarks, DeepSeek-V4-Pro-Base also exceeds both previous models. This comprehensive uplift confirms DeepSeek-V4-Pro-Base as the strongest foundation model in the DeepSeek series, outperforming its predecessors across the spectrum of knowledge, reasoning, coding, and long-context capabilities.

4.3. 评估

4.3.1. Evaluation Benchmarks

在对基础模型进行评估时，我们考量了涵盖四大关键维度的基准测试：世界知识、语言理解与推理、编程与数学能力，以及长上下文处理。

世界知识基准包括AGIEval (Zhong等人, 2023)、C-Eval (Huang等人, 2023)、CMMLU (Li等人, 2023)、MMLU (Hendrycks等人, 2020)、MMLU-Redux (Gema等人, 2024)、MMLU-Pro (Wang等人, 2024b)、MMMLU (OpenAI, 2024a)、MultiLoKo (Hupkes和Bogoychev, 2025)、Simple-QA verified (Haas等人, 2025)、SuperGPQA (Du等人, 2025)、FACTS Parametric (Cheng等人, 2025) 以及TriviaQA (Joshi等人, 2017)。

语言理解和推理基准包括BigBench Hard (BBH) (Suzgun等人, 2022)、DROP (Dua等人, 2019)、HellaSwag (Zellers等人, 2019)、CLUEWSC (Xu等人, 2020)以及WinoGrande (Sakaguchi等人, 2019)。

编码与数学基准测试包括BigCodeBench (Zhuo等人, 2025年)、HumanEval (Chen等人, 2021年)、GSM8K (Cobbe等人, 2021年)、MATH (Hendrycks等人, 2021年)、MGSM (Shi等人, 2023年) 以及CMath (Wei等人, 2023年)。

长文本基准测试包括LongBench-V2 (Bai等人, 2025b)。

4.3.2. Evaluation Results

在表1中，我们详细对比了DeepSeek-V3.2、DeepSeek-V4-Flash与DeepSeek-V4-Pro的基础模型性能，所有评估均在统一内部框架下严格按照相同设置进行。

将DeepSeek-V4-Flash-Base与DeepSeek-V3.2-Base进行对比，展现出一个引人注目的效率故事。尽管激活参数和总参数量显著减少，DeepSeek-V4-Flash-Base在广泛的基准测试中仍超越了DeepSeek-V3.2-Base。这一优势在世界知识任务和具有挑战性的长上下文场景中尤为明显。这些结果凸显出，即使采用更精简的参数规模，DeepSeek-V4-Flash-Base在架构改进、数据质量优化和训练策略上的精进仍能带来卓越性能，从而在大多数评估中有效超越了规模更大的DeepSeek-V3.2-Base。

此外，DeepSeek-V4-Pro-Base展现出能力上的进一步飞跃，确立了近乎全面优于DeepSeek-V3.2-Base和DeepSeek-V4-Flash-Base的优势。在几乎所有类别中均有提升，DeepSeek-V4-Pro-Base在最具挑战性的基准测试中创下了DeepSeek基础模型的新性能高峰。在知识密集型评估中，它实现了显著进步，同时大幅推进了长上下文理解能力。在多数推理和代码基准测试中，DeepSeek-V4-Pro-Base同样超越了前代模型。这一全面提升证实了DeepSeek-V4-Pro-Base作为DeepSeek系列中最强大的基础模型，在知识、推理、编码及长上下文处理能力上全面超越其前辈。

Table 1 | Comparison among DeepSeek-V3.2-Base, DeepSeek-V4-Flash-Base, and DeepSeek-V4-Pro-Base. All models are evaluated in our internal framework and share the same evaluation setting. Scores with a gap not exceeding 0.3 are considered to be at the same level. The highest score in each row is in **bold font**, and the second is underlined.

Benchmark (Metric)		# Shots	DeepSeek-V3.2 Base	DeepSeek-V4-Flash Base	DeepSeek-V4-Pro Base
Architecture		-	MoE	MoE	MoE
# Activated Params		-	37B	13B	49B
# Total Params		-	671B	284B	1.6T
World Knowl.	AGIEval (EM)	0-shot	80.1	<u>82.6</u>	83.1
	MMLU (EM)	5-shot	87.8	<u>88.7</u>	90.1
	MMLU-Redux (EM)	5-shot	87.5	<u>89.4</u>	90.8
	MMLU-Pro (EM)	5-shot	65.5	<u>68.3</u>	73.5
	MMMLU (EM)	5-shot	87.9	<u>88.8</u>	90.3
	C-Eval (EM)	5-shot	90.4	<u>92.1</u>	93.1
	CMMLU (EM)	5-shot	88.9	<u>90.4</u>	90.8
	MultiLoKo (EM)	5-shot	38.7	<u>42.2</u>	51.1
	Simple-QA verified (EM)	25-shot	28.3	<u>30.1</u>	55.2
	SuperGPQA (EM)	5-shot	45.0	<u>46.5</u>	53.9
	FACTS Parametric (EM)	25-shot	27.1	<u>33.9</u>	62.6
	TriviaQA (EM)	5-shot	<u>83.3</u>	82.8	85.6
Lang. & Reas.	BBH (EM)	3-shot	87.6	<u>86.9</u>	87.5
	DROP (F1)	1-shot	<u>88.2</u>	88.6	88.7
	HellaSwag (EM)	0-shot	<u>86.4</u>	85.7	88.0
	WinoGrande (EM)	0-shot	<u>78.9</u>	<u>79.5</u>	81.5
	CLUEWSC (EM)	5-shot	<u>83.5</u>	82.2	85.2
Code & Math	BigCodeBench (Pass@1)	3-shot	63.9	56.8	<u>59.2</u>
	HumanEval (Pass@1)	0-shot	62.8	<u>69.5</u>	76.8
	GSM8K (EM)	8-shot	<u>91.1</u>	90.8	92.6
	MATH (EM)	4-shot	<u>60.5</u>	57.4	64.5
	MGSM (EM)	8-shot	<u>81.3</u>	85.7	<u>84.4</u>
	CMath (EM)	3-shot	<u>92.6</u>	93.6	90.9
Long Context	LongBench-V2 (EM)	1-shot	40.2	<u>44.7</u>	51.5

5. Post-Training

5.1. Post-Training Pipeline

Following pre-training, we conducted a post-training phase to yield the final models of DeepSeek-V4 series. Although the training pipeline largely mirrored that of DeepSeek-V3.2, a critical methodological substitution was made: the mixed Reinforcement Learning (RL) stage was entirely replaced by On-Policy Distillation (OPD; Gu et al., 2024; Lu and Lab, 2025).

5.1.1. Specialist Training

The development of domain specialists was conducted by adapting the DeepSeek-V3.2 training pipeline. Specifically, each model was sequentially optimized through an initial fine-tuning phase and subsequent Reinforcement Learning (RL) guided by domain-specific prompts and reward signals. For the RL stage, we implemented the Group Relative Policy Optimization (GRPO) algorithm, maintaining hyper-parameters closely aligned with our prior research (DeepSeek-AI, 2025; DeepSeek-AI, 2025).

表1 | DeepSeek-V3.2-Base、DeepSeek-V4-Flash-Base与DeepSeek-V4-Pro-Base的对比。所有模型均在我们的内部框架下进行评估，并采用相同的评估设置。分数差距不超过0.3视为同一水平。每行最高分以粗体显示，次高分以下划线标注。

Benchmark (Metric)		# Shots	DeepSeek-V3.2 Base	DeepSeek-V4-Flash Base	DeepSeek-V4-Pro Base
Architecture		-	MoE	MoE	MoE
# Activated Params		-	37B	13B	49B
# Total Params		-	671B	284B	1.6T
World Knowl.	AGIEval (EM)	0-shot	80.1	<u>82.6</u>	83.1
	MMLU (EM)	5-shot	87.8	<u>88.7</u>	90.1
	MMLU-Redux (EM)	5-shot	87.5	<u>89.4</u>	90.8
	MMLU-Pro (EM)	5-shot	65.5	<u>68.3</u>	73.5
	MMMLU (EM)	5-shot	87.9	<u>88.8</u>	90.3
	C-Eval (EM)	5-shot	90.4	<u>92.1</u>	93.1
	CMMLU (EM)	5-shot	88.9	<u>90.4</u>	90.8
	MultiLoKo (EM)	5-shot	38.7	<u>42.2</u>	51.1
	Simple-QA verified (EM)	25-shot	28.3	<u>30.1</u>	55.2
	SuperGPQA (EM)	5-shot	45.0	<u>46.5</u>	53.9
	FACTS Parametric (EM)	25-shot	27.1	<u>33.9</u>	62.6
	TriviaQA (EM)	5-shot	<u>83.3</u>	<u>82.8</u>	85.6
Lang. & Reas.	BBH (EM)	3-shot	87.6	<u>86.9</u>	87.5
	DROP (F1)	1-shot	<u>88.2</u>	88.6	88.7
	HellaSwag (EM)	0-shot	<u>86.4</u>	85.7	88.0
	WinoGrande (EM)	0-shot	<u>78.9</u>	<u>79.5</u>	81.5
	CLUEWSC (EM)	5-shot	<u>83.5</u>	<u>82.2</u>	85.2
Code & Math	BigCodeBench (Pass@1)	3-shot	63.9	56.8	<u>59.2</u>
	HumanEval (Pass@1)	0-shot	62.8	<u>69.5</u>	76.8
	GSM8K (EM)	8-shot	<u>91.1</u>	90.8	92.6
	MATH (EM)	4-shot	<u>60.5</u>	57.4	64.5
	MGSM (EM)	8-shot	<u>81.3</u>	85.7	<u>84.4</u>
	CMath (EM)	3-shot	<u>92.6</u>	93.6	90.9
Long Context	LongBench-V2 (EM)	1-shot	40.2	<u>44.7</u>	51.5

5. 训练后

5.1. 训练后流程

在预训练之后，我们进行了后训练阶段，以产出DeepSeek-V4系列的最终模型。尽管训练流程大体沿用了DeepSeek-V3.2的框架，但一个关键的方法论替换是：混合强化学习（RL）阶段被完全替换为在线策略蒸馏（OPD；Gu等人，2024；Lu和Lab，2025）。

5.1.1. Specialist Training

领域专家的培养是通过调整DeepSeek-V3.2的训练流程实现的。具体而言，每个模型依次经历了初始微调阶段和后续由领域特定提示与奖励信号引导的强化学习（RL）优化。在强化学习阶段，我们采用了分组相对策略优化（GRPO）算法，保持超参数与先前研究（DeepSeek-AI, 2025; DeepSeek-AI, 2025）高度一致。

Reasoning Efforts. It is widely recognized that a model’s performance on reasoning tasks is fundamentally governed by the computational effort expended. Consequently, we trained distinct specialist models under divergent RL configurations to facilitate the development of models optimized for varying reasoning capacities. As detailed in Table 2, DeepSeek-V4-Pro and DeepSeek-V4-Flash both support three specific reasoning effort modes. For each mode, we apply distinct length penalties and context windows during RL training, which results in varying output token lengths for reasoning. To integrate these distinct reasoning modes, we utilize specialized response formats demarcated by the `<think>` and `</think>` tokens. Furthermore, for the "Think Max" mode, we prepend a specific instruction to the beginning of the system prompt to guide the model’s reasoning process, as shown in Table 3.

Table 2 | Comparison of three reasoning modes

Reasoning Mode	Characteristics	Typical Use Cases	Response Format
Non-think	Fast, intuitive responses based on habits or simple rules.	Routine daily tasks, emergency reactions, low-risk decisions.	<code></think></code> summary
Think High	Conscious logical analysis, slower but more accurate.	Complex problem-solving, planning, medium-risk decisions.	<code><think></code> thinking tokens <code></think></code> summary
Think Max	Push reasoning to its fullest extent. Slow but powerful.	Exploring the boundary of model reasoning capability.	1. A special system prompt at the beginning. 2. <code><think></code> thinking tokens <code></think></code> summary

Table 3 | Instruction injected into the system prompt for the "Think Max" mode.

Injected Instruction

Reasoning Effort: Absolute maximum with no shortcuts permitted.
You MUST be very thorough in your thinking and comprehensively decompose the problem to resolve the root cause, rigorously stress-testing your logic against all potential paths, edge cases, and adversarial scenarios.
Explicitly write out your entire deliberation process, documenting every intermediate step, considered alternative, and rejected hypothesis to ensure absolutely no assumption is left unchecked.

Generative Reward Model. Typically, easy-to-verify tasks can be effectively optimized using simple rule-based verifiers or test cases. In contrast, hard-to-verify tasks traditionally rely on Reinforcement Learning from Human Feedback (RLHF), which necessitates extensive human annotation to train a scalar reward model. In the post-training phase of DeepSeek-V4 series,

推理努力。业界普遍认为，模型在推理任务上的表现从根本上取决于所投入的计算努力。因此，我们通过不同的强化学习配置训练了多个专用模型，以促进针对不同推理能力的优化模型开发。如表2所述，DeepSeek-V4-Pro和DeepSeek-V4-Flash均支持三种特定的推理努力模式。针对每种模式，我们在强化学习训练中应用不同的长度惩罚和上下文窗口，从而导致推理输出令牌长度的变化。为了整合这些不同的推理模式，我们采用了由{v*}和{v*}令牌界定的专用响应格式。此外，对于"Think Max"模式，我们在系统提示开头添加特定指令以引导模型的推理过程，如表3所示。

表2 | 三种推理模式的比较

Reasoning Mode	Characteristics	Typical Use Cases	Response Format
Non-think	Fast, intuitive responses based on habits or simple rules.	Routine daily tasks, emergency reactions, low-risk decisions.	</think> summary
Think High	Conscious logical analysis, slower but more accurate.	Complex problem-solving, planning, medium-risk decisions.	<think> thinking tokens </think> summary
Think Max	Push reasoning to its fullest extent. Slow but powerful.	Exploring the boundary of model reasoning capability.	1. A special system prompt at the beginning. 2. <think> thinking tokens </think> summary

表3 | 注入系统提示的指令，用于“Think Max”模式。

注入指令

推理努力：绝对最大化，不允许任何捷径。你必须非常彻底地思考，全面分解问题以解决根本原因，严格地对所有潜在路径、边缘案例和对抗场景进行逻辑压力测试。明确写出你的整个思考过程，记录每一个中间步骤、考虑过的替代方案和被否定的假设，确保没有任何假设未经检验。

生成式奖励模型。通常，易于验证的任务可以通过基于规则的简单验证器或测试用例有效优化。相比之下，难以验证的任务传统上依赖人类反馈强化学习（RLHF），这需要大量人工标注来训练标量奖励模型。在DeepSeek-V4系列的训练后阶段，

however, we dispense with these conventional scalar-based reward models. Instead, to address hard-to-verify tasks, we curate rubric-guided RL data and employ a Generative Reward Model (GRM) to evaluate policy trajectories. Crucially, we apply RL optimization directly to the GRM itself. In this paradigm, the actor network natively functions as the GRM, enabling the joint optimization of the model’s evaluative (judging) proficiency alongside its standard generative capabilities. By unifying these roles, the model’s internal reasoning capabilities are inherently fused into its evaluative process, resulting in highly robust scoring. Furthermore, this approach achieves superior performance with only a minimal set of diverse human annotations, as the model leverages its own logic to generalize across complex tasks.

Table 4 | Tool-call schema for DeepSeek-V4 series.

Tool Call Schema
<pre> ## Tools You have access to a set of tools to help answer the user’s question. You can invoke tools by writing a "< DSML tool_calls>" block like the following: < DSML tool_calls> < DSML invoke name="\$TOOL_NAME"> < DSML parameter name="\$PARAMETER_NAME" string="true false">\$PARAMETER_VALUE </ DSML parameter> ... </ DSML invoke> < DSML invoke name="\$TOOL_NAME2"> ... </ DSML invoke> </ DSML tool_calls> String parameters should be specified as is and set ‘string="true"’. For all other types (numbers, booleans, arrays, objects), pass the value in JSON format and set ‘string="false"’. If thinking_mode is enabled (triggered by <think>), you MUST output your complete reasoning inside <think>...</think> BEFORE any tool calls or final response. Otherwise, output directly after </think> with tool calls or final response. ### Available Tool Schemas {Tool Definition...} You MUST strictly follow the above definedtool name and parameter schemas to invoke tool calls.</pre>

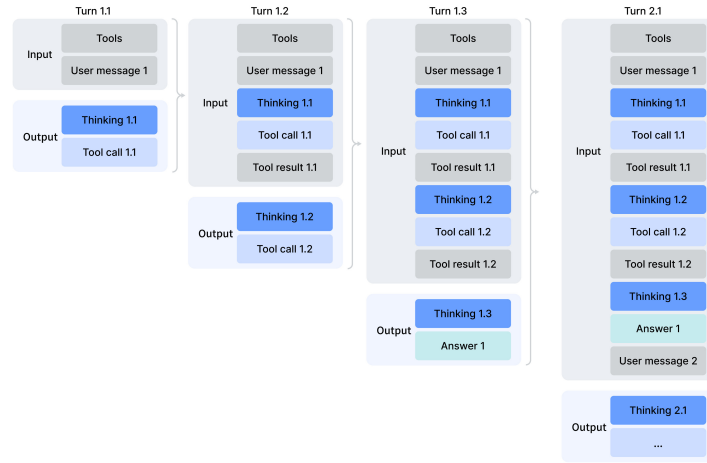
Tool-Call Schema and Special Token. Consistent with our previous version, we utilize a dedicated `<think></think>` tag to delineate the reasoning path. In DeepSeek-V4 series, we introduce a new tool-call schema that employs a special `"|DSML|"` token and utilizes an XML-based format for tool invocations, as demonstrated in Table 4. Our experiments demonstrate that the XML format effectively mitigates escaping failures and reduces tool-call errors, providing a more robust interface for model-tool interactions.

然而，我们摒弃了这些传统的基于标量的奖励模型。针对难以验证的任务，我们转而精心构建了基于评分标准的强化学习数据，并采用生成式奖励模型（GRM）来评估策略轨迹。关键在于，我们直接将强化学习优化应用于GRM本身。在此范式中，行动者网络天然地作为GRM运作，使得模型在保持标准生成能力的同时，也能联合优化其评估（判断）能力。通过统一这两种角色，模型的内在推理能力被自然地融合到评估流程中，从而产生极其稳健的评分结果。此外，该方法仅需少量多样化的人工标注即可实现卓越性能，因为模型能够运用自身逻辑来泛化处理复杂任务。

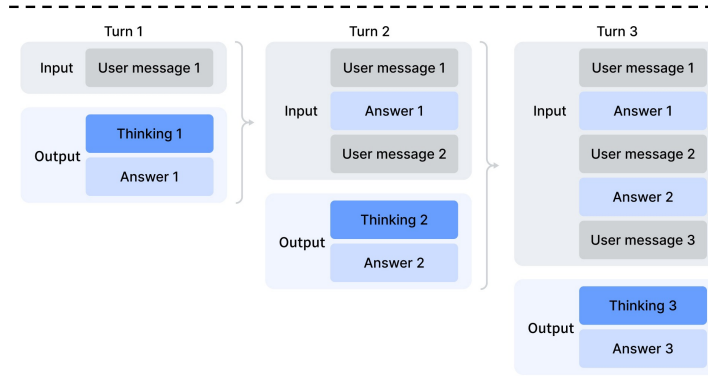
表4 | DeepSeek-V4系列的Tool-call模式。

Tool Call Schema
<pre> ## Tools You have access to a set of tools to help answer the user's question. You can invoke tools by writing a "< DSML tool_calls>" block like the following: < DSML tool_calls> < DSML invoke name="\$TOOL_NAME"> < DSML parameter name="\$PARAMETER_NAME" string="true false">\$PARAMETER_VALUE </ DSML parameter> ... </ DSML invoke> < DSML invoke name="\$TOOL_NAME2"> ... </ DSML invoke> </ DSML tool_calls> String parameters should be specified as is and set 'string="true"'. For all other types (numbers, booleans, arrays, objects), pass the value in JSON format and set 'string="false"'. If thinking_mode is enabled (triggered by <think>), you MUST output your complete reasoning inside <think>...</think> BEFORE any tool calls or final response. Otherwise, output directly after </think> with tool calls or final response. ### Available Tool Schemas {Tool Definition...} You MUST strictly follow the above defined tool name and parameter schemas to invoke tool calls. </pre>

工具调用模式与特殊标记。与我们之前的版本保持一致，我们采用专用的<think></think>标签来划分推理路径。在DeepSeek-V4系列中，我们引入了一种新的工具调用模式，该模式采用特殊的"|DSML|"标记，并使用基于XML的格式进行工具调用，如表4所示。实验证明，XML格式能有效避免转义失败并减少工具调用错误，为模型与工具间的交互提供了更稳健的接口。



a) Thinking with tools



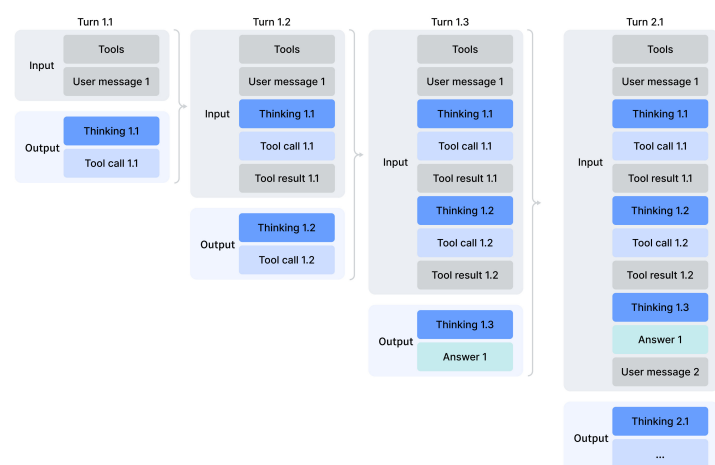
b) Thinking without tools

Figure 7 | Thinking management of DeepSeek-V4 series.

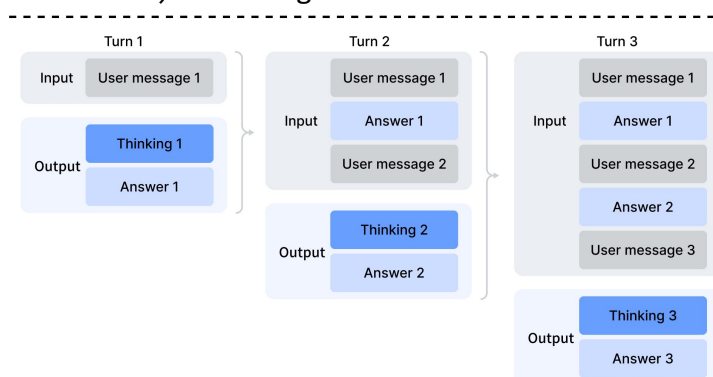
Interleaved Thinking. DeepSeek-V3.2 introduced a context management strategy that retains reasoning traces across tool-result rounds but discards them upon the arrival of new user messages. While effective, this still caused unnecessary token waste in complex agentic workflows — each new user turn would flush all accumulated reasoning content, forcing the model to reconstruct its problem-solving state from scratch. Leveraging the expanded 1M-token context window of DeepSeek-V4 series, we further refine this mechanism to maximize the effectiveness of interleaved thinking in agentic environments:

- **Tool-Calling Scenarios.** As illustrated in Figure 7(a), all reasoning content is fully preserved throughout the entire conversation. Unlike DeepSeek-V3.2, which discarded thinking traces upon each new user turn, DeepSeek-V4 series retain the complete reasoning history across all rounds, including across user message boundaries. This allows the model to maintain a coherent, cumulative chain of thought over long-horizon agent tasks.
- **General Conversational Scenarios.** As illustrated in Figure 7(b), the original strategy is preserved: reasoning content from previous turns is discarded when a new user message arrives, keeping the context concise for settings where persistent reasoning traces provide limited benefit.

As with DeepSeek-V3.2, agent frameworks that simulate tool interactions via user messages (e.g., Terminus) may not trigger the tool-calling context path and thus may not benefit from enhanced



a) Thinking with tools



b) Thinking without tools

图7 | DeepSeek-V4系列的思维管理。

交错式思考。DeepSeek-V3.2引入了一种上下文管理策略，该策略在工具-结果轮次间保留推理痕迹，但在新用户消息到达时将其丢弃。虽然有效，但这在复杂的代理 workflows 中仍导致不必要的令牌浪费——每个新的用户回合都会清空所有累积的推理内容，迫使模型从零开始重建其问题解决状态。借助DeepSeek-V4系列扩展的1M令牌上下文窗口，我们进一步优化了这一机制，以最大化代理环境中交错式思考的效能：

- 工具调用场景。如图7(a)所示，整个对话过程中所有推理内容都被完整保留。与DeepSeek-V3.2在每个新用户轮次丢弃思考痕迹不同，DeepSeek-V4系列保留了跨越所有轮次的完整推理历史，包括跨越用户消息边界的情况。这使得模型能够在长周期智能体任务中保持连贯、累积的思维链。
- 通用对话场景。如图7(b)所示，原始策略得以保留：当新用户消息到达时，先前轮次的推理内容会被丢弃，从而在持久化推理痕迹收益有限的场景中保持上下文简洁。

与DeepSeek-V3.2类似，通过用户消息模拟工具交互的代理框架（如Terminus）可能无法触发工具调用上下文路径，因此可能无法受益于增强的

reasoning persistence. We continue to recommend non-think models for such architectures.

Table 5 | Quick Instruction special tokens for auxiliary tasks.

Special Token	Description	Format
< action >	Determines whether the user prompt requires a web search or can be answered directly.	...< User >{prompt}< Assistant > <think>< action >
< title >	Generates a concise conversation title after the first assistant response.	...< Assistant >{response} < end_of_sentence >< title >
< query >	Generates search queries for the user prompt.	...< User >{prompt}< query >
< authority >	Classifies the user prompt’s demand for source authoritativeness.	...< User >{prompt}< authority >
< domain >	Identifies the domain of the user prompt.	...< User >{prompt}< domain >
< extracted_url > < read_url >	Determines whether each URL in the user prompt should be fetched and read.	...< User >{prompt} < extracted_url >{url} < read_url >

Quick Instruction. In chatbot scenarios, a number of auxiliary tasks (e.g., determining whether to trigger a web search, intent recognition, etc.) must be executed before generating the response. Conventionally, these tasks are handled by a separate small model, requiring redundant prefilling since it cannot reuse the existing KV cache. To overcome this limitation, we introduce Quick Instruction. We append a set of dedicated special tokens directly to the input sequence, where each token corresponds to a specific auxiliary task. By directly reusing the already-computed KV cache, this mechanism completely avoids redundant prefilling and allows certain tasks, such as generating search queries and determining authority and domain, to be executed in parallel. Consequently, this approach significantly reduces the user-perceived time-to-first-token (TTFT) and eliminates the engineering overhead of maintaining and iterating an extra small model. The supported Quick Instruction tokens are summarized in Table 5.

5.1.2. On-Policy Distillation

After training multiple domain-specific experts via specialized fine-tuning and reinforcement learning, we employ multi-teacher On-Policy Distillation (OPD; Gu et al. 2024; Lu and Lab 2025) as the primary technique for merging expert capabilities into the final model. OPD has emerged as an effective post-training paradigm for efficiently transferring the knowledge and capabilities of domain experts to a single, unified model. This is achieved by having the student learn from the output distributions of teacher models on its own generated trajectories. Formally, given a set of N expert models $\{\pi_{E_1}, \pi_{E_2}, \dots, \pi_{E_N}\}$, the OPD objective function is defined as:

$$\mathcal{L}_{\text{OPD}}(\theta) = \sum_{i=1}^N w_i \cdot \text{D}_{\text{KL}}(\pi_{\theta} \parallel \pi_{E_i}). \quad (29)$$

In this formulation, w_i represents the assigned weight for each expert, typically determined by the relative importance of the expert. Computing the reverse KL loss $\text{D}_{\text{KL}}(\pi_{\theta} \parallel \pi_{E_i})$ requires

推理持久性。对于此类架构，我们仍然推荐非思考模型。

表5 | 辅助任务的快速指令特殊标记。

Special Token	Description	Format
< action >	Determines whether the user prompt requires a web search or can be answered directly.	...< User >{prompt}< Assistant > <think>< action >
< title >	Generates a concise conversation title after the first assistant response.	...< Assistant >{response} < end_of_sentence >< title >
< query >	Generates search queries for the user prompt.	...< User >{prompt}< query >
< authority >	Classifies the user prompt's demand for source authority.	...< User >{prompt}< authority >
< domain >	Identifies the domain of the user prompt.	...< User >{prompt}< domain >
< extracted_url > < read_url >	Determines whether each URL in the user prompt should be fetched and read.	...< User >{prompt} < extracted_url >{url} < read_url >

快速指令。在聊天机器人场景中，生成响应前需执行多项辅助任务（如判断是否触发网络搜索、意图识别等）。传统做法需依赖独立的小模型处理这些任务，由于无法复用现有KV缓存，导致冗余的预填充计算。为突破这一限制，我们引入快速指令机制：直接在输入序列后追加一组专用特殊标记，每个标记对应特定辅助任务。通过直接复用已计算的KV缓存，该机制彻底避免冗余预填充，并支持搜索查询生成、权限与领域判定等任务并行执行。因此，该方法显著降低用户感知的首令牌延迟（TTFT），同时省去维护和迭代额外小模型的工程开销。支持的快速指令标记详见表5。

5.1.2. On-Policy Distillation

在通过专业化微调与强化学习训练出多个领域专家后，我们采用多教师同策略蒸馏（OPD；Gu等，2024；Lu与Lab，2025）作为核心技术，将专家能力融合至最终模型中。OPD已成为一种高效的训练后范式，能有效将领域专家的知识与能力迁移至单一统一模型。其实现方式是让学生模型从教师模型在其自身生成轨迹上的输出分布中学习。形式化地，给定一组 N 专家模型 $\{\pi_{E_1}, \pi_{E_2}, \dots, \pi_{E_N}\}$ ，OPD目标函数定义为：

$$\mathcal{L}_{\text{OPD}}(\theta) = \sum_{i=1}^N w_i \cdot \text{D}_{\text{KL}}(\pi_{\theta} \parallel \pi_{E_i}). \quad (29)$$

在这一表述中， w_i 代表为每位专家分配的权重，通常由专家的相对重要性决定。计算反向KL散度 $\text{D}_{\text{KL}}(\pi_{\theta} \parallel \pi_{E_i})$ 需要

sampling training trajectories from the student π_θ to maintain on-policy learning. The underlying logic ensures that the unified policy π_θ selectively learns from the specialized expert relevant to the current task context (e.g., aligning with the mathematics expert for math reasoning tasks and the coding expert for programming tasks). Through this mechanism, the knowledge from physically distinct expert weights is consolidated into a unified parameter space via logits-level alignment, practically circumventing the performance degradation often encountered in traditional weight-merging or mixed RL techniques. In this stage, more than ten teacher models covering various domains are employed to distill a single student model.

In handling the above OPD objective, prior works usually simplify the full-vocabulary KL loss into a token-level KL estimate at each token position, and reuse RL framework by replacing $\text{sg}[\log \frac{\pi_{E_t}(y_t|x, y_{<t})}{\pi_\theta(y_t|x, y_{<t})}]$ (sg represents the stop gradient operation) as the per-token advantage estimate in the policy loss calculation. Although this approach is resource-efficient, it leads to high variance in gradient estimation and often causes training instability. Therefore, we adopt full-vocabulary logit distillation in our OPD. Preserving the complete logit distribution in calculating reverse KL loss yields more stable gradient estimates and ensures faithful distillation of the teachers’ knowledge. In the following subsection, we describe the engineering efforts that make full-vocabulary OPD feasible at scale.

5.2. Post-Training Infrastructures

Our post-training infrastructure is built upon the scalable framework developed for DeepSeek-V3.2. Specifically, we integrate the same distributed training stack described in Section 3.4 and the rollout engine introduced earlier for efficient auto-regressive sampling. Building on this foundation, we introduce the following principal enhancements in the present work. These designs enable efficient execution of ultra-long-context RL and OPD merging tasks involving over ten distinct teacher models, thereby substantially accelerating the iteration cycle for model releases.

5.2.1. FP4 Quantization-Aware Training

To achieve inference acceleration and reducing memory traffic at deployment, we introduce Quantization-Aware Training (QAT) (Jacob et al., 2018) during the post-training stage, enabling the model, including those of teacher and reference models, to adapt to the precision degradation introduced by quantization. We apply FP4 (MXFP4) quantization (Rouhani et al., 2023) to two components: (1) MoE expert weights, which are a major source of GPU memory occupancy (OpenAI, 2025), and (2) the Query-Key (QK) path in the indexer of CSA, where QK activations are cached, loaded, and multiplied entirely in FP4, accelerating attention score computation in long-context scenarios. In addition, we further quantize the index scores $I_{:,}$ from FP32 to BF16 during this QAT process. This optimization achieves a 2× speedup for the top-k selector, while preserving a 99.7% recall rate of KV entries.

For MoE expert weights, following the common practice of QAT, the FP32 master weights maintained by the optimizer are first quantized to FP4, then dequantized back to FP8 for computation. Notably, our FP4-to-FP8 dequantization is lossless. This is because FP8 (E4M3) has 2 additional exponent bits compared with FP4 (E2M1), offering a larger dynamic range. Consequently, as long as the ratio between the maximum and minimum scale factors of the FP4 sub-blocks (1×32 tiles) within each FP8 quantization block (128×128 tiles) does not exceed a certain threshold, the fine-grained scale information can be fully absorbed by the extended dynamic range of FP8. We empirically verify that current weights satisfy this condition. This

从学生模型 π_θ 中采样训练轨迹以维持在线策略学习。其内在逻辑确保统一策略 π_θ 能够根据当前任务情境选择性地从相关专业专家处学习（例如，数学推理任务与数学专家对齐，编程任务则与编程专家对齐）。通过这一机制，来自物理上独立专家权重的知识经由逻辑层面(logits-level)的对齐，被整合到一个统一的参数空间中，从而有效规避了传统权重合并或混合强化学习技术中常见的性能下降问题。在此阶段，采用了涵盖多个领域的十余个教师模型来蒸馏出一个单一的学生模型。

在处理上述OPD目标时，先前的研究通常将全词汇表的KL损失简化为每个词符位置上的词符级KL估计，并通过在策略损失计算中将 $\text{sg}\left[\log \frac{\pi_{E_t}(y_t|x_t, y_{<t})}{\pi_\theta(y_t|x_t, y_{<t})}\right]$ （sg（表示停止梯度操作））替换为单词优势估计来复用强化学习框架。尽管这种方法资源效率高，但它会导致梯度估计的高方差，并常常引发训练不稳定性。因此，我们在OPD中采用了全词汇表逻辑蒸馏。在计算反向KL损失时保留完整的逻辑分布，能产生更稳定的梯度估计，并确保忠实蒸馏教师模型的知识。在接下来的小节中，我们将描述使大规模全词汇表OPD成为可能的工程实现努力。

5.2. 训练后基础设施

我们的训练后基础设施建立在为DeepSeek-V3.2开发的可扩展框架之上。具体而言，我们整合了第3.4节所述的相同分布式训练堆栈以及先前推出的rollout引擎，以实现高效的自回归采样。在此基础上，我们在当前工作中引入了以下主要改进。这些设计使得涉及十余种不同教师模型的超长上下文RL和OPD合并任务能够高效执行，从而大幅加速模型发布的迭代周期。

5.2.1. FP4 Quantization-Aware Training

为实现部署时的推理加速与内存流量降低，我们在训练后阶段引入了量化感知训练（QAT）（Jacob等人，2018），使模型（包括教师模型和参考模型）能够适应量化带来的精度下降。我们对两个组件应用FP4（MXFP4）量化（Rouhani等人，2023）：(1) MoE专家权重——这是GPU内存占用的主要来源（OpenAI，2025）；(2) CSA索引器中的查询-键（QK）路径，该路径的QK激活值以FP4格式全程缓存、加载并相乘，从而加速长上下文场景下的注意力得分计算。此外，我们在此QAT过程中进一步将索引分数 $I_{:,j}$ 从FP32量化为BF16。该优化使top-k选择器获得2×倍加速，同时保持99.7%的KV条目召回率。

对于MoE专家权重，遵循QAT的常规做法，首先将优化器维护的FP32主权重量化为FP4，随后反量化为FP8用于计算。值得注意的是，我们的FP4至FP8反量化是无损的。这是因为FP8（E4M3）相比FP4（E2M1）具有额外的2个指数位，提供了更大的动态范围。因此，只要每个FP8量化块（128×128分块）内FP4子块（1×32分块）的最大与最小缩放因子之比不超过特定阈值，细粒度的缩放信息就能被FP8扩展的动态范围完全吸收。我们通过实验验证当前权重满足这一条件。

allows the entire QAT pipeline to fully reuse the existing FP8 training framework without any modification. In the backward pass, gradients are computed with respect to the same FP8 weights in the forward pass and directly propagated back to the FP32 master weights, equivalent to applying the Straight-Through Estimator (STE) through the quantization operation. This also avoids the need to re-quantize transposed weights.

During the inference and rollout phases of RL training, which do not involve backward passes, we directly use native FP4 quantized weights instead of simulated quantization. This ensures that model behavior during sampling is fully consistent with online deployment, while also reducing kernel memory loading for actual speedup and significantly lowering memory consumption. We process the QK path in the indexer of CSA similarly.

5.2.2. *Efficient Teacher Scheduling for Full-Vocabulary OPD*

Our framework supports full-vocabulary On-Policy Distillation (OPD) with an effectively unbounded number of teachers, each potentially comprising trillions of parameters. To enable this, all teacher weights are offloaded to a centralized distributed storage and are loaded on demand during the teacher forward pass with ZeRO-like parameter sharding to alleviate both I/O and DRAM pressure. Furthermore, naively materializing logits for a vocabulary size $|V| > 100k$ across all teachers is prohibitive, even when spooled to disk. We address this by caching only the last-layer teacher hidden states in a centralized buffer during the forward pass. At training time, these cached states are retrieved and passed through the corresponding prediction head module to reconstruct the full logits on the fly. This design incurs negligible recomputation overhead while completely circumventing the memory burden associated with explicit logits materialization. To mitigate the GPU memory footprint of the teacher prediction head, we order training samples by teacher index during data dispatching. This arrangement ensures that each distinct teacher head is loaded only once per mini-batch and that at most one teacher head resides in device memory at any given time. All parameters and hidden state loading/offloading operations proceed asynchronously in the background, without blocking computation on the critical path. Finally, the exact KL divergences between teacher and student logits are computed using a specialized TileLang kernel, which accelerates the computation and curtails dynamic memory allocation.

5.2.3. *Preemptible and Fault-Tolerant Rollout Service*

To maximize GPU resource utilization while enabling rapid hardware provisioning for high-priority tasks, our GPU cluster employs a cluster-wide preemptive task scheduler, where any running task may be preempted at any time. Also, hardware failures are prevalent in large-scale GPU clusters. To this end, we implement a preemptible and fault-tolerant LLM generation service for RL/OPD rollout.

Specifically, we implement a token-granular Write-Ahead Log (WAL) for each generation request. Whenever a new token is generated for a request, we immediately append it to that request’s WAL. During preemption, we pause the inference engine and save the KV cache of unfinished requests. Upon resumption, we use the persisted WALs and saved KV cache to continue decoding. Even when a fatal hardware error occurs, we can re-run the prefill phase using the persisted tokens in WAL to reconstruct the KV cache.

Importantly, it is mathematically incorrect to regenerate unfinished requests from scratch, as this introduces length bias. Because shorter responses are more likely to survive interruption, regenerating from scratch makes the model more prone to producing shorter sequences

使得整个QAT流程能够完全复用现有的FP8训练框架，无需任何修改。在反向传播过程中，梯度是根据前向传播中相同的FP8权重计算的，并直接回传到FP32主权重，相当于通过量化操作应用了直通估计器（STE）。这也避免了重新量化转置权重的需求。

在RL训练的推理与推演阶段，由于不涉及反向传播过程，我们直接采用原生FP4量化权重而非模拟量化。此举确保了采样阶段的模型行为与线上部署完全一致，同时减少了核心内存加载以实现实际加速，并显著降低了内存消耗。对于CSA索引器中的QK路径，我们同样采用此处理方式。

5.2.2. *Efficient Teacher Scheduling for Full-Vocabulary OPD*

我们的框架支持全词汇表规模的同策略蒸馏（OPD），教师模型数量理论上无上限，每个模型可能包含数万亿参数。为实现这一点，所有教师权重均卸载至中央分布式存储系统，并在教师前向传播时按需加载，采用类ZeRO的参数分片技术以缓解I/O和DRAM压力。此外，即使将数据暂存至磁盘，为所有教师模型实例化 $|V| > 10$ 万量级词汇表的logits也极其困难。我们通过在正向传播期间仅将教师最后一层隐藏状态缓存至中央缓冲区来解决该问题。训练时，这些缓存状态被检索并通过对应的预测头模块动态重构完整logits。该设计在完全规避显式logits实例化内存负担的同时，仅引入可忽略的重计算开销。为降低教师预测头的GPU内存占用，我们在数据分发时按教师索引对训练样本排序。这种安排确保每个独立教师预测头每小批次仅加载一次，且任一时刻设备内存中至多驻留一个教师预测头。所有参数和隐藏状态的加载/卸载操作均在后台异步执行，不阻塞关键路径上的计算。最终，教师与学生logits间的精确KL散度通过专用TileLang内核计算，该内核加速了运算过程并抑制了动态内存分配。

5.2.3. *Preemptible and Fault-Tolerant Rollout Service*

为了在最大化GPU资源利用率的同时，为高优先级任务实现快速硬件配置，我们的GPU集群采用了集群级抢占式任务调度器，任何运行中的任务都可能随时被抢占。此外，大规模GPU集群中硬件故障频发。为此，我们为RL/OPD rollout实现了一个可抢占且容错的LLM生成服务。

具体而言，我们为每个生成请求实现了令牌粒度的预写日志（WAL）。每当为请求生成一个新令牌时，我们立即将其追加到该请求的WAL中。在抢占期间，我们会暂停推理引擎并保存未完成请求的KV缓存。恢复时，我们利用持久化的WAL和保存的KV缓存继续解码。即使发生致命硬件错误，我们也能通过WAL中持久化的令牌重新运行预填充阶段，以重建KV缓存。

重要的是，从未完成的请求从头开始重新生成在数学上是错误的，因为这引入了长度偏差。由于较短的响应更有可能在中断后保留，从头开始重新生成会使模型更容易产生较短的序列

whenever an interruption occurs. If the inference stack is batch-invariant and deterministic, this correctness issue could also be addressed by regenerating with a consistent seed for the pseudorandom number generator used in the sampler. However, this approach still incurs the extra cost of re-running the decoding phase, making it far less efficient than our token-granular WAL method.

5.2.4. *Scaling RL Framework for Million-Token Context*

We introduce targeted optimizations for efficient RL and OPD on million-token sequences. During the rollout phase, we adopt a preemptible and fault-tolerant rollout service, detailed in Section 5.2.3. For the inference and training phase, we decompose the rollout data format into lightweight metadata and heavy per-token fields. During data dispatching, the metadata for the entire rollout data can be loaded to perform global shuffling and packing layout computation. Heavy per-token fields are loaded via a shared-memory data loader to eliminate intra-node data redundancy and are released immediately upon consumption at the mini-batch granularity, substantially reducing both CPU and GPU memory pressure. The number of on-device mini-batches is dynamically determined based on workload, allowing an efficient trade-off between computational throughput and I/O overlap.

5.2.5. *Sandbox Infrastructure for Agentic AI*

To meet the diverse execution demands of agentic AI during post-training and evaluation, we build a production-grade sandbox platform, **DeepSeek Elastic Compute (DSec)**. DSec comprises three Rust components — the API gateway (**Apiserver**), per-host agent (**Edge**), and the cluster monitor (**Watcher**) — that are interconnected by a custom RPC protocol and scale horizontally atop the 3FS distributed filesystem (DeepSeek-AI, 2025). In production, a single DSec cluster manages hundreds of thousands of concurrent sandbox instances.

The design of DSec is motivated by four observations: (1) agentic workloads are highly heterogeneous, spanning lightweight function calls to full software-engineering pipelines with diverse OS and security requirements; (2) environment images are numerous and large, yet must load quickly and support iterative customization; (3) high-density deployment demands efficient CPU and memory utilization; (4) sandbox lifecycles must coordinate with GPU training schedules, including preemption and checkpoint-based resumption. Based on these observations, we elaborate on the four core designs of DSec individually in the following.

Four Execution Substrates Behind One Unified Interface. DSec exposes a single Python SDK (**libdsec**) that abstracts four execution substrates. **Function Call** dispatches stateless invocations to a pre-warmed container pool, eliminating cold-start overhead. **Container** is fully Docker-compatible and leverages EROFS (Gao et al., 2019) on-demand loading for efficient image assembly. **microVM**, built on Firecracker (Agache et al., 2020), adds VM-level isolation for security-sensitive, high-density deployments. **fullVM**, built on QEMU (Bellard, 2005), supports arbitrary guest operating systems. All four share a common API surface — command execution, file transfer, and TTY access — and switching between them requires only a parameter change.

Fast Image Loading via Layered Storage. DSec reconciles fast startup with a large and growing corpus of environment images through layered, on-demand loading. For containers, base images and filesystem commits are stored as 3FS-backed readonly EROFS layers mounted directly into overlay lowerdirs. We keep file metadata readily available on the local disk at mount time;

每当发生中断时。如果推理堆栈是批次不变且确定性的，这种正确性问题也可以通过为采样器中使用的伪随机数生成器设定一致的种子来重新生成而解决。然而，这种方法仍需承担重新运行解码阶段的额外成本，使其效率远低于我们的令牌粒度WAL方法。

5.2.4. *Scaling RL Framework for Million-Token Context*

我们针对百万令牌序列上的高效强化学习（RL）和在线策略蒸馏（OPD）引入了定向优化。在推演阶段，我们采用了一种可抢占且容错的推演服务，具体细节见5.2.3节。对于推理和训练阶段，我们将推演数据格式分解为轻量级元数据和重量级的每令牌字段。在数据分发过程中，可加载整个推演数据的元数据以执行全局混洗和打包布局计算。重量级的每令牌字段通过共享内存数据加载器加载，以消除节点内数据冗余，并在小批量粒度消费后立即释放，显著降低了CPU和GPU内存压力。设备端小批量数量根据工作负载动态确定，实现了计算吞吐量与I/O重叠之间的高效权衡。

5.2.5. *Sandbox Infrastructure for Agentic AI*

为满足智能体AI在后训练及评估阶段的多样化执行需求，我们构建了一个生产级沙箱平台——DeepSeek弹性计算（DSec）。DSec由三个Rust组件构成：API网关（Apiserver）、单机代理（Edge）和集群监控器（Watcher），它们通过定制RPC协议互联，并基于3FS分布式文件系统（DeepSeek-AI, 2025）实现横向扩展。在生产环境中，单个DSec集群可管理数十万个并发沙箱实例。

DSec的设计动机源于四个观察：(1) 代理工作负载高度异构，涵盖从轻量级函数调用到具有多样化操作系统和安全需求的完整软件工程流水线；(2) 环境镜像数量庞大且体积巨大，却需快速加载并支持迭代定制；(3) 高密度部署要求高效的CPU和内存利用率；(4) 沙盒生命周期必须与GPU训练计划协调，包括抢占和基于检查点的恢复。基于这些观察，我们将在下文逐一阐述DSe c的四个核心设计。

一个统一接口背后的四种执行基础。DSec 提供了一个单一的Python SDK（libdsec），它抽象了四种执行基础。Function Call 将无状态调用分派到预热的容器池，消除了冷启动开销。Container 完全兼容Docker，并利用EROFS（Gao等人，2019年）的按需加载技术实现高效的镜像组装。microVM基于Firecracker（Agache等人，2020年）构建，为安全敏感、高密度的部署场景增加了虚拟机级别的隔离。fullVM则基于QEMU（Bellard，2005年）构建，支持任意客户操作系统。这四种基础共享一套通用API接口——命令执行、文件传输和TTY访问——切换它们仅需更改一个参数。

快速图像加载通过分层存储实现。DSec通过分层按需加载机制，在快速启动与庞大且不断增长的环境镜像库之间取得平衡。对于容器而言，基础镜像和文件系统提交被存储为3FS支持的只读EROFS层，直接挂载到覆盖层lowerdir中。我们在挂载时确保文件元数据可即时从本地磁盘获取；

meanwhile, data blocks are fetched from 3FS upon request. For microVMs, DSec uses the overlaybd (Li et al., 2020) disk format: the read-only base layer resides on 3FS for cross-instance sharing, while writes go to a local copy-on-write layer. Such snapshots are chainable, facilitating efficient versioning and millisecond-scale resumption.

Density Optimizations Under Massive Concurrency. To accommodate hundreds of thousands of sandboxes per cluster, DSec tackles two resource bottlenecks. First, it mitigates duplicate page-cache footprints in virtualized environments and applies memory reclamation to enable safe overcommitment. Second, it alleviates spinlock contention in the container runtime and therefore, reduces per-sandbox CPU overhead, significantly increasing per-host packing density.

Trajectory Logging and Preemption-Safe Resumption. DSec maintains a globally ordered trajectory log for each sandbox, persistently recording every command invocation and its results. The trajectory serves three purposes: (1) **client fast-forwarding** — when a training task is preempted, sandbox resources are retained nonetheless; upon resumption, DSec replays cached results for previously completed commands, accelerating task recovery whilst also preventing errors from re-execution of non-idempotent operations; (2) **fine-grained provenance** — the origin and corresponding outcomes of each state change are traceable; (3) **deterministic replay** — any historical session can be faithfully reproduced from its trajectory.

5.3. Standard Benchmark Evaluation

5.3.1. Evaluation Setup

Knowledge and Reasoning. Knowledge and reasoning datasets include MMLU-Pro (Wang et al., 2024b), GPQA (Rein et al., 2023), Human Last Exam (Phan et al., 2025), Simple-QA Verified (Haas et al., 2025), Chinese-SimpleQA (He et al., 2024), LiveCodeBench-v6 (Jain et al., 2024), CodeForces (Internal Benchmark), HMMT 2026 Feb, Apex (Balunović et al., 2025), Apex Shortlist (Balunović et al., 2025), IMOAnswerBench (Luong et al., 2025), and PutnamBench (Tsoukalas et al., 2024).

For code, we evaluate DeepSeek-V4 series on LiveCodeBench-v6 and an internal Codeforces benchmark. For Codeforces, we collect 14 Codeforces Division 1 contests comprising 114 problems (May 2025 - November 2025). The Elo rating is computed as follows. For each contest, we generate 32 candidate solutions per problem. For each problem independently, we sample 10 of these solutions without replacement and arrange them in a random order to form the submission sequence. Each submission is judged against a test suite constructed by domain experts. The score for a solved problem follows the penalty scheme of OpenAI (2025): the model receives the median score of human participants who solved the same problem with the same number of prior failed attempts. This yields a total contest score for each sampled submission sequence, which is then converted into a contest rank and subsequently into an estimated rating via the standard Codeforces rating system. The contest-level expected rating is defined as the expectation of this estimated rating over all possible random selections and orderings of the 10 submissions per problem. The model’s overall rating is the average of these contest-level expected ratings across all 14 contests.

For reasoning and knowledge tasks, we set the temperature to 1.0 and the context window to 8K, 128K, and 384K tokens for the Non-think, High, and Max modes, respectively. For math tasks (e.g., HMMT, IMOAnswerBench, Apex, and HLE), we evaluate using the following template: `"{question}\nPlease reason step by step, and put your final answer within`

与此同时，数据块按需从3FS中获取。对于微虚拟机（microVM），DSec采用overlaybd (Li等人, 2020)提出的磁盘格式：只读基础层存放于3FS以实现跨实例共享，而写入操作则定向至本地的写时复制层。此类快照支持链式存储，既实现了高效的版本控制，又能以毫秒级速度恢复运行。

大规模并发下的密度优化。为支持每集群数十万个沙箱，DSec解决了两个资源瓶颈问题。首先，它减少了虚拟化环境中重复的页缓存占用，并应用内存回收技术实现安全超售。其次，它缓解了容器运行时自旋锁的争用，从而降低了每个沙箱的CPU开销，显著提升了单主机部署密度。

轨迹日志记录与抢占安全恢复。DSec为每个沙盒维护一个全局有序的轨迹日志，持久化记录每条命令调用及其结果。该轨迹服务于三个目的：(1) 客户端快速前推——当训练任务被抢占时，沙盒资源仍被保留；恢复时，DSec会重放已缓存的前期完成命令结果，加速任务恢复同时避免非幂等操作重复执行导致的错误；(2) 细粒度溯源——每个状态变更的来源与对应结果均可追溯；(3) 确定性重放——任何历史会话均可从其轨迹中忠实复现。

5.3. 标准基准评估

5.3.1. Evaluation Setup

知识与推理。知识与推理数据集包括MMLU-Pro (Wang等人, 2024b)、GPQA (Rein等人, 2023)、人类终考 (Phan等人, 2025)、Simple-QA验证集 (Haas等人, 2025)、中文Simple-QA (He等人, 2024)、LiveCodeBench-v6 (Jain等人, 2024)、CodeForces (内部基准)、HMMT 2026年2月、Apex (Balunović等人, 2025)、Apex候选集 (Balunović等人, 2025)、IMOAnswerBench (Luong等人, 2025) 以及PutnamBench (Tsoukalas等人, 2024)。

在代码能力方面，我们对DeepSeek-V4系列在LiveCodeBench-v6及内部Codeforces基准测试上进行了评估。针对Codeforces测试，我们收集了2025年5月至2025年11月期间举办的14场Codeforces Division 1竞赛，共计114道题目。Elo评分计算方式如下：每道题目生成32个候选解决方案后，独立抽取其中10个方案形成无重复的随机提交序列。每个提交会由领域专家构建的测试套件进行评判。解题得分采用OpenAI (2025) 的罚分机制：模型获得与人类选手在相同失败尝试次数下解决同一题目时的中位数分数。通过该机制计算出每个抽样提交序列的总竞赛得分，随后转换为竞赛排名，并依据标准Codeforces评级系统换算为预估评级。竞赛级别期望评级的定义为：针对每道题目10个提交的所有可能随机组合及排序情况，取预估评级的期望值。模型最终评级为14场竞赛中所有竞赛级别期望评级的平均值。

对于推理与知识任务，我们将温度参数设为1.0，并为Non-think、High和Max模式分别设置8K、128K和384K token的上下文窗口。针对数学任务（如HMMT、IMOAnswerBench、Apex和HLE），我们采用以下模板进行评估：

```
"{question}"\nPlease reason step by step, and put your final answer within
```

`\boxed{}`. " For DeepSeek-V4-Pro-Max on math tasks, we use the following template to elicit deeper reasoning: "Solve the following problem. The problem may ask you to prove a statement, or ask for an answer. If finding an answer is required, you should come up with the answer, and your final solution should also be a rigorous proof of that answer being valid.\n\n{question}".

For formal math tasks, we evaluate in an agentic setting on Lean v4.28.0-rc1 (Moura and Ullrich, 2021), with access to the Lean compiler and a semantic tactic search engine, running up to 500 tool calls with max reasoning effort. In addition, we evaluate a more compute-intensive pipeline in which candidate natural-language solutions are first generated and filtered by self-verification(Shao et al., 2025), and the retained solutions are then provided as guidance to a formal agent for proving the corresponding Lean statement. This design uses informal reasoning to improve exploration while preserving strict correctness through formal verification. A submission is counted as correct only if the strict verifier Comparator accepts it for both settings.

We have left some entries blank for K2.6 and GLM-5.1, as their APIs were too busy to return responses to our queries.

1M-Token Context. Since DeepSeek-V4 series supports 1M-token contexts, we evaluate model performance in a long context scenario by selecting OpenAI MRCR (OpenAI, 2024b) and CorpusQA (Lu et al., 2026) as the benchmarks. We re-evaluate Claude Opus 4.6 and Gemini 3.1 Pro on these tasks with the goal of standardizing the configuration across all models. We did not evaluate GPT-5.4 because its API failed to respond to a large portion of our queries.

Agent. Agent datasets include Terminal Bench 2.0 (Merrill et al., 2026), SWE-Verified (OpenAI, 2024e), SWE Multilingual (Yang et al., 2025), SWE-Pro (Deng et al., 2025), BrowseComp (Wei et al., 2025), the public evaluation set of MCPAtlas (Bandi et al., 2026), GDPval-AA (AA, 2025; Patwardhan et al., 2025), and Tool-Decathlon (Li et al., 2025).

For code agent tasks (SWE-Verified, Terminal-Bench, SWE-Pro, SWE Multilingual), we evaluate DeepSeek-V4 series using an internally developed evaluation framework. This framework provides a minimal set of tools — a bash tool and a file-edit tool. The maximum number of interaction steps is set to 500, and the maximum context length is set to 512K tokens. Regarding Terminal-Bench 2.0, we acknowledge the environment-related issues noted by GLM-5.1. Nevertheless, we report our performance on the original Terminal-Bench 2.0 dataset for consistency. On the Terminal-Bench 2.0 Verified subset, DeepSeek-V4-Pro achieves a score of approximately 72.0.

For search agent tasks (BrowseComp, HLE w/ tool), we also use an in-house harness with websearch and Python tool, and set maximum interaction steps to 500 and the maximum context length to 512K tokens. For BrowseComp, we use the same discard-all context management strategy as DeepSeek-V3.2 (DeepSeek-AI, 2025).

5.3.2. Evaluation Results

The comparison of DeepSeek-V4-Pro-Max and other closed/open source models is presented in Table 6. Also, we evaluate different modes of DeepSeek-V4-Flash and DeepSeek-V4-Pro and show the results in Table 7.

`\boxed{}`." 对于DeepSeek-V4-Pro-Max在数学任务上的应用，我们采用以下模板以激发更深层次的推理："Solve the following problem. The problem may ask you to prove a statement, or ask for an answer. If finding an answer is required, you should come up with the answer, and your final solution should also be a rigorous proof of that answer being valid.\n\n{question}""。

对于正式的数学任务，我们在Lean v4.28.0-rc1（Moura和Ulrich，2021）的自主代理环境下进行评估，该环境配备了Lean编译器和语义策略搜索引擎，最多可运行500次工具调用，并施加最大推理努力。此外，我们还评估了一个计算更为密集的流程：首先生成候选自然语言解并通过自验证（Shao等人，2025）进行筛选，随后将保留的解作为指导提供给形式化代理，以证明相应的Lean命题。这一设计利用非形式化推理来增强探索能力，同时通过形式化验证确保严格正确性。只有当严格验证器Comparator在两种设置下均接受某提交时，该提交才会被判定为正确。

我们在K2.6和GLM-5.1部分留空了一些条目，因为它们的API过于繁忙，未能对我们的查询返回响应。

1M Token上下文。由于DeepSeek-V4系列支持1M Token的上下文长度，我们通过选用OpenAI MRCR（OpenAI，2024b）和CorpusQA（Lu等人，2026）作为基准，在长上下文场景下评估模型性能。我们重新评估了Claude Opus 4.6和Gemini 3.1 Pro在这些任务上的表现，旨在统一所有模型的配置标准。未对GPT-5.4进行评估，因其API未能响应我们的大部分查询请求。

智能体。智能体数据集包括Terminal Bench 2.0（Merrill等人，2026）、SWE-Verified（OpenAI，2024e）、SWE Multilingual（Yang等人，2025）、SWE-Pro（Deng等人，2025）、BrowseComp（Wei等人，2025）、MCPAtlas的公共评估集（Bandi等人，2026）、GDPval-AA（AA，2025；Patwardhan等人，2025）以及Tool-Decathlon（Li等人，2025）。

针对代码代理任务（SWE-Verified、Terminal-Bench、SWE-Pro、SWE多语言场景），我们采用内部开发的评估框架对DeepSeek-V4系列进行测试。该框架提供最精简的工具集——bash工具与文件编辑工具，交互步数上限设为500步，上下文长度上限设为512K tokens。关于Terminal-Bench 2.0，我们注意到GLM-5.1指出的环境相关问题，但为保持一致性仍基于原始Terminal-Bench 2.0数据集汇报性能表现。在Terminal-Bench 2.0 Verified子集上，DeepSeek-V4-Pro取得了约72.0分的成绩。

对于搜索代理任务（BrowseComp、带工具的HLE），我们还采用了一个内置的网络搜索和Python工具套件，并将最大交互步数设置为500步，上下文最大长度设为512K tokens。在BrowseComp中，我们采用了与DeepSeek-V3.2（DeepSeek-AI，2025年）相同的全丢弃式上下文管理策略。

5.3.2. Evaluation Results

DeepSeek-V4-Pro-Max与其他闭源/开源模型的对比展示在表6中。此外，我们还评估了DeepSeek-V4-Flash和DeepSeek-V4-Pro的不同模式，结果呈现在表7中。

Table 6 | Comparison between DeepSeek-V4-Pro-Max and closed/open source models. "Max", "xHigh", and "High" denote reasoning effort. The best results are highlighted in bold; the second-best results are underlined.

Benchmark (Metric)		Opus-4.6 Max	GPT-5.4 xHigh	Gemini-3.1-Pro High	K2.6 Thinking	GLM-5.1 Thinking	DS-V4-Pro Max
Knowledge & Reasoning	MMLU-Pro (EM)	<u>89.1</u>	87.5	91.0	87.1	86.0	87.5
	SimpleQA-Verified (Pass@1)	46.2	45.3	75.6	36.9	38.1	<u>57.9</u>
	Chinese-SimpleQA (Pass@1)	76.4	76.8	85.9	75.9	75.0	<u>84.4</u>
	GPQA Diamond (Pass@1)	91.3	<u>93.0</u>	94.3	90.5	86.2	90.1
	HLE (Pass@1)	<u>40.0</u>	39.8	44.4	36.4	34.7	37.7
	LiveCodeBench (Pass@1)	88.8	-	<u>91.7</u>	89.6	-	93.5
	Codeforces (Rating)	-	<u>3168</u>	3052	-	-	3206
	HMMT 2026 Feb (Pass@1)	<u>96.2</u>	97.7	94.7	92.7	89.4	95.2
	IMOAnswerBench (Pass@1)	75.3	91.4	81.0	86.0	83.8	<u>89.8</u>
	Apex (Pass@1)	34.5	<u>54.1</u>	60.9	24.0	11.5	38.3
	Apex Shortlist (Pass@1)	85.9	78.1	<u>89.1</u>	75.5	72.4	90.2
Long	MRCR 1M (MMR)	92.9	-	76.3	-	-	<u>83.5</u>
	CorpusQA 1M (ACC)	71.7	-	53.8	-	-	<u>62.0</u>
Agentic	Terminal Bench 2.0 (Acc)	65.4	75.1	<u>68.5</u>	66.7	63.5	67.9
	SWE Verified (Resolved)	80.8	-	<u>80.6</u>	80.2	-	<u>80.6</u>
	SWE Pro (Resolved)	57.3	57.7	54.2	58.6	<u>58.4</u>	55.4
	SWE Multilingual (Resolved)	77.5	-	-	<u>76.7</u>	73.3	<u>76.2</u>
	BrowseComp (Pass@1)	<u>83.7</u>	82.7	85.9	83.2	79.3	83.4
	HLE w/ tools (Pass@1)	<u>53.1</u>	52.0	51.6	54.0	50.4	48.2
	GDPval-AA (Elo)	<u>1619</u>	1674	1314	1482	1535	1554
	MCPAtlas Public(Pass@1)	73.8	67.2	69.2	66.6	71.8	<u>73.6</u>
	Toolathlon (Pass@1)	47.2	54.6	48.8	50.0	40.7	<u>51.8</u>

Knowledge. In the evaluation of general world knowledge, DeepSeek-V4-Pro-Max, the maximum reasoning effort mode of DeepSeek-V4-Pro, establishes a new state-of-the-art among open-source large language models. As demonstrated by the SimpleQA-Verified, DeepSeek-V4-Pro-Max significantly outperforms all existing open-source baselines by a margin of 20 absolute percentage points. Despite these advances, it currently trails the leading proprietary model, Gemini-3.1-Pro. In the domain of educational knowledge and reasoning, DeepSeek-V4-Pro-Max marginally outperforms Kimi and GLM across the MMLU-Pro, GPQA, and HLE benchmarks, although it lags behind leading proprietary models. Broadly, DeepSeek-V4-Pro-Max marks a significant milestone in enhancing the world knowledge capabilities of open-source models.

In addition, a significant performance gap exists between DeepSeek-V4-Flash and DeepSeek-V4-Pro on knowledge-based tasks; this is anticipated, as larger parameter counts facilitate greater knowledge retention during pre-training. Notably, both models demonstrate improved results on knowledge benchmarks when allocated higher reasoning effort.

Reasoning. DeepSeek-V4-Pro-Max outperforms all prior open models across reasoning benchmarks, and matches state-of-the-art closed models on many metrics, while the smaller DeepSeek-V4-Flash-Max also surpasses the previous best open-source model, K2.6-Thinking, on code and math reasoning tasks. Meanwhile, DeepSeek-V4-Pro and DeepSeek-V4-Flash excel in cod-

表6 | DeepSeek-V4-Pro-Max与闭源/开源模型的对比。"Max"、"xHigh"和"High"表示推理强度。最佳结果以**粗体**标出；次优结果以下划线标注。

Benchmark (Metric)		Opus-4.6 Max	GPT-5.4 xHigh	Gemini-3.1-Pro High	K2.6 Thinking	GLM-5.1 Thinking	DS-V4-Pro Max
g n o o s a e R & e g d e W o n K	MMLU-Pro (EM)	<u>89.1</u>	87.5	91.0	87.1	86.0	87.5
	SimpleQA-Verified (Pass@1)	46.2	45.3	75.6	36.9	38.1	<u>57.9</u>
	Chinese-SimpleQA (Pass@1)	76.4	76.8	85.9	75.9	75.0	<u>84.4</u>
	GPQA Diamond (Pass@1)	91.3	<u>93.0</u>	94.3	90.5	86.2	90.1
	HLE (Pass@1)	<u>40.0</u>	39.8	44.4	36.4	34.7	37.7
	LiveCodeBench (Pass@1)	88.8	-	<u>91.7</u>	89.6	-	93.5
	Codeforces (Rating)	-	<u>3168</u>	3052	-	-	3206
	HMMT 2026 Feb (Pass@1)	<u>96.2</u>	97.7	94.7	92.7	89.4	95.2
	IMOAnswerBench (Pass@1)	75.3	91.4	81.0	86.0	83.8	<u>89.8</u>
	Apex (Pass@1)	34.5	<u>54.1</u>	60.9	24.0	11.5	38.3
	Apex Shortlist (Pass@1)	85.9	78.1	<u>89.1</u>	75.5	72.4	90.2
	MRCR 1M (MMR)	92.9	-	76.3	-	-	<u>83.5</u>
c i n e g A	CorpusQA 1M (ACC)	71.7	-	53.8	-	-	<u>62.0</u>
	Terminal Bench 2.0 (Acc)	65.4	75.1	<u>68.5</u>	66.7	63.5	67.9
	SWE Verified (Resolved)	80.8	-	<u>80.6</u>	80.2	-	<u>80.6</u>
	SWE Pro (Resolved)	57.3	57.7	54.2	58.6	<u>58.4</u>	55.4
	SWE Multilingual (Resolved)	77.5	-	-	<u>76.7</u>	73.3	<u>76.2</u>
	BrowseComp (Pass@1)	<u>83.7</u>	82.7	85.9	83.2	79.3	83.4
	HLE w/ tools (Pass@1)	<u>53.1</u>	52.0	51.6	54.0	50.4	48.2
	GDPval-AA (Elo)	<u>1619</u>	1674	1314	1482	1535	1554
	MCPAtlas Public(Pass@1)	73.8	67.2	69.2	66.6	71.8	<u>73.6</u>
	Toolathlon (Pass@1)	47.2	54.6	48.8	50.0	40.7	<u>51.8</u>

知识。在通用世界知识的评估中，DeepSeek-V4-Pro-Max作为DeepSeek-V4-Pro的最大推理效能模式，为开源大语言模型树立了新的标杆。如SimpleQA-Verified所示，DeepSeek-V4-Pro-Max以20个绝对百分点的优势显著超越所有现有开源基线模型。尽管取得这些进展，它目前仍落后于领先的专有模型Gemini-3.1-Pro。在教育知识与推理领域，DeepSeek-V4-Pro-Max在MMLU-Pro、GPQA和HLE基准测试中略微优于Kimi和GLM，但与头部专有模型相比仍存在差距。总体而言，DeepSeek-V4-Pro-Max标志着开源模型在世界知识能力提升上的重要里程碑。

此外，DeepSeek-V4-Flash与DeepSeek-V4-Pro在知识型任务上存在显著的性能差距；这在意料之中，因为更大的参数量有助于在预训练阶段保留更多知识。值得注意的是，当分配更高推理资源时，两款模型在知识基准测试中均展现出提升效果。

推理能力。DeepSeek-V4-Pro-Max 在所有先前的开放模型上超越了所有推理基准测试，并在多项指标上媲美最先进的闭源模型，而更小型的 DeepSeek-V4-Flash-Max 在代码和数学推理任务上也超越了之前的最佳开源模型 K2.6-Thinking。与此同时，DeepSeek-V4-Pro 和 DeepSeek-V4-Flash 在代码领-

Table 7 | Comparison among different sizes and modes of DeepSeek-V4 series. "Non-Think", "High", and "Max" denote reasoning effort.

Benchmark (Metric)		DeepSeek-V4-Flash			DeepSeek-V4-Pro		
		Non-Think	High	Max	Non-Think	High	Max
Knowledge & Reasoning	MMLU-Pro (EM)	83.0	86.4	86.2	82.9	87.1	87.5
	SimpleQA-Verified (Pass@1)	23.1	28.9	34.1	45.0	46.2	57.9
	Chinese-SimpleQA (Pass@1)	71.5	73.2	78.9	75.8	77.7	84.4
	GPQA Diamond (Pass@1)	71.2	87.4	88.1	72.9	89.1	90.1
	HLE (Pass@1)	8.1	29.4	34.8	7.7	34.5	37.7
	LiveCodeBench (Pass@1-COT)	55.2	88.4	91.6	56.8	89.8	93.5
	Codeforces (Rating)	-	2816	3052	-	2919	3206
	HMMT 2026 Feb (Pass@1)	40.8	91.9	94.8	31.7	94.0	95.2
	IMOAnswerBench (Pass@1)	41.9	85.1	88.4	35.3	88.0	89.8
	Apex (Pass@1)	1.0	19.1	33.0	0.4	27.4	38.3
	Apex Shortlist (Pass@1)	9.3	72.1	85.7	9.2	85.5	90.2
Long	MRCR 1M(MMR)	37.5	76.9	78.7	44.7	83.3	83.5
	CorpusQA 1M(ACC)	15.5	59.3	60.5	35.6	56.5	62.0
Agentic	Terminal Bench 2.0 (Acc)	49.1	56.6	56.9	59.1	63.3	67.9
	SWE Verified (Resolved)	73.7	78.6	79.0	73.6	79.4	80.6
	SWE Pro (Resolved)	49.1	52.3	52.6	52.1	54.4	55.4
	SWE Multilingual (Resolved)	69.7	70.2	73.3	69.8	74.1	76.2
	BrowseComp (Pass@1)	-	53.5	73.2	-	80.4	83.4
	HLE w/ tools (Pass@1)	-	40.3	45.1	-	44.7	48.2
	MCPAtlas Public (Pass@1)	64.0	67.4	69.0	69.4	74.2	73.6
	GDPval-AA (Elo)	-	-	1395	-	-	1554
	Toolathlon (Pass@1)	40.7	43.5	47.8	46.3	49.0	51.8

ing competitions. According to our evaluation, their performance is comparable to GPT-5.4, making this the first time an open model has matched a closed model on this task. On the Codeforces leaderboard, DeepSeek-V4-Pro-Max currently ranks 23rd among human candidates. DeepSeek-V4 also demonstrates strong performance on formal mathematical task under both agentic and compute-intensive settings. Under an agentic setup, it achieves state-of-the-art results, shown in Figure 8, outperforming prior models such as Seed Prover (Chen et al., 2025). With a more compute-intensive pipeline, performance further improves, surpassing systems including Aristotle(Achim et al., 2025) and matching the best known results under this setting.

Agent. The DeepSeek-V4 series demonstrates strong agent performance in evaluations. For code agent tasks, DeepSeek-V4-Pro achieves results comparable to K2.6 and GLM-5.1, though all these open models still lag behind their closed-source counterparts. DeepSeek-V4-Flash underperforms DeepSeek-V4-Pro on coding tasks, particularly on Terminal Bench 2.0. A similar trend is observed across other agent evaluations. It is worth noting that DeepSeek-V4-Pro performs well on MCPAtlas and Toolathlon—two evaluation test sets that include a wide range of tools and MCP services—indicating that our model has excellent generalization capability and does not perform well only on internal frameworks.

1M-Token Context. DeepSeek-V4-Pro outperforms Gemini-3.1-Pro on the MRCR task, which measures in-context retrieval, but remains behind Claude Opus 4.6. As illustrated in Figure 9, retrieval performance remains highly stable within a 128K context window. While a performance

表7 | DeepSeek-V4系列不同尺寸与模式的比较。"Non-Think"、"High"和"Max"表示推理力度。

Benchmark (Metric)		DeepSeek-V4-Flash			DeepSeek-V4-Pro		
		Non-Think	High	Max	Non-Think	High	Max
g n n o s a a e R & e g d e w o n K	MMLU-Pro (EM)	83.0	86.4	86.2	82.9	87.1	87.5
	SimpleQA-Verified (Pass@1)	23.1	28.9	34.1	45.0	46.2	57.9
	Chinese-SimpleQA (Pass@1)	71.5	73.2	78.9	75.8	77.7	84.4
	GPQA Diamond (Pass@1)	71.2	87.4	88.1	72.9	89.1	90.1
	HLE (Pass@1)	8.1	29.4	34.8	7.7	34.5	37.7
	LiveCodeBench (Pass@1-COT)	55.2	88.4	91.6	56.8	89.8	93.5
	Codeforces (Rating)	-	2816	3052	-	2919	3206
	HMMT 2026 Feb (Pass@1)	40.8	91.9	94.8	31.7	94.0	95.2
	IMOAnswerBench (Pass@1)	41.9	85.1	88.4	35.3	88.0	89.8
	Apex (Pass@1)	1.0	19.1	33.0	0.4	27.4	38.3
	Apex Shortlist (Pass@1)	9.3	72.1	85.7	9.2	85.5	90.2
g n o L	MRCR 1M(MMR)	37.5	76.9	78.7	44.7	83.3	83.5
	CorpusQA 1M(ACC)	15.5	59.3	60.5	35.6	56.5	62.0
c i t i n e g A	Terminal Bench 2.0 (Acc)	49.1	56.6	56.9	59.1	63.3	67.9
	SWE Verified (Resolved)	73.7	78.6	79.0	73.6	79.4	80.6
	SWE Pro (Resolved)	49.1	52.3	52.6	52.1	54.4	55.4
	SWE Multilingual (Resolved)	69.7	70.2	73.3	69.8	74.1	76.2
	BrowseComp (Pass@1)	-	53.5	73.2	-	80.4	83.4
	HLE w/ tools (Pass@1)	-	40.3	45.1	-	44.7	48.2
	MCPAtlas Public (Pass@1)	64.0	67.4	69.0	69.4	74.2	73.6
	GDPval-AA (Elo)	-	-	1395	-	-	1554
	Toolathlon (Pass@1)	40.7	43.5	47.8	46.3	49.0	51.8

编程竞赛中。根据我们的评估，其性能与GPT-5.4相当，这标志着开源模型在该任务上首次与闭源模型匹敌。在Codeforces排行榜上，DeepSeek-V4-Pro-Max目前位列人类选手第23名。DeepSeek-V4在形式化数学任务中也展现出强劲表现，无论是智能体模式还是计算密集型场景下。在智能体配置下，如图8所示，它取得了最先进的成果，超越了Seed Prover（Chen等人，2025）等先前模型。采用更计算密集的流程后，性能进一步提升，超越了包括Aristotle（Achim等人，2025）在内的系统，并达到了该设置下已知的最佳结果。

代理。DeepSeek-V4系列在评估中展现出强大的代理性能。对于代码代理任务，DeepSeek-V4-Pro取得了与K2.6和GLM-5.1相当的结果，尽管所有这些开源模型仍落后于闭源模型。DeepSeek-V4-Flash在编码任务上表现不如DeepSeek-V4-Pro，特别是在Terminal Bench 2.0上。其他代理评估中也观察到了类似的趋势。值得注意的是，DeepSeek-V4-Pro在MCPAtlas和Toolathlon上表现优异——这两个评估测试集涵盖了广泛的工具和MCP服务——这表明我们的模型具有出色的泛化能力，而不仅仅在内部框架中表现良好。

1M-Token上下文。DeepSeek-V4-Pro在衡量上下文检索的MRCR任务上表现优于Gemini-3.1-Pro，但仍落后于Claude Opus 4.6。如图9所示，检索性能在128K上下文窗口内保持高度稳定。尽管性能

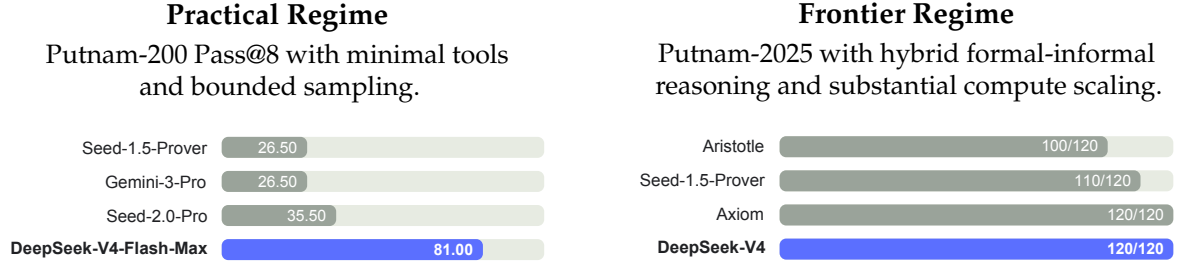


Figure 8 | Formal reasoning under practical and frontier regimes. Left: Putnam-200 Pass@8 evaluates a fixed random subset of PutnamBench (Tsoukalas et al., 2024) following the setup introduced by Seed-Prover; all models are tested on the same problem set. We follow the Seed-Prover protocol but replace proprietary search tools with the open-source LeanExplore (Asher, 2025), yielding a lightweight setting with minimal agent tools and bounded sampling. Right: Putnam-2025 probes the frontier of mathematical reasoning in a scaled hybrid formal-informal regime, where informal reasoning is combined with formal verification to expose gaps and improve rigor; DeepSeek-V4 reaches a proof-perfect 120/120.

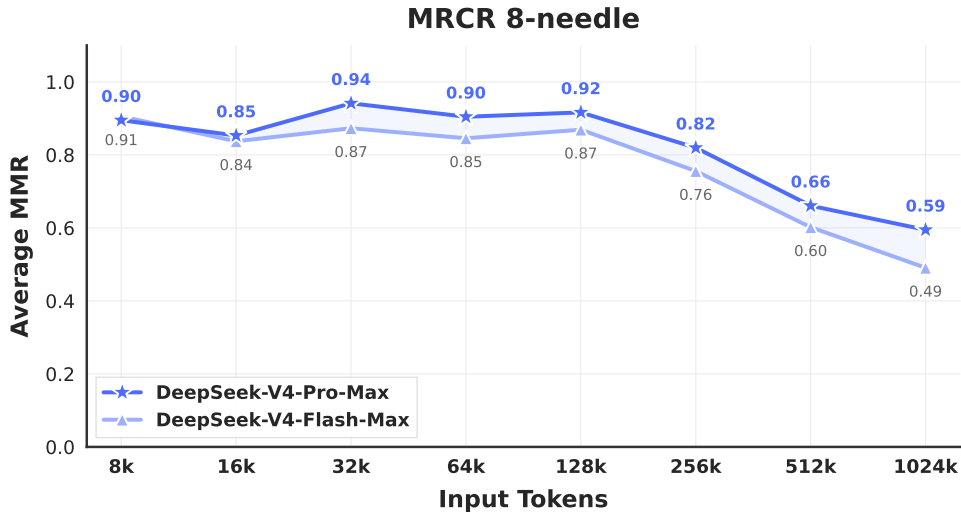


Figure 9 | DeepSeek-V4 series performance on the MRCR task.

degradation becomes visible beyond the 128K mark, the model’s retrieval capabilities at 1M tokens remain remarkably strong compared to both proprietary and open-source counterparts. Unlike MRCR, CorpusQA is similar to real scenarios. The evaluation results also indicate that DeepSeek-V4-Pro is better than Gemini-3.1-Pro.

Reasoning Effort. As shown in Table 7, the Max mode, which employs longer contexts and reduced length penalties in RL, outperforms the High mode on the most challenging tasks. Figure 10 presents a comparison of performance and cost among DeepSeek-V4-Pro, DeepSeek-V4-Flash, and DeepSeek-V3.2 on representative reasoning and agentic tasks. By scaling test-time compute, DeepSeek-V4 series achieve substantial improvements over the predecessor. Furthermore, on reasoning tasks like HLE, DeepSeek-V4-Pro demonstrates higher token efficiency than DeepSeek-V3.2.



图8 | 实践与前沿机制下的形式化推理。左图：Putnam-200 Pass@8按照Seed-Prover引入的评估框架，对PutnamBench（Tsoukalas等人，2024年）的固定随机子集进行测试；所有模型均使用同一问题集。我们遵循Seed-Prover协议，但将私有搜索工具替换为开源工具LeanExplore（Asher，2025年），从而构建了一个仅含基础智能体工具与有限采样的轻量化环境。右图：Putnam-2025探究了数学推理前沿的混合形式化-非形式化机制，通过结合非形式化推理与形式化验证来暴露缺陷并提升严谨性；DeepSeek-V4在此实现了120/120的完美验证率。

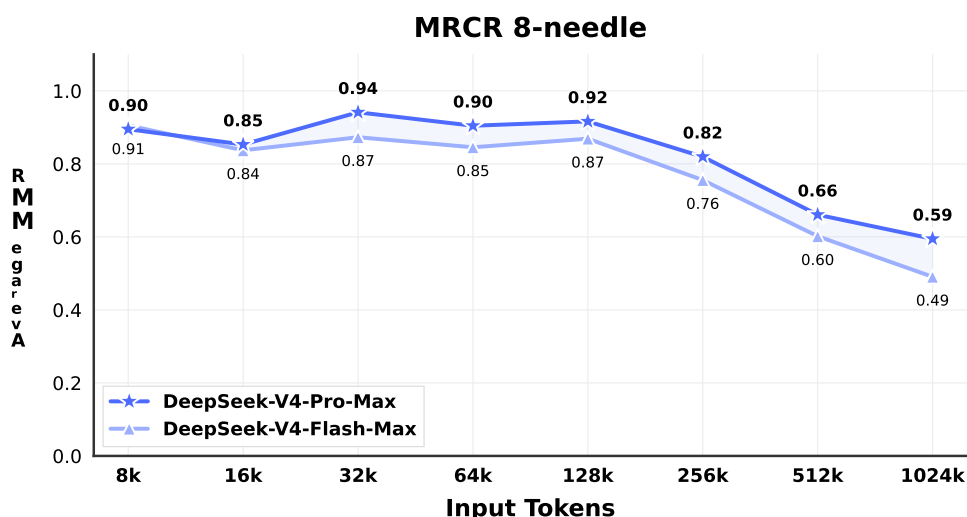


图9 | DeepSeek-V4系列在MRCR任务上的性能表现。

在超过128K标记后，性能退化开始显现，但在1M tokens长度下，该模型的检索能力相较于专有和开源竞品仍保持显著优势。与MRCR不同，CorpusQA更贴近真实场景。评估结果还表明，DeepSeek-V4-Pro优于Gemini-3.1-Pro。

推理效率。如表7所示，采用更长上下文及强化学习中更低长度惩罚的Max模式，在最具挑战性的任务上表现优于High模式。图10对比了DeepSeek-V4-Pro、DeepSeek-V4-Flash与DeepSeek-V3.2在典型推理与代理任务中的性能与成本表现。通过扩展测试时计算量，DeepSeek-V4系列相较前代实现了显著提升。此外，在HLE等推理任务上，DeepSeek-V4-Pro展现出比DeepSeek-V3.2更高的标记效率。

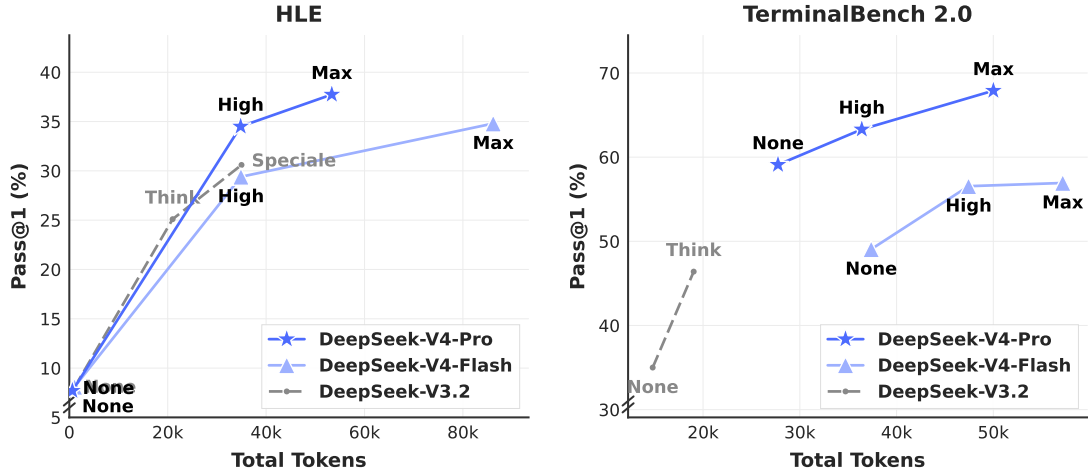


Figure 10 | HLE and TerminalBench 2.0 performance by reasoning effort. “None” indicates Non-think mode, and “Speciale” indicates DeepSeek-V3.2-Speciale model.

5.4. Performance on Real-World Tasks

Standardized benchmarks often struggle to capture the complexities of diverse, real-world tasks, creating a gap between test results and actual user experience. To bridge this, we have developed proprietary internal metrics that prioritize real-world usage patterns over traditional benchmarks. This approach ensures that our optimizations translate into tangible benefits. Our evaluation framework specifically targets the primary use cases of the DeepSeek API and Chatbot, aligning model performance with practical demands.

5.4.1. Chinese Writing

One of the primary use cases for DeepSeek is Chinese writing. We conducted a rigorous evaluation on functional writing and creative writing. Table 12 presents a pairwise comparison between DeepSeek-V4-Pro and Gemini-3.1-Pro on functional writing tasks. These tasks consist of common daily writing queries, where prompts are typically concise and straightforward. Gemini-3.1-Pro was selected as the baseline, as it stands as the top-performing external model for Chinese writing in our evaluations. The results indicate that DeepSeek-V4-Pro outperforms the baseline with an overall win rate of 62.7% versus 34.1%; this is primarily because Gemini occasionally allows its inherent stylistic preferences to override the user’s explicit requirements in Chinese writing scenarios.

Table 13 presents the creative writing comparison, which is evaluated along two axes: instruction following and writing quality. Compared with Gemini-3.1-Pro, DeepSeek-V4-Pro achieves a 60.0% win rate in instruction following and 77.5% in writing quality, demonstrating a marginal improvement in instruction following and a substantial gain in writing quality. Although DeepSeek-V4-Pro yields superior results in aggregate user case analysis, an evaluation restricted to the most challenging prompts — specifically those involving high-complexity constraints or multi-turn scenarios — reveals that Claude Opus 4.5 retains a performance advantage over DeepSeek-V4-Pro. As shown in Table 14, Claude Opus 4.5 achieves a 52.0% win rate versus 45.9%.

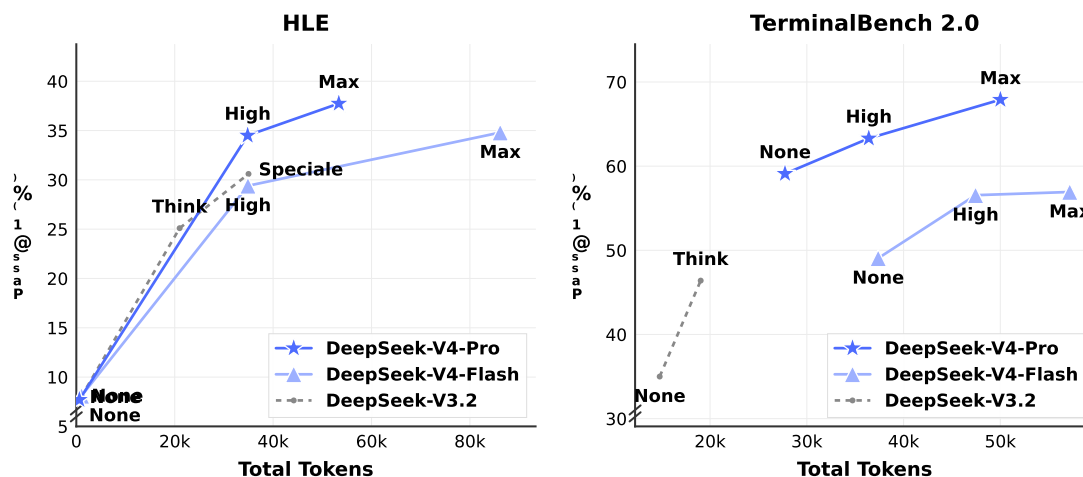


图10 | HLE与Terminal Bench 2.0在不同推理努力下的性能表现。“无”表示非思考模式，“Speciale”代表DeepSeek-V3.2-Speciale模型。

5.4. 现实任务中的表现

标准化基准测试往往难以捕捉多样化的现实任务的复杂性，导致测试结果与实际用户体验之间存在差距。为弥合这一差异，我们开发了专有的内部指标，优先考虑真实使用模式而非传统基准。这一方法确保我们的优化转化为切实效益。我们的评估框架特别针对DeepSeek API和Chatbot的主要用例，使模型性能与实际需求保持一致。

5.4.1. Chinese Writing

DeepSeek的主要应用场景之一是中文写作。我们对功能性写作与创意写作进行了严格评估。表12展示了DeepSeek-V4-Pro与Gemini-3.1-Pro在功能性写作任务上的两两对比结果。这类任务包含日常常见的写作需求，其提示词通常简洁明了。我们选择Gemini-3.1-Pro作为基线模型，因其在我们的评测中属于中文写作表现最优的外部模型。结果表明，DeepSeek-V4-Pro以62.7%的整体胜率超越基线的34.1%，这主要源于Gemini在中文写作场景中偶尔会让其固有风格偏好凌驾于用户的明确需求之上。

表13展示了创意写作的比较结果，该评估从两个维度进行：指令遵循与写作质量。与Gemini-3.1-Pro相比，DeepSeek-V4-Pro在指令遵循上取得了60.0%的胜率，在写作质量上达到77.5%，显示出在指令遵循方面的小幅提升及写作质量上的显著进步。尽管DeepSeek-V4-Pro在整体用户案例分析中表现更优，但当评估仅限于最具挑战性的提示——特别是涉及高复杂度约束或多轮交互的场景时，Claude Opus 4.5仍保持对DeepSeek-V4-Pro的性能优势。如表14所示，Claude Opus 4.5以52.0%的胜率先于DeepSeek-V4-Pro的45.9%。

5.4.2. Search

Search-augmented question answering is a core capability of the DeepSeek chatbot. On the DeepSeek web and app, the "non-think" mode employs Retrieval-Augmented Search (RAG), whereas the "thinking" mode utilizes agentic search.

Retrieval Augmented Search. We conducted a pairwise evaluation comparing DeepSeek-V4-Pro and DeepSeek-V3.2 across both objective and subjective Q&A categories. As presented in Table 11, DeepSeek-V4-Pro outperforms DeepSeek-V3.2 by a substantial margin, demonstrating a consistent advantage across both categories. The most pronounced gains are observed in single-value search and planning & strategy tasks, suggesting that DeepSeek-V4-Pro excels at locating precise factual answers and synthesizing structured plans from retrieved context. However, DeepSeek-V3.2 remains relatively competitive on comparison and recommendation tasks, indicating potential room for improvement for DeepSeek-V4-Pro in scenarios requiring balanced, multi-perspective reasoning over search results.

Agentic Search. Unlike standard RAG, agentic search empowers the model to iteratively invoke search and fetch tools per query, significantly enhancing overall search performance. For the thinking mode in DeepSeek-Chat, we optimized the agentic search function to maximize response accuracy within a predefined "thinking budget". As shown in Table 9, agentic search consistently outperforms RAG, particularly on complex tasks. Furthermore, its cost remains highly efficient, with agentic search being only marginally more expensive than standard RAG (see Table 10).

5.4.3. White-Collar Task

To rigorously evaluate the model’s utility in sophisticated enterprise productivity scenarios, we constructed a comprehensive suite of 30 advanced Chinese professional tasks. These workflows deliberately encompass high-level cognitive demands, including in-depth information analysis, comprehensive document generation, and nuanced document editing, spanning a diverse spectrum of 13 critical industries (e.g., finance, education, law, and technology). The evaluation was conducted within an in-house agent harness equipped with basic tools, including Bash and web search.

Given the open-ended nature of these tasks, automated metrics usually fall short in capturing the nuances of a high-quality response. Therefore, we conducted human evaluations to compare the performance of DeepSeek-V4-Pro-Max against Opus-4.6-Max. Annotators blindly assessed the model outputs across four dimensions:

- **Task Completion:** Whether the core problem was successfully resolved.
- **Instruction Following:** Adherence to specific constraints and directives.
- **Content Quality:** Factual accuracy, logical coherence, and professional tone.
- **Formatting Aesthetics:** Layout readability and visual presentation.

As illustrated in Figure 11, DeepSeek-V4-Pro-Max outperforms Opus-4.6-Max on diverse Chinese white-collar tasks, achieving an impressive non-loss rate of 63%, and demonstrating consistent advantages across analysis, generation, and editing tasks. The detailed dimension scores shown in Figure 12 highlight the model’s primary strengths in Task Completion and

5.4.2. Search

搜索增强问答是DeepSeek聊天机器人的核心能力。在DeepSeek网页端和应用程序中，“非思考”模式采用检索增强搜索（RAG），而“思考”模式则运用代理式搜索。

检索增强搜索。我们进行了一项配对评估，对比DeepSeek-V4-Pro与DeepSeek-V3.2在客观和主观问答类别上的表现。如表11所示，DeepSeek-V4-Pro以显著优势超越DeepSeek-V3.2，在两大类别中均展现出一致的领先性。最突出的提升体现在单值检索及规划与策略任务上，这表明DeepSeek-V4-Pro擅长定位精确的事实性答案，并能基于检索上下文合成结构化方案。然而，DeepSeek-V3.2在比较和推荐任务上仍保持相对竞争力，这意味着DeepSeek-V4-Pro在需要对搜索结果进行平衡、多角度推理的场景中仍有改进空间。

自主式搜索。与标准RAG不同，自主式搜索赋予模型按查询迭代调用搜索及获取工具的能力，显著提升整体搜索性能。针对DeepSeek-Chat的思考模式，我们优化了自主式搜索功能，以在预设的“思考预算”内最大化响应准确性。如表9所示，自主式搜索始终优于RAG，尤其在复杂任务上表现突出。此外，其成本仍保持高效，自主式搜索仅比标准RAG略高（参见表10）。

5.4.3. White-Collar Task

为严谨评估模型在复杂企业生产力场景中的效用，我们构建了一套包含30项高级中文专业任务的全面测试集。这些 workflow 刻意设计了高层次的认知需求，涵盖深度信息分析、综合性文档生成及精细文档编辑，横跨13个关键行业（如金融、教育、法律和科技）。评测在配备基础工具（包括Bash和网络搜索）的内部智能体框架中进行。

鉴于这些任务的开放性，自动化指标通常难以捕捉高质量回答的细微差别。因此，我们进行了人工评估，以比较DeepSeek-V4-Pro-Max与Opus-4.6-Max的表现。标注者从四个维度对模型输出进行了盲评：

- 任务完成：核心问题是否得到成功解决。
- 指令遵循：遵守特定约束和指示。
- 内容质量：事实准确性、逻辑连贯性及专业语调。
- 格式美学：布局可读性与视觉呈现。

如图11所示，DeepSeek-V4-Pro-Max在多样化中文白领任务上表现优于Opus-4.6-Max，实现了63%的惊人无损率，并在分析、生成和编辑任务中展现出持续优势。图12所示的详细维度评分凸显了该模型在任务完成度和{v*}方面的主要优势。

Content Quality. Specifically, DeepSeek-V4-Pro-Max proactively anticipates implicit user intents by frequently providing supplementary insights and self-verification steps. It also excels in long-form generation, delivering in-depth, coherent narratives rather than relying on the overly simplistic bullet points frequently produced by Opus-4.6-Max. Additionally, the model strictly conforms to formal professional conventions, such as standardized Chinese hierarchical numbering. However, in terms of Instruction Following, it occasionally overlooks specific formatting constraints and slightly trails Opus. Furthermore, the model is less proficient at condensing extensive text inputs into succinct summaries. Finally, its Formatting Aesthetics still have substantial room for improvement regarding the overall visual design of presentation slides. Figure 13, 14, and 15 present several test cases; due to the extensive length of certain outputs, only partial pages are displayed.

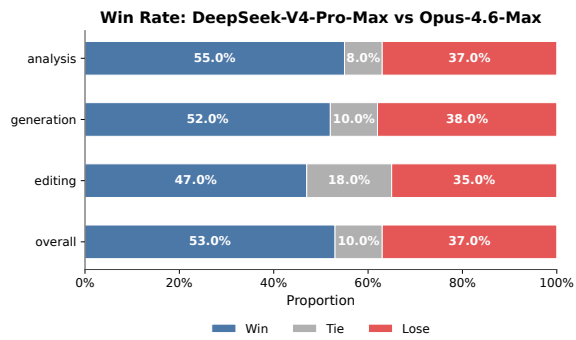


Figure 11 | Win-rate comparison across analysis, generation, editing tasks, and the overall performance.

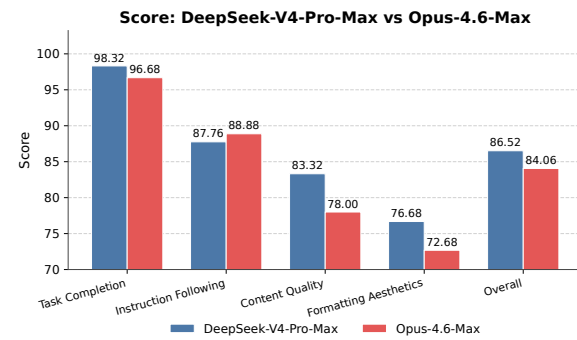


Figure 12 | Detailed dimension scores including Task Completion, Content Quality, Formatting Aesthetics, and Instruction Following.



Figure 13 | Example output of a task which requires drafting a joint marketing proposal for a popular bubble tea brand and the Beijing Subway.

内容质量。具体而言，DeepSeek-V4-Pro-Max能主动预判用户潜在意图，频繁提供补充性见解与自我验证步骤。该模型在长篇内容生成上表现尤为出色，呈现深度连贯的叙述，而非如Opus-4.6-Max那样频繁产出过度简化的要点罗列。此外，它严格遵循正式专业规范，例如标准化的中文层级编号体系。但在指令遵循方面，该模型偶尔会忽略特定格式约束，稍逊于Opus。同时，模型在将大量文本输入压缩为简洁摘要方面的能力仍有不足。最后，其格式美学在演示文稿整体视觉设计方面仍有较大提升空间。图13、14和15展示了若干测试案例；由于部分输出篇幅过长，仅展示局部页面。

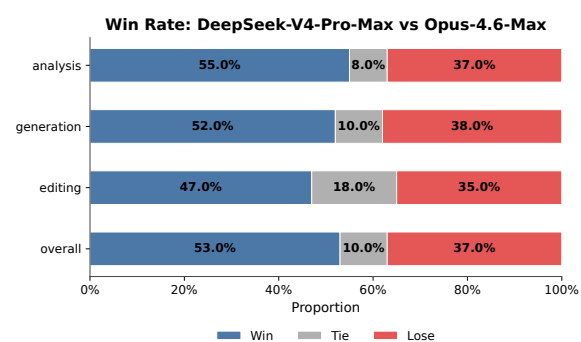


图11 | 分析、生成、编辑任务及整体性能的胜率对比。

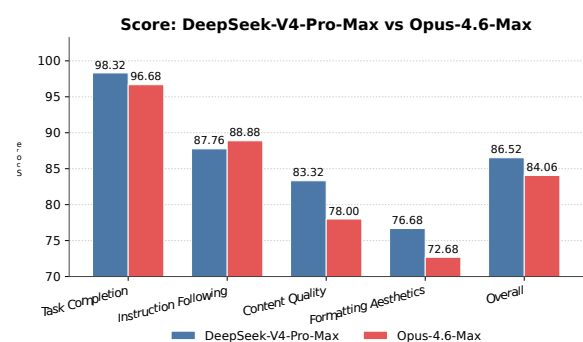


图12 | 详细维度分数包括任务完成度、内容质量、格式美观度及指令遵循情况。



图13 | 一项任务的示例输出，该任务要求为一家知名奶茶品牌与北京地铁起草联合营销提案。

5.4.4. Code Agent

To benchmark our coding agent capability, we curate tasks from real internal R&D workloads. We collect ~200 challenging tasks from 50+ internal engineers, spanning feature development, bug fixing, refactoring, and diagnostics across diverse technology stacks including PyTorch, CUDA, Rust, and C++. Each task is accompanied by its original repository, the corresponding execution environment, and human-annotated scoring rubrics; after rigorous quality filtering, 30 tasks are retained as the evaluation set. As shown in Table 8, DeepSeek-V4-Pro significantly outperforms Claude Sonnet 4.5 and approaches the level of Claude Opus 4.5.

Table 8 | Comparison on R&D Coding Benchmark (external models included strictly for evaluation purposes).

Model	Haiku 4.5	Sonnet 4.5	DeepSeek-V4-Pro-Max	Opus 4.5	Opus 4.5 Thinking	Opus 4.6 Thinking
Pass Rate (%)	13	47	67	70	73	80

In a survey asking DeepSeek developers and researchers ($N = 85$) — all with experience of using DeepSeek-V4-Pro for agentic coding in their daily work — whether DeepSeek-V4-Pro is ready to serve as their default and primary coding model compared to other frontier models, 52% said yes, 39% leaned toward yes, and fewer than 9% said no. Respondents find DeepSeek-V4-Pro to deliver satisfactory results across most tasks, but note trivial mistakes, misinterpretation of vague prompts, and occasional over-thinking.

6. Conclusion, Limitations, and Future Directions

In this work, we present a preview version of DeepSeek-V4 series, aiming at next-generation large language models that break the efficiency barrier of ultra-long-context processing. By combining a hybrid attention architecture that integrates CSA and HCA, DeepSeek-V4 series achieve a dramatic leap in long-sequence efficiency. The architectural innovations, together with extensive infrastructure optimization, enable efficient native support for million-token contexts and establish a necessary foundation for future test-time scaling, long-horizon tasks, and emerging paradigms such as online learning. Evaluation results demonstrate that DeepSeek-V4-Pro-Max, the maximum reasoning effort mode of DeepSeek-V4-Pro, redefines the state-of-the-art for open models. It substantially outperforms prior open-source models on knowledge benchmarks, achieves superior reasoning performance close to the frontier proprietary models, and delivers competitive agent capabilities. Meanwhile, DeepSeek-V4-Flash-Max attains comparable reasoning performance to leading closed models while maintaining a highly cost-efficient architecture. We believe DeepSeek-V4 series usher in a new era of million-length contexts for open models and pave the way toward better efficiency, scale, and intelligence.

In pursuit of extreme long-context efficiency, DeepSeek-V4 series adopted a bold architectural design. To minimize risk, we retained many preliminarily validated components and tricks, which, while effective, made the architecture relatively complex. In future iterations, we will carry out more comprehensive and principled investigations to distill the architecture down to its most essential designs, making it more elegant without sacrificing performance. Meanwhile, although Anticipatory Routing and SwiGLU Clamping have been proven effective in mitigating training instabilities, their underlying principles remain insufficiently understood. We will actively study foundational problems on training stability and strengthen internal metric monitoring, aiming for a more principled and predictive approach to stable large-scale training.

5.4.4. Code Agent

为了评估我们的编码代理能力，我们从真实的内部研发工作中精选了一系列任务。我们从50名内部工程师中收集了~200项具有挑战性的任务，涵盖功能开发、错误修复、重构和诊断等多个领域，涉及PyTorch、CUDA、Rust和C++等多样化的技术栈。每项任务都附带原始代码库、相应的执行环境以及人工标注的评分标准；经过严格的质量筛选后，保留了30项任务作为评估集。如表8所示，DeepSeek-V4-Pro显著优于Claude Sonnet 4.5，并接近Claude Opus 4.5的水平。

表8 | 研发编码基准对比（外部模型仅用于评估目的）。

Model	Haiku 4.5	Sonnet 4.5	DeepSeek-V4-Pro-Max	Opus 4.5	Opus 4.5 Thinking	Opus 4.6 Thinking
Pass Rate (%)	13	47	67	70	73	80

在一项针对DeepSeek开发者与研究人员（N = 85人）——他们均具备在日常工作中使用Deep Seek-V4-Pro进行代理编码的经验——的调查中，询问相较于其他前沿模型，DeepSeek-V4-Pro是否已准备好作为其默认首选编码模型时，52%的受访者表示肯定，39%倾向于认可，不足9%持否定意见。受访者认为DeepSeek-V4-Pro在多数任务中能提供令人满意的结果，但也指出存在细微错误、对模糊提示的误解以及偶发的过度思考现象。

6. 结论、局限性与未来方向

在这项工作中，我们推出了DeepSeek-V4系列的预览版本，旨在打造下一代大型语言模型，突破超长上下文处理的效率壁垒。通过融合CSA与HCA的混合注意力架构，DeepSeek-V4系列实现了长序列效率的跨越式提升。架构创新与全方位基础设施优化相结合，使其能够高效原生支持百万级token上下文，为未来测试时扩展、长程任务及在线学习等新兴范式奠定必要基础。评估结果表明，DeepSeek-V4-Pro的最强推理模式DeepSeek-V4-Pro-Max重新定义了开源模型的业界标杆——它在知识基准测试中显著超越先前开源模型，推理性能接近前沿闭源模型水平，同时展现出极具竞争力的智能体能力。而DeepSeek-V4-Flash-Max在保持高成本效益架构的同时，其推理性能已可比肩顶尖闭源模型。我们相信DeepSeek-V4系列将为开源模型开启百万长度上下文的新纪元，并朝着更高效率、更大规模和更强智能的方向开拓道路。

在追求极致长上下文效率的过程中，DeepSeek-V4系列采用了大胆的架构设计。为降低风险，我们保留了众多经过初步验证的组件与技巧——这些设计虽行之有效，却使架构相对复杂。在未来的迭代中，我们将开展更全面、更具原则性的研究，将架构提炼至最核心的设计，使其在不牺牲性能的前提下更加优雅。与此同时，尽管预期路由（Anticipatory Routing）和SwiGLU钳位技术已被证明能有效缓解训练不稳定性，但其底层原理仍未得到充分理解。我们将深入研究训练稳定性的基础问题，并加强内部指标监控，以期建立更具原则性和预测性的大规模稳定训练方法。

In addition, beyond the MoE and sparse attention architecture, we will also proactively explore model sparsity along new dimensions — such as more sparse embedding modules (Cheng et al., 2026) — to further improve computational and memory efficiency without compromising capability. We will also continuously investigate low-latency architectures and system techniques to make long-context deployment and interaction more responsive. Furthermore, we recognize the importance and practical value of long-horizon, multi-round agentic tasks, and will continue to iterate and explore in this direction. We are also working on incorporating multimodal capabilities to our models. Finally, we are committed to developing better data curation and synthesis strategies to consistently enhance model intelligence, robustness, and practical usability across an increasingly broad range of scenarios and tasks.

References

- AA. Gdpval-aa leaderboard, 2025. URL <https://artificialanalysis.ai/methodology/intelligence-benchmarking#gdpval-aa>.
- T. Achim, A. Best, A. Bietti, K. Der, M. Fédérico, S. Gukov, D. Halpern-Leistner, K. Henningsgard, Y. Kudryashov, A. Meiburg, et al. Aristotle: Imo-level automated theorem proving. arXiv preprint arXiv:2510.01346, 2025.
- A. Agache, M. Brooker, A. Florescu, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa. Firecracker: lightweight virtualization for serverless applications. In Proceedings of the 17th Usenix Conference on Networked Systems Design and Implementation, NSDI’20, page 419–434, USA, 2020. USENIX Association. ISBN 9781939133137.
- O. J. Aimuyo, B. Oh, and R. Singh. Flashmoe: Fast distributed moe in a single kernel. Advances in Neural Information Processing Systems, 2025. URL <https://neurips.cc/virtual/2025/poster/119124>.
- J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. arXiv preprint arXiv:2305.13245, 2023.
- J. Asher. LeanExplore: A search engine for Lean 4 declarations, 2025. URL <https://arxiv.org/abs/2506.11085>.
- Y. Bai, Y. Bao, G. Chen, J. Chen, N. Chen, R. Chen, Y. Chen, Y. Chen, Y. Chen, Z. Chen, J. Cui, H. Ding, M. Dong, A. Du, C. Du, D. Du, Y. Du, Y. Fan, Y. Feng, K. Fu, B. Gao, H. Gao, P. Gao, T. Gao, X. Gu, L. Guan, H. Guo, J. Guo, H. Hu, X. Hao, T. He, W. He, W. He, C. Hong, Y. Hu, Z. Hu, W. Huang, Z. Huang, Z. Huang, T. Jiang, Z. Jiang, X. Jin, Y. Kang, G. Lai, C. Li, F. Li, H. Li, M. Li, W. Li, Y. Li, Y. Li, Z. Li, Z. Li, H. Lin, X. Lin, Z. Lin, C. Liu, C. Liu, H. Liu, J. Liu, J. Liu, L. Liu, S. Liu, T. Y. Liu, T. Liu, W. Liu, Y. Liu, Y. Liu, Y. Liu, Y. Liu, Z. Liu, E. Lu, L. Lu, S. Ma, X. Ma, Y. Ma, S. Mao, J. Mei, X. Men, Y. Miao, S. Pan, Y. Peng, R. Qin, B. Qu, Z. Shang, L. Shi, S. Shi, F. Song, J. Su, Z. Su, X. Sun, F. Sung, H. Tang, J. Tao, Q. Teng, C. Wang, D. Wang, F. Wang, and H. Wang. Kimi K2: open agentic intelligence. CoRR, abs/2507.20534, 2025a. URL <https://doi.org/10.48550/arXiv.2507.20534>.
- Y. Bai, S. Tu, J. Zhang, H. Peng, X. Wang, X. Lv, S. Cao, J. Xu, L. Hou, Y. Dong, et al. Longbench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 3639–3664, 2025b.

此外，除了MoE和稀疏注意力架构之外，我们还将主动探索模型稀疏化的新维度——例如更稀疏的嵌入模块（Cheng等人，2026）——以在不牺牲能力的前提下进一步提升计算和内存效率。我们也将持续研究低延迟架构与系统技术，使长上下文部署与交互更加迅捷。同时，我们认识到长周期、多轮次代理任务的重要性与实际价值，并将持续迭代探索这一方向。我们还在致力于为模型整合多模态能力。最后，我们承诺开发更优的数据筛选与合成策略，以持续增强模型在日益广泛场景和任务中的智能性、鲁棒性及实际可用性。

参考文献

AA. Gdpval-aa 排行榜, 2025。网址 <https://artificialanalysis.ai/methodology/intelligence-benchmarking#gdpval-aa>。T. Achim、A. Best、A. Bietti、K. Der、M. Féderico、S. Gukov、D. Halpern-Leistner、K. Henningsgard、Y. Kudryashov、A. Meiburg 等。Aristotle: IMO级自动定理证明。arXiv预印本 arXiv:2510.01346, 2025年。A. Agache、M. Brooker、A. Florescu、A. Iordache、A. Liguori、R. Neugebauer、P. Piwonka 和 D.-M. Popa。Firecracker: 无服务器应用的轻量级虚拟化。载于《第17届USENIX网络系统设计与实现会议论文集》, NSDI' 20, 第419–434页, 美国, 2020年。USENIX协会。ISBN 9781939133137。O. J. Aimuyo、B. Oh 和 R. Singh。Flashmoe: 单一内核中的快速分布式专家混合。神经信息处理系统进展, 2025年。网址 <https://neurips.cc/virtual/2025/poster/119124>。J. Ainslie、J. Lee-Thorp、M. de Jong、Y. Zemlyanskiy、F. Lebrón 和 S. Sanghai。Gqa: 从多头检查点训练广义多查询Transformer模型。arXiv预印本 arXiv:2305.13245, 2023年。J. Asher。LeanExplore: Lean 4声明的搜索引擎, 2025年。网址 <https://arxiv.org/abs/2506.11085>。Y. Bai、Y. Bao、G. Chen、J. Chen、N. Chen、R. Chen、Y. Chen、Y. Chen、Z. Chen、J. Cui、H. Ding、M. Dong、A. Du、C. Du、D. Du、Y. Du、Y. Fan、Y. Feng、K. Fu、B. Gao、H. Gao、P. Gao、T. Gao、X. Gu、L. Guan、H. Guo、J. Guo、H. Hu、X. Hao、T. He、W. He、W. He、C. Hong、Y. Hu、Z. Hu、W. Huang、Z. Huang、Z. Huang、T. Jiang、Z. Jiang、X. Jin、Y. Kang、G. Lai、C. Li、F. Li、H. Li、M. Li、W. Li、Y. Li、Y. Li、Z. Li、Z. Li、H. Lin、X. Lin、Z. Lin、C. Liu、C. Liu、H. Liu、J. Liu、J. Liu、L. Liu、S. Liu、T. Y. Liu、T. Liu、W. Liu、Y. Liu、Y. Liu、Y. Liu、Z. Liu、E. Lu、L. Lu、S. Ma、X. Ma、Y. Ma、S. Mao、J. Mei、X. Men、Y. Miao、S. Pan、Y. Peng、R. Qin、B. Qu、Z. Shang、L. Shi、S. Shi、F. Song、J. Su、Z. Su、X. Sun、F. Sung、H. Tang、J. Tao、Q. Teng、C. Wang、D. Wang、F. Wang 和 H. Wang。Kimi K2: 开放式代理智能。《CoRR》, abs/2507.20534, 2025a。网址 <https://doi.org/10.48550/arXiv.2507.20534>。

Y. Bai, S. Tu, J. Zhang, H. Peng, X. Wang, X. Lv, S. Cao, J. Xu, L. Hou, Y. Dong, 等。Longbench v2: 面向现实长上下文多任务的深度理解与推理。载于《第63届计算语言学协会年会论文集（第一卷：长论文）》, 第3639–3664页, 2025b。

- M. Balunović, J. Dekoninck, I. Petrov, N. Jovanović, and M. Vechev. Matharena: Evaluating llms on uncontaminated math competitions. Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmark, 2025.
- C. Bandi, B. Hertzberg, G. Boo, T. Polakam, J. Da, S. Hassaan, M. Sharma, A. Park, E. Hernandez, D. Rambado, et al. Mcp-atlas: A large-scale benchmark for tool-use competency with real mcp servers. arXiv preprint arXiv:2602.00933, 2026.
- F. Bellard. Qemu, a fast and portable dynamic translator. In Proceedings of the Annual Conference on USENIX Annual Technical Conference, ATEC '05, page 41, USA, 2005. USENIX Association.
- I. Bello, H. Pham, Q. V. Le, M. Norouzi, and S. Bengio. Neural combinatorial optimization with reinforcement learning, 2017. URL <https://openreview.net/forum?id=rJY3vK9eg>.
- J. Chen, W. Chen, J. Du, J. Hu, Z. Jiang, A. Jie, X. Jin, X. Jin, C. Li, W. Shi, Z. Wang, M. Wang, C. Wei, S. Wei, H. Xin, F. Yang, W. Gao, Z. Yuan, T. Zhan, Z. Zheng, T. Zhou, and T. H. Zhu. Seed-prover 1.5: Mastering undergraduate-level theorem proving via learning from experience, 2025. URL <https://arxiv.org/abs/2512.17260>.
- M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba. Evaluating large language models trained on code. CoRR, abs/2107.03374, 2021. URL <https://arxiv.org/abs/2107.03374>.
- T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy. TVM: An automated End-to-End optimizing compiler for deep learning. In 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18), pages 578–594, Carlsbad, CA, Oct. 2018. USENIX Association. ISBN 978-1-939133-08-3. URL <https://www.usenix.org/conference/osdi18/presentation/chen>.
- A. Cheng, A. Jacovi, A. Globerson, B. Golan, C. Kwong, C. Alberti, C. Tao, E. Ben-David, G. S. Tomar, L. Haas, et al. The facts leaderboard: A comprehensive benchmark for large language model factuality. arXiv preprint arXiv:2512.10791, 2025.
- X. Cheng, W. Zeng, D. Dai, Q. Chen, B. Wang, Z. Xie, K. Huang, X. Yu, Z. Hao, Y. Li, H. Zhang, H. Zhang, D. Zhao, and W. Liang. Conditional memory via scalable lookup: A new axis of sparsity for large language models. CoRR, abs/2601.07372, 2026. doi: 10.48550/ARXIV.2601.07372. URL <https://doi.org/10.48550/arXiv.2601.07372>.
- K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, et al. Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168, 2021.
- D. Dai, C. Deng, C. Zhao, R. X. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu, Z. Xie, Y. K. Li, P. Huang, F. Luo, C. Ruan, Z. Sui, and W. Liang. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. CoRR, abs/2401.06066, 2024. URL <https://doi.org/10.48550/arXiv.2401.06066>.

M. Balunović、J. Dekoninck、I. Petrov、N. Jovanović与M. Vechev合著。《Matharena：在无污染数学竞赛中评估LLMs》。收录于《神经信息处理系统数据集与基准追踪会议录》，2025年。C. Bandi、B. Hertzberg、G. Boo、T. Polakam、J. Da、S. Hassaan、M. Sharma、A. Park、E. Hernandez、D. Rambado等人合作。《Mcp-atlas：基于真实MCP服务器的大规模工具使用能力基准测试》。arXiv预印本arXiv:2602.00933，2026年。F. Bellard。《QEMU：一种快速便携的动态翻译器》。载于《USENIX年度技术会议年会论文集》（ATEC '05），第41页，美国，2005年。USENIX协会出版。I. Bello、H. Pham、Q. V. Le、M. Norouzi及S. Bengio。《基于强化学习的神经组合优化》，2017年。网址<https://openreview.net/forum?id=rJY3vK9eg>。

J. Chen, W. Chen, J. Du, J. Hu, Z. Jiang, A. Jie, X. Jin, X. Jin, C. Li, W. Shi, Z. Wang, M. Wang, C. Wei, S. Wei, H. Xin, F. Yang, W. Gao, Z. Yuan, T. Zhan, Z. Zheng, T. Zhou, 与 T. H. Zhu。Seed-prover 1.5：通过经验学习掌握本科水平定理证明，2025。URL <https://arxiv.org/abs/2512.17260>。

M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, 和 W. Zaremba。评估基于代码训练的大型语言模型。CoRR, abs/2107.03374, 2021。URL {v*}。

T. Chen、T. Moreau、Z. Jiang、L. Zheng、E. Yan、H. Shen、M. Cowan、L. Wang、Y. Hu、L. Ceze、C. Guestrin与A. Krishnamurthy。TVM：一种端到端自动优化的深度学习编译器。见《第13届USENIX操作系统设计与实现研讨会（OSDI 18）》，第578–594页，美国加州卡尔斯巴德，2018年10月。USENIX协会。ISBN 978-1-939133-08-3。网址 <https://www.usenix.org/conference/osdi18/presentation/chen>。

A. Cheng, A. Jacovi, A. Globerson, B. Golan, C. Kwong, C. Alberti, C. Tao, E. Ben-David, G. S. Tomar, L. Haas等。《事实排行榜：大型语言模型事实性的综合基准》。arXiv预印本arXiv:2512.10791，2025年。

X. Cheng, W. Zeng, D. Dai, Q. Chen, B. Wang, Z. Xie, K. Huang, X. Yu, Z. Hao, Y. Li, H. Zhang, H. Zhang, D. Zhao, 与 W. Liang。基于可扩展查找的条件记忆：大语言模型稀疏性的新维度。《CoRR》，卷abs/2601.07372，2026年。doi: 10.48550/ARXIV.2601.07372。网址{v*}。

K. Cobbe、V. Kosaraju、M. Bavarian、M. Chen、H. Jun、L. Kaiser、M. Plappert、J. Tworek、J. Hilton、R. Nakano 等。《训练验证器解决数学应用题》。arXiv预印本arXiv:2110.14168，2021年。

D. Dai、C. Deng、C. Zhao、R. X. Xu、H. Gao、D. Chen、J. Li、W. Zeng、X. Yu、Y. Wu、Z. Xie、Y. K. Li、P. Huang、F. Luo、C. Ruan、Z. Sui与W. Liang。Deepseekmoe：迈向混合专家语言模型中专家专业化的终极境界。《CoRR》，abs/2401.06066，2024年。网址 <https://doi.org/10.48550/arXiv.2401.06066>。

- T. Dao, D. Haziza, F. Massa, and G. Sizov. Flash-decoding for long-context inference, 2023. URL <https://pytorch.org/blog/flash-decoding/>.
- L. De Moura and N. Bjørner. Z3: an efficient smt solver. In *Proceedings of the Theory and Practice of Software, 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems, TACAS'08/ETAPS'08*, page 337–340, Berlin, Heidelberg, 2008. Springer-Verlag. ISBN 3540787992.
- DeepSeek-AI. Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence. *CoRR*, abs/2406.11931, 2024. URL <https://doi.org/10.48550/arXiv.2406.11931>.
- DeepSeek-AI. Deepseek-v3 technical report. *CoRR*, abs/2412.19437, 2024. URL <https://doi.org/10.48550/arXiv.2412.19437>.
- DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *CoRR*, abs/2405.04434, 2024. URL <https://doi.org/10.48550/arXiv.2405.04434>.
- DeepSeek-AI. Fire-flyer file system, 2025. URL <https://github.com/deepseek-ai/3FS>.
- DeepSeek-AI. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nat.*, 645(8081):633–638, 2025. URL <https://doi.org/10.1038/s41586-025-09422-z>.
- DeepSeek-AI. Deepseek-v3.2: Pushing the frontier of open large language models, 2025. URL <https://arxiv.org/abs/2512.02556>.
- X. Deng, J. Da, E. Pan, Y. Y. He, C. Ide, K. Garg, N. Lauffer, A. Park, N. Pasari, C. Rane, K. Sampath, M. Krishnan, S. Kundurthy, S. Hendryx, Z. Wang, V. Bharadwaj, J. Holm, R. Aluri, C. B. C. Zhang, N. Jacobson, B. Liu, and B. Kenstler. Swe-bench pro: Can ai agents solve long-horizon software engineering tasks?, 2025. URL <https://arxiv.org/abs/2509.16941>.
- H. Ding, Z. Wang, G. Paolini, V. Kumar, A. Deoras, D. Roth, and S. Soatto. Fewer truncations improve language modeling. *arXiv preprint arXiv:2404.10830*, 2024.
- X. Dong, Y. Fu, S. Diao, W. Byeon, Z. CHEN, A. S. Mahabaleshwarkar, S.-Y. Liu, M. V. keirsbilck, M.-H. Chen, Y. Suhara, Y. C. Lin, J. Kautz, and P. Molchanov. Hymba: A hybrid-head architecture for small language models. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=A1ztozypga>.
- X. Du, Y. Yao, K. Ma, B. Wang, T. Zheng, K. Zhu, M. Liu, Y. Liang, X. Jin, Z. Wei, et al. Supergpqa: Scaling llm evaluation across 285 graduate disciplines. *arXiv preprint arXiv:2502.14739*, 2025.
- D. Dua, Y. Wang, P. Dasigi, G. Stanovsky, S. Singh, and M. Gardner. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In J. Burstein, C. Doran, and T. Solorio, editors, *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*, pages 2368–2378. Association for Computational Linguistics, 2019. doi: 10.18653/V1/N19-1246. URL <https://doi.org/10.18653/v1/n19-1246>.
- X. Gao, M. Dong, X. Miao, W. Du, C. Yu, and H. Chen. Erofs: a compression-friendly readonly file system for resource-scarce devices. In *Proceedings of the 2019 USENIX Conference on Usenix Annual Technical Conference, USENIX ATC '19*, page 149–162, USA, 2019. USENIX Association. ISBN 9781939133038.

T. Dao、D. Haziza、F. Massa 与 G. Sizov。《长上下文推理的闪存解码》，2023年。网址 <https://pytorch.org/blog/flash-decoding/>。L. De Moura 与 N. Bjørner。《Z3: 高效 SMT 求解器》，收录于《软件理论与实践: 第14届工具与算法构建及分析系统国际会议论文集》(TACAS'08/ETAPS'08)，第337–340页，柏林/海德堡，2008年。Springer-Verlag出版社。ISBN 3540787992。DeepSeek-AI。《Deepseek-coder-v2: 打破代码智能中闭源模型的壁垒》，CoRR期刊，abs/2406.11931，2024年。网址<https://doi.org/10.48550/arXiv.2406.11931>。

DeepSeek-AI. Deepseek-v3技术报告. CoRR, abs/2412.19437, 2024. URL <https://doi.org/10.48550/arXiv.2412.19437>.

深度求索AI. DeepSeek-v2: 一款强大、经济且高效的专家混合语言模型. CoRR, abs/2405.04434, 2024. URL <https://doi.org/10.48550/arXiv.2405.04434>.

深度求索AI. 萤火虫文件系统, 2025. 网址<https://github.com/deepseek-ai/3FS>.

DeepSeek-AI. Deepseek-r1通过强化学习激励大语言模型中的推理能力。《自然》，645(8081):633–638，2025年。URL {v*}。

DeepSeek-AI. Deepseek-v3.2: 推进开放大语言模型的前沿，2025年。URL {v*}。

X. Deng, J. Da, E. Pan, Y. Y. He, C. Ide, K. Garg, N. Lauffer, A. Park, N. Pasari, C. Rane, K. Sampath, M. Krishnan, S. Kundurthy, S. Hendryx, Z. Wang, V. Bharadwaj, J. Holm, R. Aluri, C. B. C. Zhang, N. Jacobson, B. Liu, 及 B. Kenstler。Swe-bench pro: AI代理能否解决长期软件工程任务?，2025。网址 {v*}。

H. Ding、Z. Wang、G. Paolini、V. Kumar、A. Deoras、D. Roth 和 S. Soatto。减少截断可提升语言建模效果。arXiv预印本 arXiv:2404.10830，2024年。

X. Dong, Y. Fu, S. Diao, W. Byeon, Z. Chen, A. S. Mahabaleshwarkar, S.-Y. Liu, M. V. keirsbilck, M.-H. Chen, Y. Suhara, Y. C. Lin, J. Kautz, 与 P. Molchanov。Hymba: 面向小型语言模型的混合头架构。载于第十三届国际学习表征会议，2025。URL {v*}。

杜旭、姚远、马柯、王斌、郑涛、朱坤、刘明、梁毅、金鑫、魏征等。SuperGPQA: 在285个研究生学科中扩展LLM评估。arXiv预印本arXiv:2502.14739，2025年。

D. Dua, Y. Wang, P. Dasigi, G. Stanovsky, S. Singh, 和 M. Gardner。DROP: 一个需要对段落进行离散推理的阅读理解基准测试。载于J. Burstein、C. Doran和T. Solorio编，《2019年北美计算语言学协会人类语言技术会议论文集》，NAACL-HLT 2019，美国明尼苏达州明尼阿波利斯市，2019年6月2-7日，第1卷（长文与短文），第2368–2378页。计算语言学协会，2019年。doi: 10.18653/V1/N19-1246。URL {v*}。

X. Gao, M. Dong, X. Miao, W. Du, C. Yu, 和 H. Chen。Erofs: 面向资源稀缺设备的压缩友好型只读文件系统。载于《2019年USENIX年度技术会议论文集》，USENIX ATC '19，第149–162页，美国，2019年。USENIX协会。ISBN 9781939133038。

- A. P. Gema, J. O. J. Leang, G. Hong, A. Devoto, A. C. M. Mancino, R. Saxena, X. He, Y. Zhao, X. Du, M. R. G. Madani, C. Barale, R. McHardy, J. Harris, J. Kaddour, E. van Krieken, and P. Minervini. Are we done with mmlu? *CoRR*, abs/2406.04127, 2024. URL <https://doi.org/10.48550/arXiv.2406.04127>.
- F. Gloeckle, B. Y. Idrissi, B. Rozière, D. Lopez-Paz, and G. Synnaeve. Better & faster large language models via multi-token prediction. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=pEWAcEjiU2>.
- Y. Gu, L. Dong, F. Wei, and M. Huang. Minillm: Knowledge distillation of large language models. In *The Twelfth International Conference on Learning Representations*, 2024.
- L. Haas, G. Yona, G. D’Antonio, S. Goldshtein, and D. Das. Simpleqa verified: A reliable factuality benchmark to measure parametric knowledge. *arXiv preprint arXiv:2509.07968*, 2025.
- Y. He, S. Li, J. Liu, Y. Tan, W. Wang, H. Huang, X. Bu, H. Guo, C. Hu, B. Zheng, et al. Chinese simpleqa: A chinese factuality evaluation for large language models. *arXiv preprint arXiv:2411.07140*, 2024.
- D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.
- D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.
- Y. Huang, Y. Bai, Z. Zhu, J. Zhang, J. Zhang, T. Su, J. Liu, C. Lv, Y. Zhang, J. Lei, et al. C-Eval: A multi-level multi-discipline chinese evaluation suite for foundation models. *arXiv preprint arXiv:2305.08322*, 2023.
- D. Hupkes and N. Bogoychev. Multiloko: a multilingual local knowledge benchmark for llms spanning 31 languages. *CoRR*, abs/2504.10356, 2025. doi: 10.48550/ARXIV.2504.10356. URL <https://doi.org/10.48550/arXiv.2504.10356>.
- B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2018.
- N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.
- K. Jordan, Y. Jin, V. Boza, J. You, F. Cesista, L. Newhouse, and J. Bernstein. Muon: An optimizer for hidden layers in neural networks. *Cited on*, page 10, 2024.
- M. Joshi, E. Choi, D. Weld, and L. Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In R. Barzilay and M.-Y. Kan, editors, *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1601–1611, Vancouver, Canada, July 2017. Association for Computational Linguistics. doi: 10.18653/v1/P17-1147. URL <https://aclanthology.org/P17-1147>.

A. P. Gema, J. O. J. Leang, G. Hong, A. Devoto, A. C. M. Mancino, R. Saxena, X. He, Y. Zhao, X. Du, M. R. G. Madani, C. Barale, R. McHardy, J. Harris, J. Kaddour, E. van Krieken 和 P. Minervini. 我们是否已完成MMLU? CoRR, abs/2406.04127, 2024. URL <https://doi.org/10.48550/arXiv.2406.04127>.

F. Gloeckle, B. Y. Idrissi, B. Rozière, D. Lopez-Paz 和 G. Synnaeve. 通过多令牌预测实现更优更快的语言大模型。载于《第四十一届国际机器学习会议》(ICML 2024), 2024年7月21-27日, 奥地利维也纳。OpenReview.net, 2024年。URL {v*}。

Y. Gu, L. Dong, F. Wei, 与 M. Huang. Minillm: 大语言模型的知识蒸馏。收录于第十二届国际学习表征会议, 2024。

L. Haas, G. Yona, G. D' Antonio, S. Goldshtein 和 D. Das. 《Simpleqa 验证: 衡量参数化知识的可靠事实性基准》。arXiv预印本 arXiv:2509.07968, 2025年。

Y. He, S. Li, J. Liu, Y. Tan, W. Wang, H. Huang, X. Bu, H. Guo, C. Hu, B. Zheng, 等。Chinese SimpleQA: 面向大语言模型的中文事实性评估。arXiv预印本 arXiv:2411.07140, 2024年。

D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song 与 J. Steinhardt 合著。《评估海量多任务语言理解能力》。arXiv预印本, 编号arXiv:2009.03300, 2020年。

D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song 和 J. Steinhardt. 使用数学数据集衡量数学问题解决能力。arXiv预印本 arXiv:2103.03874, 2021年。

黄毅, 白雨, 朱子俊, 张俊, 张杰, 苏涛, 刘杰, 吕晨, 张颖, 雷静, 等。C-Eval: 面向基础模型的多层次多学科中文评估套件。arXiv预印本 arXiv:2305.08322, 2023年。

D. Hupkes 与 N. Bogoychev. 《Multiloko: 覆盖31种语言的大语言模型多语言本地知识基准测试》。CoRR, abs/2504.10356, 2025年。doi: 10.48550/ARXIV.2504.10356. URL <https://doi.org/10.48550/arXiv.2504.10356>。

B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, 与 D. Kalenichenko. 量化与训练神经网络以实现纯整数算术高效推理。见《IEEE计算机视觉与模式识别会议(CVPR)论文集》, 2018年6月。

N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, 和 I. Stoica. Livecodebench: 针对代码大型语言模型的整体且无污染评估。arXiv预印本 arXiv:2403.07974, 2024。

K. Jordan, Y. Jin, V. Boza, J. You, F. Cesista, L. Newhouse, 和 J. Bernstein. Muon: 神经网络隐藏层的优化器。引用自2024年, 第10页。

M. Joshi, E. Choi, D. Weld 与 L. Zettlemoyer 合著。《TriviaQA: 一个大规模远程监督的阅读理解挑战数据集》。收录于 R. Barzilay 与 M.-Y. Kan 主编的《第55届计算语言学协会年会论文集(第一卷: 长论文)》, 第1601-1611页, 加拿大温哥华, 2017年7月。计算语言学协会出版。doi: 10.18653/v1/P17-1147. URL {v*}。

- H. Li, Y. Yuan, R. Du, K. Ma, L. Liu, and W. Hsu. DADI: Block-Level image service for agile and elastic application deployment. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pages 727–740. USENIX Association, July 2020. ISBN 978-1-939133-14-4. URL <https://www.usenix.org/conference/atc20/presentation/li-huiba>.
- H. Li, Y. Zhang, F. Koto, Y. Yang, H. Zhao, Y. Gong, N. Duan, and T. Baldwin. CMMLU: Measuring massive multitask language understanding in Chinese. *arXiv preprint arXiv:2306.09212*, 2023.
- J. Li, W. Zhao, J. Zhao, W. Zeng, H. Wu, X. Wang, R. Ge, Y. Cao, Y. Huang, W. Liu, et al. The tool decathlon: Benchmarking language agents for diverse, realistic, and long-horizon task execution. *arXiv preprint arXiv:2510.25726*, 2025.
- Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE: speculative sampling requires rethinking feature uncertainty. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=1NdN7eXyb4>.
- J. Liu, J. Su, X. Yao, Z. Jiang, G. Lai, Y. Du, Y. Qin, W. Xu, E. Lu, J. Yan, Y. Chen, H. Zheng, Y. Liu, S. Liu, B. Yin, W. He, H. Zhu, Y. Wang, J. Wang, M. Dong, Z. Zhang, Y. Kang, H. Zhang, X. Xu, Y. Zhang, Y. Wu, X. Zhou, and Z. Yang. Muon is scalable for LLM training. *CoRR*, abs/2502.16982, 2025. URL <https://doi.org/10.48550/arXiv.2502.16982>.
- I. Loshchilov and F. Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.
- K. Lu and T. M. Lab. On-policy distillation. Thinking Machines Lab: Connectionism, 2025. doi: 10.64434/tml.20251026. <https://thinkingmachines.ai/blog/on-policy-distillation>.
- Z. Lu, C. Li, Y. Shi, W. Shen, M. Yan, and F. Huang. Corpusqa: A 10 million token benchmark for corpus-level analysis and reasoning. *arXiv preprint arXiv:2601.14952*, 2026.
- T. Luong, D. Hwang, H. H. Nguyen, G. Ghiasi, Y. Chervonyi, I. Seo, J. Kim, G. Bingham, J. Lee, S. Mishra, A. Zhai, C. H. Hu, H. Michalewski, J. Kim, J. Ahn, J. Bae, X. Song, T. H. Trinh, Q. V. Le, and J. Jung. Towards robust mathematical reasoning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025. URL <https://aclanthology.org/2025.emnlp-main.1794/>.
- M. A. Merrill, A. G. Shaw, N. Carlini, B. Li, H. Raj, I. Bercovich, L. Shi, J. Y. Shin, T. Walshe, E. K. Buchanan, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868*, 2026.
- MiniMax. Meet minimax-m2, 2025. URL <https://github.com/MiniMax-AI/MiniMax-M2>.
- L. d. Moura and S. Ullrich. The lean 4 theorem prover and programming language. In *International Conference on Automated Deduction*, pages 625–635. Springer, 2021.
- Y. Nesterov. A method of solving a convex programming problem with convergence rate $O(1/k^2)$. *Soviet Mathematics Doklady*, 27:372–376, 1983.
- NVIDIA Corporation. cublas documentation, 2024. URL <https://docs.nvidia.com/cuda/cublas/>. Version 12.4. Accessed: 2024-09-16.
- OpenAI. Multilingual massive multitask language understanding (mmmlu), 2024a. URL <https://huggingface.co/datasets/openai/MMMLU>.

H. Li, Y. Yuan, R. Du, K. Ma, L. Liu 和 W. Hsu. DADI: 面向敏捷弹性应用部署的块级镜像服务。载于《2020 USENIX 年度技术会议 (USENIX ATC 20) 》, 第727–740页。USENIX协会, 2020年7月。ISBN 978-1-939133-14-4。网址{v*}。

H. Li, Y. Zhang, F. Koto, Y. Yang, H. Zhao, Y. Gong, N. Duan, 与 T. Baldwin. CMMLU: 衡量中文大规模多任务语言理解能力。arXiv预印本arXiv:2306.09212, 2023年。

李杰, 赵伟, 赵静, 曾文, 吴昊, 王鑫, 葛荣, 曹阳, 黄毅, 刘伟等. 工具十项全能: 面向多样化、现实化及长周期任务执行的语言智能体基准测试. arXiv预印本 arXiv:2510.25726, 2025.

Y. Li, F. Wei, C. Zhang 和 H. Zhang. EAGLE: 投机采样需重新思考特征不确定性。载于第四十一届国际机器学习会议 (ICML 2024), 奥地利维也纳, 2024年7月21-27日。OpenReview.net, 2024。网址 <https://openreview.net/forum?id=1NdN7eXyb4>。

刘杰、苏杰、姚翔、江哲、赖国栋、杜宇、秦勇、许伟、陆恩、严静、陈阳、郑华、刘洋、刘松、殷斌、何伟、朱浩、王勇、王军、董明、张哲、康宇、张华、徐翔、张阳、吴勇、周翔、杨哲。 μ 子在大型语言模型训练中具有可扩展性。《CoRR》, abs/2502.16982, 2025年。网址 <https://doi.org/10.48550/arXiv.2502.16982>。

I. Loshchilov 和 F. Hutter. 解耦权重衰减正则化。arXiv预印本 arXiv:1711.05101, 2017年。

K. Lu 和 T. M. Lab. 《在线策略蒸馏》。Thinking Machines Lab: 连接主义, 2025年。doi: 10.64434/tml.20251026。 <https://thinkingmachines.ai/blog/on-policy-distillation>。

Z. Lu, C. Li, Y. Shi, W. Shen, M. Yan 和 F. Huang. CorpusQA: 一个用于语料库级分析与推理的千万标记基准。arXiv预印本 arXiv:2601.14952, 2026年。

T. Luong, D. Hwang, H. H. Nguyen, G. Ghiasi, Y. Chervonyi, I. Seo, J. Kim, G. Bingham, J. Lee, S. Mishra, A. Zhai, C. H. Hu, H. Michalewski, J. Kim, J. Ahn, J. Bae, X. Song, T. H. Trinh, Q. V. Le 以及 J. Jung. 《迈向稳健的数学推理》。载于《2025年自然语言处理实证方法会议论文集》, 2025年。网址 {v*}。

M. A. Merrill, A. G. Shaw, N. Carlini, B. Li, H. Raj, I. Bercovich, L. Shi, J. Y. Shin, T. Walsh e, E. K. Buchanan等. Terminal-bench: 在命令行界面中对智能体进行高难度现实任务的基准测试。arXiv预印本 arXiv:2601.11868, 2026年。

MiniMax。认识一下minimax-m2, 2025年。网址<https://github.com/MiniMax-AI/MiniMax-M2>。

L. d. Moura 与 S. Ullrich. Lean 4 定理证明器及编程语言。见《自动化演绎国际会议》, 第625–635页。Springer出版社, 2021年。

Y. Nesterov. 求解收敛速率为 $O(1/k^2)$ 的凸规划问题的方法. 苏联数学学报, 27:372–376, 1983.

英伟达公司. cublas 文档, 2024。网址 <https://docs.nvidia.com/cuda/cublas/>。版本 12.4。访问日期: 2024-09-16。

OpenAI. 多语言大规模多任务语言理解 (MMMLU), 2024a。网址 <https://huggingface.co/datasets/openai/MMMLU>。

- OpenAI. Openai mrcr: Long context multiple needle in a haystack benchmark, 2024b. URL <https://huggingface.co/datasets/openai/mrcr>.
- OpenAI. Learning to reason with llms, 2024c. URL <https://openai.com/index/learning-to-reason-with-llms>.
- OpenAI. Introducing SimpleQA, 2024d. URL <https://openai.com/index/introducing-simpleqa/>.
- OpenAI. Introducing SWE-bench verified we’re releasing a human-validated subset of swe-bench that more, 2024e. URL <https://openai.com/index/introducing-swe-bench-verified/>.
- OpenAI. gpt-oss-120b & gpt-oss-20b model card. *CoRR*, abs/2508.10925, 2025. doi: 10.48550/ARXIV.2508.10925. URL <https://doi.org/10.48550/arXiv.2508.10925>.
- M. Osama, D. Merrill, C. Cecka, M. Garland, and J. D. Owens. Stream-k: Work-centric parallel decomposition for dense matrix-matrix multiplication on the gpu. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, pages 429–431, 2023.
- T. Patwardhan, R. Dias, E. Proehl, G. Kim, M. Wang, O. Watkins, S. P. Fishman, M. Aljubei, P. Thacker, L. Fauconnet, et al. Gdpval: Evaluating ai model performance on real-world economically valuable tasks. *arXiv preprint arXiv:2510.04374*, 2025.
- L. Phan, A. Gatti, Z. Han, N. Li, J. Hu, H. Zhang, C. B. C. Zhang, M. Shaaban, J. Ling, S. Shi, et al. Humanity’s last exam. *arXiv preprint arXiv:2501.14249*, 2025.
- W. Qi, Y. Yan, Y. Gong, D. Liu, N. Duan, J. Chen, R. Zhang, and M. Zhou. Prophetnet: Predicting future n-gram for sequence-to-sequence pre-training. In T. Cohn, Y. He, and Y. Liu, editors, *Findings of the Association for Computational Linguistics: EMNLP 2020, Online Event, 16-20 November 2020*, volume EMNLP 2020 of *Findings of ACL*, pages 2401–2410. Association for Computational Linguistics, 2020. URL <https://doi.org/10.18653/v1/2020.findings-emnlp.217>.
- Qwen. Qwen3 technical report. *CoRR*, abs/2505.09388, 2025. doi: 10.48550/ARXIV.2505.09388. URL <https://doi.org/10.48550/arXiv.2505.09388>.
- S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–16. IEEE, 2020.
- J. K. Reed, Z. DeVito, H. He, A. Ussery, and J. Ansel. Torch.fx: Practical program capture and transformation for deep learning in python, 2022. URL <https://arxiv.org/abs/2112.08429>.
- D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, and S. R. Bowman. GPQA: A graduate-level google-proof q&a benchmark. *arXiv preprint arXiv:2311.12022*, 2023.
- G. T. M. Riviere, S. Pathak, P. G. Sessa, C. Hardin, S. Bhupatiraju, L. Hussenot, T. Mesnard, B. Shahriari, A. Ram’e, J. Ferret, P. Liu, P. D. Tafti, A. Friesen, M. Casbon, S. Ramos, R. Kumar, C. L. Lan, S. Jerome, A. Tsitsulin, N. Vieillard, P. Stańczyk, S. Girgin, N. Momchev, M. Hoffman, S. Thakoor, J.-B. Grill, B. Neyshabur, A. Walton, A. Severyn, A. Parrish, A. Ahmad, A. Hutchison, A. Abdagic, A. Carl, A. Shen, A. Brock, A. Coenen, A. Laforge, A. Paterson,

OpenAI. Openai mrcr: 长上下文多针寻干草堆基准测试, 2024b. 网址 {v*}。

OpenAI. 学习利用大语言模型进行推理, 2024c. URL <https://openai.com/index/learning-to-reason-with-llms>. OpenAI. 推出SimpleQA, 2024d. URL <https://openai.com/index/introducing-simpleqa/>. OpenAI. 介绍SWE-bench验证版: 我们发布了经过人工验证的swe-bench子集, 该子集包含更多内容, 2024e. URL <https://openai.com/index/introducing-swe-bench-verified/>.

OpenAI. gpt-oss-120b 和 gpt-oss-20b 模型卡片. CoRR, abs/2508.10925, 2025年. doi: 10.48550/ARXIV.2508.10925. URL {v*}。

M. Osama, D. Merrill, C. Cecka, M. Garland 与 J. D. Owens. 《Stream-k: 面向GPU稠密矩阵乘法的工作中心并行分解》。载于《第28届ACM SIGPLAN并行编程原理与实践年度研讨会论文集》, 第429–431页, 2023年。

T. Patwardhan, R. Dias, E. Proehl, G. Kim, M. Wang, O. Watkins, S. P. Fishman, M. Aljubeih, P. Thacker, L. Fauconnet等。GDPVal: 评估AI模型在现实世界经济价值任务中的表现. arXiv预印本 arXiv:2510.04374, 2025年。

L. Phan, A. Gatti, Z. Han, N. Li, J. Hu, H. Zhang, C. B. C. Zhang, M. Shaaban, J. Ling, S. Shi, 等. 人类最后的考验. arXiv预印本 arXiv:2501.14249, 2025。

W. Qi, Y. Yan, Y. Gong, D. Liu, N. Duan, J. Chen, R. Zhang, 和 M. Zhou. ProphetNet: 面向序列到序列预训练的未来n元预测。见T. Cohn, Y. He, 和 Y. Liu编, 《计算语言学协会发现: EMNLP 2020》, 在线会议, 2020年11月16-20日, ACL发现系列EMNLP 2020卷, 页码2401–2410。计算语言学协会, 2020年。URL <https://doi.org/10.18653/v1/2020.findings-emnlp.217>。

Qwen. Qwen3技术报告. CoRR, abs/2505.09388, 2025. doi: 10.48550/ARXIV.2505.09388. URL <https://doi.org/10.48550/arXiv.2505.09388>。

S. Rajbhandari, J. Rasley, O. Ruwase和Y. He. 《Zero: 面向训练万亿参数模型的内存优化》。收录于SC20: 国际高性能计算、网络、存储与分析会议, 第1-16页。IEEE, 2020年。

J. K. Reed, Z. DeVito, H. He, A. Ussery 和 J. Ansel. 《Torch.fx: Python 中深度学习的实用程序捕获与转换》, 2022年。URL {v*} {v*}。

D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, 和 S. R. Bowman. GPQA: 一项研究生级别防谷歌问答基准. arXiv预印本 arXiv:2311.12022, 2023年。

G. T. M. 里维埃尔, S. 帕塔克, P. G. 塞萨, C. 哈丁, S. 布帕蒂拉朱, L. 胡塞诺特, T. 梅斯纳德, B. 沙哈里亚里, A. 拉梅, J. 费雷特, P. 刘, P. D. 塔夫蒂, A. 弗里森, M. 卡斯本, S. 拉莫斯, R. 库马尔, C. L. 兰, S. 杰罗姆, A. 齐楚林, N. 维拉尔, P. 斯塔尼茨基, S. 吉尔金, N. 莫姆切夫, M. 霍夫曼, S. 塔库尔, J.-B. 格里尔, B. 内沙布尔, A. 沃尔顿, A. 塞韦林, A. 帕里什, A. 艾哈迈德, A. 哈奇森, A. 阿布达吉奇, A. 卡尔, A. 申, A. 布罗克, A. 科南, A. 拉福格, A. 佩特森,

- B. Bastian, B. Piot, B. Wu, B. Royal, C. Chen, C. Kumar, C. Perry, C. A. Welty, C. A. Choquette-Choo, D. Sinopalnikov, D. Weinberger, D. Vijaykumar, D. Rogozińska, D. Herbison, E. Bandy, E. Wang, E. Noland, E. Moreira, E. Senter, E. Eltyshv, F. Visin, G. Rasskin, G. Wei, G. Cameron, G. Martins, H. Hashemi, H. Klimczak-Plucińska, H. Batra, H. Dhand, I. Nardini, J. Mein, J. Zhou, J. Svensson, J. Stanway, J. Chan, J. Zhou, J. Carrasqueira, J. Iljazi, J. Becker, J. Fernandez, J. R. van Amersfoort, J. Gordon, J. Lipschultz, J. Newlan, J. Ji, K. Mohamed, K. Badola, K. Black, K. Millican, K. McDonell, K. Nguyen, K. Sodhia, K. Greene, L. L. Sjoesund, L. Usui, L. Sifre, L. Heuermann, L. cia Lago, L. McNealus, L. B. Soares, L. Kilpatrick, L. Dixon, L. L. B. Martins, M. Reid, M. Singh, M. Iverson, M. Gorner, M. Velloso, M. Wirth, M. Davidow, M. Miller, M. Rahtz, M. Watson, M. Risdal, M. Kazemi, M. Moynihan, M. Zhang, M. Kahng, M. Park, M. Rahman, M. Khatwani, N. Dao, N. shad Bardoliwalla, N. Devanathan, N. Dumai, N. Chauhan, O. Wahltinez, P. Botarda, P. Barnes, P. Barham, P. Michel, P. chong Jin, P. Georgiev, P. Culliton, P. Kuppala, R. Comanescu, R. Merhej, R. Jana, R. A. Rokni, R. Agarwal, R. Mullins, S. Saadat, S. M. M. Carthy, S. Perrin, S. M. R. Arnold, S. bastian Krause, S. Dai, S. Garg, S. Sheth, S. Ronstrom, S. Chan, T. Jordan, T. Yu, T. Eccles, T. Hennigan, T. Kociský, T. Doshi, V. Jain, V. Yadav, V. Meshram, V. Dharmadhikari, W. Barkley, W. Wei, W. Ye, W. Han, W. Kwon, X. Xu, Z. Shen, Z. Gong, Z. Wei, V. Cotruta, P. Kirk, A. Rao, M. Giang, L. Peran, T. Warkentin, E. Collins, J. Barral, Z. Ghahramani, R. Hadsell, D. Sculley, J. Banks, A. Dragan, S. Petrov, O. Vinyals, J. Dean, D. Hassabis, K. Kavukcuoglu, C. Farabet, E. Buchatskaya, S. Borgeaud, N. Fiedel, A. Joulin, K. Kenealy, R. Dadashi, and A. Andreev. Gemma 2: Improving open language models at a practical size. arXiv preprint arXiv:2408.00118, 2024.
- S. Roller, S. Sukhbaatar, A. Szlam, and J. Weston. Hash layers for large sparse models. In M. Ranzato, A. Beygelzimer, Y. N. Dauphin, P. Liang, and J. W. Vaughan, editors, Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual, pages 17555–17566, 2021. URL <https://proceedings.neurips.cc/paper/2021/hash/92bf5e6240737e0326ea59846a83e076-Abstract.html>.
- B. D. Rouhani, R. Zhao, A. More, M. Hall, A. Khodamoradi, S. Deng, D. Choudhary, M. Cornea, E. Dellinger, K. Denolf, S. Dusan, V. Elango, M. Golub, A. Heinecke, P. James-Roxby, D. Jani, G. Kolhe, M. Langhammer, A. Li, L. Melnick, M. Mesmakhosroshahi, A. Rodriguez, M. Schulte, R. Shafipour, L. Shao, M. Siu, P. Dubey, P. Micikevicius, M. Naumov, C. Verrilli, R. Wittig, D. Burger, and E. Chung. Microscaling data formats for deep learning, 2023.
- K. Sakaguchi, R. L. Bras, C. Bhagavatula, and Y. Choi. Winogrande: An adversarial winograd schema challenge at scale, 2019.
- Z. Shao, Y. Luo, C. Lu, Z. Z. Ren, J. Hu, T. Ye, Z. Gou, S. Ma, and X. Zhang. Deepseekmath-v2: Towards self-verifiable mathematical reasoning, 2025. URL <https://arxiv.org/abs/2511.22570>.
- N. Shazeer. Fast transformer decoding: One write-head is all you need. CoRR, abs/1911.02150, 2019. URL <http://arxiv.org/abs/1911.02150>.
- N. Shazeer. Glu variants improve transformer. arXiv preprint arXiv:2002.05202, 2020.
- F. Shi, M. Suzgun, M. Freitag, X. Wang, S. Srivats, S. Vosoughi, H. W. Chung, Y. Tay, S. Ruder, D. Zhou, D. Das, and J. Wei. Language models are multilingual chain-of-thought reasoners. In The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023. OpenReview.net, 2023. URL <https://openreview.net/forum?id=fr3wGCk-IXp>.

B. Bastian, B. Piot, B. Wu, B. Royal, C. Chen, C. Kumar, C. Perry, C. A. Welty, C. A. Choquette-Choo, D. Sinopalnikov, D. Weinberger, D. Vijaykumar, D. Rogozińska, D. Herbison, E. Bandy, E. Wang, E. Noland, E. Moreira, E. Senter, E. Eltyshv, F. Visin, G. Rasskin, G. Wei, G. Cameron, G. Martins, H. Hashemi, H. Klimczak-Plucińska, H. Batra, H. Dhand, I. Nardini, J. Mein, J. Zhou, J. Svensson, J. Stanway, J. Chan, J. Zhou, J. Carrasqueira, J. Iljazi, J. Becker, J. Fernandez, J. R. van Amersfoort, J. Gordon, J. Lipschultz, J. Newlan, J. Ji, K. Mohamed, K. Badola, K. Black, K. Millican, K. McDonell, K. Nguyen, K. Sodhia, K. Greene, L. L. Sjøstrand, L. Usui, L. Sifre, L. Heuermann, L. Cia Lago, L. McNealus, L. B. Soares, L. Kilpatrick, L. Dixon, L. L. B. Martins, M. Reid, M. Singh, M. Iverson, M. Görner, M. Velloso, M. Wirth, M. Davidow, M. Miller, M. Rahtz, M. Watson, M. Risdal, M. Kazemi, M. Moynihan, M. Zhang, M. Kahng, M. Park, M. Rahman, M. Khatwani, N. Dao, N. Shad Bardoliwalla, N. Devanathan, N. Dumai, N. Chauhan, O. Wahltinez, P. Botarda, P. Barnes, P. Barham, P. Michel, P. Chong Jin, P. Georgiev, P. Culliton, P. Kuppala, R. Comanescu, R. Merhej, R. Jana, R. A. Rokni, R. Agarwal, R. Mullins, S. Saadat, S. M. M. Carthy, S. Perrin, S. M. R. Arnold, S. Bastian Krause, S. Dai, S. Garg, S. Seth, S. Rönström, S. Chan, T. Jordan, T. Yu, T. Eccles, T. Hennigan, T. Koiský, T. Doshi, V. Jain, V. Yadav, V. Meshram, V. Dharmadhikari, W. Barkley, W. Wei, W. Ye, W. Han, W. Kwon, X. Xu, Z. Shen, Z. Gong, Z. Wei, V. Cotruta, P. Kirk, A. Rao, M. Giang, L. Peran, T. Warkentin, E. Collins, J. Barral, Z. Ghahramani, R. Hadsell, D. Sculley, J. Banks, A. Dragan, S. Petrov, O. Vinyals, J. Dean, D. Hassabis, K. Kavukcuoglu, C. Farabet, E. Buchatskaya, S. Borgeaud, N. Fiedel, A. Joulin, K. Kenealy, R. Dadashi, A. Andreev. Gemma 2: 实用规模下提升开放语言模型性能. arXiv预印本 arXiv:2408.00118, 2024. S. Roller, S. Sukhbaatar, A. Szlam, J. Weston. 大型稀疏模型的哈希层. 收录于M. Ranzato, A. Beygelzimer, Y. N. Dauphin, P. Liang, J. W. Vaughan编, 《神经信息处理系统进展34: 2021年神经信息处理系统年会》, NeurIPS 2021, 2021年12月6-14日, 虚拟会议, 第17555-17566页, 2021. 网址 <https://proceedings.neurips.cc/paper/2021/hash/92bf5e6240737e0326ea59846a83e076-Abstract.html>. B. D. Rouhani, R. Zhao, A. More, M. Hall, A. Khodamoradi, S. Deng, D. Choudhary, M. Cornea, E. Dellinger, K. Denolf, S. Dusan, V. Elango, M. Golub, A. Heinecke, P. James-Roxby, D. Jani, G. Kolhe, M. Langhammer, A. Li, L. Melnick, M. Mesmakhosroshahi, A. Rodriguez, M. Schulte, R. Shafipour, L. Shao, M. Siu, P. Dubey, P. Micikevicius, M. Naumov, C. Verrilli, R. Wittig, D. Burger, E. Chung. 深度学习微缩放数据格式, 2023. K. Sakaguchi, R. L. Bras, C. Bhagavatula, Y. Choi. Winogrande: 大规模对抗性Winograd模式挑战, 2019. Z. Shao, Y. Luo, C. Lu, Z. Z. Ren, J. Hu, T. Ye, Z. Gou, S. Ma, X. Zhang. Deepseekmath-v2: 迈向可自验证的数学推理, 2025. 网址 <https://arxiv.org/abs/2511.22570>. N. Shazeer. 快速Transformer解码: 单写头足矣. CoRR, abs/1911.02150, 2019. 网址 <http://arxiv.org/abs/1911.02150>. N. Shazeer. GLU变体改进Transformer. arXiv预印本 arXiv:2002.05202, 2020. F. Shi, M. Suzgun, M. Freitag, X. Wang, S. Srivats, S. Vosoughi, H. W. Chung, Y. Tay, S. Ruder, D. Zhou, D. Das, J. Wei. 语言模型是多语言思维链推理者. 收录于《第十一届国际学习表征会议》, ICLR 2023, 卢旺达基加利, 2023年5月1-5日. OpenReview.net, 2023. 网址 <https://openreview.net/forum?id=fR3wGCK-IXp>.

- J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.
- M. Suzgun, N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. V. Le, E. H. Chi, D. Zhou, et al. Challenging big-bench tasks and whether chain-of-thought can solve them. *arXiv preprint arXiv:2210.09261*, 2022.
- G. Tsoukalas, J. Lee, J. Jennings, J. Xin, M. Ding, M. Jennings, A. Thakur, and S. Chaudhuri. Putnambench: Evaluating neural theorem-provers on the putnam mathematical competition, 2024. URL <https://arxiv.org/abs/2407.11214>.
- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- L. Wang, H. Gao, C. Zhao, X. Sun, and D. Dai. Auxiliary-loss-free load balancing strategy for mixture-of-experts. *CoRR*, abs/2408.15664, 2024a. URL <https://doi.org/10.48550/arXiv.2408.15664>.
- L. Wang, Y. Cheng, Y. Shi, Z. Mo, Z. Tang, W. Xie, T. Wu, L. Ma, Y. Xia, J. Xue, et al. Tilelang: Bridge programmability and performance in modern neural kernels. In *The Fourteenth International Conference on Learning Representations*, 2026.
- Y. Wang, X. Ma, G. Zhang, Y. Ni, A. Chandra, S. Guo, W. Ren, A. Arulraj, X. He, Z. Jiang, T. Li, M. Ku, K. Wang, A. Zhuang, R. Fan, X. Yue, and W. Chen. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. *CoRR*, abs/2406.01574, 2024b. URL <https://doi.org/10.48550/arXiv.2406.01574>.
- J. Wei, Z. Sun, S. Papay, S. McKinney, J. Han, I. Fulford, H. W. Chung, A. T. Passos, W. Fedus, and A. Glaese. Browsecomp: A simple yet challenging benchmark for browsing agents. *arXiv preprint arXiv:2504.12516*, 2025.
- T. Wei, J. Luan, W. Liu, S. Dong, and B. Wang. Cmath: Can your language model pass chinese elementary school math test?, 2023.
- G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. Efficient streaming language models with attention sinks. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=NG7sS51zVF>.
- Z. Xie, Y. Wei, H. Cao, C. Zhao, C. Deng, J. Li, D. Dai, H. Gao, J. Chang, K. Yu, L. Zhao, S. Zhou, Z. Xu, Z. Zhang, W. Zeng, S. Hu, Y. Wang, J. Yuan, L. Wang, and W. Liang. mhc: Manifold-constrained hyper-connections, 2026. URL <https://arxiv.org/abs/2512.24880>.
- L. Xu, H. Hu, X. Zhang, L. Li, C. Cao, Y. Li, Y. Xu, K. Sun, D. Yu, C. Yu, Y. Tian, Q. Dong, W. Liu, B. Shi, Y. Cui, J. Li, J. Zeng, R. Wang, W. Xie, Y. Li, Y. Patterson, Z. Tian, Y. Zhang, H. Zhou, S. Liu, Z. Zhao, Q. Zhao, C. Yue, X. Zhang, Z. Yang, K. Richardson, and Z. Lan. CLUE: A chinese language understanding evaluation benchmark. In D. Scott, N. Bel, and C. Zong, editors, *Proceedings of the 28th International Conference on Computational Linguistics, COLING 2020, Barcelona, Spain (Online), December 8-13, 2020*, pages 4762–4772. International Committee on Computational Linguistics, 2020. doi: 10.18653/V1/2020.COLING-MAIN.419. URL <https://doi.org/10.18653/v1/2020.coling-main.419>.

J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, 与 Y. Liu. 旋转位置嵌入增强的Transformer: Roformer. 《神经计算》, 568卷:127063页, 2024年. M. Suzgun, N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. V. Le, E. H. Chi, D. Zhou, 等. 挑战Big-Bench任务及思维链能否解决之. arXiv预印本 arXiv:2210.09261, 2022年.

G. Tsoukalas, J. Lee, J. Jennings, J. Xin, M. Ding, M. Jennings, A. Thakur 和 S. Chaudhuri. Putnambench: 在普特南数学竞赛上评估神经定理证明器, 2024年. 网址 {v*}。

A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, . Kaiser 和 I. Polosukhin. 注意力就是您所需的一切. 神经信息处理系统进展, 30, 2017.

王立、高航、赵晨、孙晓、戴东. 面向专家混合模型的无辅助损失负载均衡策略. CoRR, abs/2408.15664, 2024a. URL <https://doi.org/10.48550/arXiv.2408.15664>.

L. Wang, Y. Cheng, Y. Shi, Z. Mo, Z. Tang, W. Xie, T. Wu, L. Ma, Y. Xia, J. Xue等. Tilelang: 现代神经核心中程序化与性能的桥梁. 收录于第十四届国际学习表征会议, 2026.

王Y.、马X.、张G.、倪Y.、钱德拉A.、郭S.、任W.、阿鲁拉吉A.、何X.、姜Z.、李T.、库M.、王K.、庄A.、范R.、岳X.、陈W.。MMLU-Pro: 一个更鲁棒且更具挑战性的多任务语言理解基准。CoRR, abs/2406.01574, 2024b. URL {v*}。

J. Wei, Z. Sun, S. Papay, S. McKinney, J. Han, I. Fulford, H. W. Chung, A. T. Passos, W. Fedus与A. Glaese. 《Browsecomp: 一个简单却极具挑战性的浏览智能体基准测试》。arXiv预印本 arXiv:2504.12516, 2025年.

T. Wei, J. Luan, W. Liu, S. Dong, 和 B. Wang. CMATH: 你的语言模型能通过中国小学数学考试吗?, 2023.

G. Xiao, Y. Tian, B. Chen, S. Han和M. Lewis. 《高效流式语言模型与注意力汇》。载于第十二届国际学习表征会议 (ICLR 2024), 2024年5月7-11日, 奥地利维也纳. OpenReview.net, 2024年. URL <https://openreview.net/forum?id=NG7sS51zVF>.

谢志、魏阳、曹晖、赵晨、邓超、李杰、戴东、高华、常健、余凯、赵亮、周松、许哲、张震、曾伟、胡硕、王洋、袁杰、王磊、梁伟. MHC: 流形约束的超连接, 2026. 网址 <https://arxiv.org/abs/2512.24880>.

L徐、胡、张、李、曹、李、徐、孙、余、余、田、董、刘、石、崔、李、曾、王、谢、李、Patterson、田、张、周、刘、赵、赵、岳、张、杨、Richardson与兰. CLUE: 一个中文语言理解评估基准. 载于D. Scott, N. Bel与C. Zong编辑的《第28届国际计算语言学会议论文集》, COLING 2020, 西班牙巴塞罗那 (线上), 2020年12月8-13日, 页码4762–4772. 国际计算语言学委员会, 2020年. doi: 10.18653/V1/2020.COLING-MAIN.419. URL {v*}.

- J. Yang, K. Lieret, C. E. Jimenez, A. Wettig, K. Khandpur, Y. Zhang, B. Hui, O. Press, L. Schmidt, and D. Yang. Swe-smith: Scaling data for software engineering agents, 2025. URL <https://arxiv.org/abs/2504.21798>.
- R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi. HellaSwag: Can a machine really finish your sentence? In A. Korhonen, D. R. Traum, and L. Màrquez, editors, *Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers*, pages 4791–4800. Association for Computational Linguistics, 2019. doi: 10.18653/v1/p19-1472. URL <https://doi.org/10.18653/v1/p19-1472>.
- C. Zhang, K. Du, S. Liu, W. Kwon, X. Mo, Y. Wang, X. Liu, K. You, Z. Li, M. Long, J. Zhai, J. Gonzalez, and I. Stoica. Jenga: Effective memory management for serving llm with heterogeneity. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles, SOSP '25*, page 446–461, New York, NY, USA, 2025a. Association for Computing Machinery. ISBN 9798400718700. doi: 10.1145/3731569.3764823. URL <https://doi.org/10.1145/3731569.3764823>.
- S. Zhang, N. Zheng, H. Lin, Z. Jiang, W. Bao, C. Jiang, Q. Hou, W. Cui, S. Zheng, L.-W. Chang, Q. Chen, and X. Liu. Comet: Fine-grained computation-communication overlapping for mixture-of-experts. 2025b. URL <https://arxiv.org/abs/2502.19811>.
- C. Zhao, L. Zhao, J. Li, Z. Xu, and C. Xu. Deepgemm: clean and efficient fp8 gemm kernels with fine-grained scaling. <https://github.com/deepseek-ai/DeepGEMM>, 2025.
- W. Zhong, R. Cui, Y. Guo, Y. Liang, S. Lu, Y. Wang, A. Saied, W. Chen, and N. Duan. AGIEval: A human-centric benchmark for evaluating foundation models. *CoRR*, abs/2304.06364, 2023. doi: 10.48550/arXiv.2304.06364. URL <https://doi.org/10.48550/arXiv.2304.06364>.
- D. Zhu, H. Huang, Z. Huang, Y. Zeng, Y. Mao, B. Wu, Q. Min, and X. Zhou. Hyperconnections. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025. URL <https://openreview.net/forum?id=9FqARW7dwB>.
- X. Zhu, D. Cheng, H. Li, K. Zhang, E. Hua, X. Lv, N. Ding, Z. Lin, Z. Zheng, and B. Zhou. How to synthesize text data without model collapse? *arXiv preprint arXiv:2412.14689*, 2024.
- T. Y. Zhuo, M. C. Vu, J. Chim, H. Hu, W. Yu, R. Widayarsi, I. N. B. Yusuf, H. Zhan, J. He, I. Paul, S. Brunner, C. Gong, J. Hoang, A. R. Zebaze, X. Hong, W. Li, J. Kaddour, M. Xu, Z. Zhang, P. Yadav, and et al. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025. URL <https://openreview.net/forum?id=YrycTj1lL0>.

J. Yang, K. Lieret, C. E. Jimenez, A. Wettig, K. Khandpur, Y. Zhang, B. Hui, O. Press, L. Schmidt 和 D. Yang。《Swe-smith: 为软件工程智能体扩展数据》，2025年。网址 <https://arxiv.org/abs/2504.21798>。

R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi 与 Y. Choi 合著。《HellaSwag: 机器真能完成你的句子吗?》收录于 A. Korhonen, D. R. Traum 和 L. Màrquez 编辑的《第57届计算语言学协会年会论文集 (ACL 2019)》，第1卷: 长论文, 第4791–4800页。意大利佛罗伦萨, 2019年7月28日至8月2日。计算语言学协会, 2019年出版。DOI: 10.18653/v1/p19-1472。URL <https://doi.org/10.18653/v1/p19-1472>。

张诚、杜康、刘帅、权伟、莫晓、王阳、刘欣、游凯、李正、龙明、翟佳、冈萨雷斯与斯托伊卡。Jenga: 面向异构化大语言模型服务的高效内存管理。见《ACM SIGOPS第31届操作系统原理研讨会论文集》，SOSP '25, 第446–461页, 美国纽约州, 2025a。美国计算机协会。ISBN 9798400718700。doi: 10.1145/3731569.3764823。URL {v*}。

张S.、郑N.、林H.、姜Z.、鲍W.、姜C.、侯Q.、崔W.、郑S.、常L.-W.、陈Q.、刘X.。Comet: 面向专家混合模型的细粒度计算-通信重叠优化。2025b。网址 <https://arxiv.org/abs/2502.19811>。

C. Zhao, L. Zhao, J. Li, Z. Xu 和 C. Xu。《Deepgemm: 具有细粒度缩放的高效清洁fp8通用矩阵乘法内核》。 <https://github.com/deepseek-ai/DeepGEMM>, 2025。

W. Zhong, R. Cui, Y. Guo, Y. Liang, S. Lu, Y. Wang, A. Saied, W. Chen, and N. Duan. AGIEval: 一个以人为中心的基准测试, 用于评估基础模型。《计算机研究预印本》(CoRR), 编号abs/2304.06364, 2023年。数字对象标识符: 10.48550/arXiv.2304.06364。网址{v*}。

朱东、黄浩、黄震、曾毅、毛勇、吴斌、闵琦和周晓。超连接。第十三届国际学习表征会议 (ICLR 2025), 新加坡, 2025年4月24-28日。OpenReview.net, 2025年。网址 <https://openreview.net/forum?id=9FqARW7dwB>。

X. Zhu, D. Cheng, H. Li, K. Zhang, E. Hua, X. Lv, N. Ding, Z. Lin, Z. Zheng, 和 B. Zhou。如何合成文本数据而不导致模型崩溃? arXiv预印本 arXiv:2412.14689, 2024年。

T. Y. Zhuo, M. C. Vu, J. Chim, H. Hu, W. Yu, R. Widiasari, I. N. B. Yusuf, H. Zhan, J. He, I. Paul, S. Brunner, C. Gong, J. Hoang, A. R. Zebaze, X. Hong, W. Li, J. Kaddour, M. Xu, Z. Zhang, P. Yadav等。Bigcodebench: 通过多样化函数调用与复杂指令评测代码生成能力。收录于第十三届国际学习表征会议 (ICLR 2025), 新加坡, 2025年4月24-28日。OpenReview.net, 2025年。网址 <https://openreview.net/forum?id=YrycTj1lL0>。

Appendix

A. Author List and Acknowledgment

A.1. Author List

Authors are listed alphabetically by their first name. Names marked with * denote individuals who have departed from our team.

Research & Engineering: Anyi Xu, Bangcai Lin, Bing Xue, Bingxuan Wang*, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenhao Xu, Chenze Shao, Chong Ruan*, Conner Sun, Damai Dai, Daya Guo*, Dejian Yang, Deli Chen, Donghao Li, Erhang Li, Fangyun Lin, Fangzhou Yuan, Feiyu Xia, Fucong Dai, Guangbo Hao, Guanting Chen, Guoai Cao, Guolai Meng, Guowei Li, Han Yu, Han Zhang, Hanwei Xu, Hao Li, Haofen Liang, Haoling Zhang, Haoming Luo, Haoran Wei*, Haotian Yuan, Haowei Zhang*, Haowen Luo, Haoyu Chen, Haozhe Ji, Honghui Ding, Hongxuan Tang, Huanqi Cao, Huazuo Gao, Hui Qu, Hui Zeng, J. Yang, J.Q. Zhu, Jia Yu, Jialiang Huang, Jiasheng Ye, Jiashi Li, Jiaxin Xu, Jiewen Hu, Jin Yan, Jingchang Chen, Jingli Zhou, Jingting Xiang, Jingyang Yuan, Jingyuan Cheng, Jinhua Zhu, Jiping Yu, Joseph Sun, Jun Ran*, Junguang Jiang, Junjie Qiu, Junlong Li*, Junxiao Song, Kai Dong, Kaige Gao, Kang Guan, Kexing Zhou, Kezhao Huang*, Kuai Yu, Lean Wang, Lecong Zhang, Lei Wang, Li Zhang, Liang Zhao, Lihua Guo, Lingxiao Luo, Linwang Ma, Litong Wang, Liyu Cai, Liyue Zhang, Longhao Chen, M.S. Di, M.Y. Xu, Max Mei, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Panpan Huang, Peixin Cong, Peiyi Wang, Qiancheng Wang, Qihao Zhu, Qingyang Li, Qinyu Chen, Qiushi Du, Qiwei Jiang, Rui Tian, Ruifan Xu, Ruijie Lu, Ruiling Xu, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runqian Chen, Runqiu Yin, Runxin Xu, Ruomeng Shen, Ruoyu Zhang, S.H. Liu, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaofei Cai, Shaoheng Nie, Shaoyuan Chen, Shengding Hu, Shengyu Liu, Shiqiang Hu, Shirong Ma, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, Shuying Yu, Songyang Zhou, Tao Ni, Tao Yun, Tian Jin, Tian Pei, Tian Ye, Tianle Lin, Tianran Ji, Tianyi Cui, Tianyuan Yue, Tingting Yu, Tun Wang, W. Zhang, Wangding Zeng, Weilin Zhao, Wen Liu, Wenfeng Liang, Wenjie Pang, Wenjing Luo, Wenjing Yao, Wenjun Gao, Wenkai Yang, Wenlve Huang, Wentao Zhang, Wenting Ma, Xi Gao, Xiang He, Xiangwen Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaokang Zhang, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingchen Liu, Xingkai Yu, Xingyou Li, Xinyu Yang, Xu Chen, Xuanyu Wang, Xuecheng Su, Xuheng Lin, Xuwei Fu, Y.C. Yan, Y.Q. Wang*, Y.W. Ma, Yanfeng Luo, Yang Zhang, Yanhong Xu, Yanru Ma, Yanwen Huang, Yao Li, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Qian, Yi Yu, Yichao Zhang, Yifan Ding, Yifan Shi, Yijia Wu, Yiliang Xiong, Ying He, Ying Zhou, Yingjia Luo, Yinmin Zhong, Yishi Piao, Yisong Wang, Yixiang Zhang, Yixiao Chen, Yixuan Tan, Yixuan Wei, Yiyang Ma, Yiyuan Liu, Yonglun Yang, Yongqiang Guo, Yongtong Wu, Yu Wu, Yuan Cheng, Yuan Ou, Yuanfan Xu, Yuanhao Li, Yudian Wang, Yuhao Wu, Yuhao Meng, Yuheng Zou, YuKun Li, Yunfan Xiong, Yupeng Chen, Yuqian Cao, Yuqian Wang, Yushun Zhang, Yutong Lin, Yuxian Gu, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuxuan Zhou, Yuyang Zhou, Yuzhen Huang, Z.F. Wu, Zehao Wang, Zehua Zhao, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhixian Huang, Zhixuan Chen, Zhiyu Wu, Zhizhou Ren, Zhuoshu Li, Zhuping Zhang, Zian Xu, Zihao Wang, Zihui Gu, Zijia Zhu, Zilin Li, Zipeng Zhang*, Ziwei Xie, Ziyi Gao, Zizheng Pan, Zongqing Yao.

Business & Compliance: Chenchen Ling, Chengyu Hou, Dongjie Ji, Fang Wei, Hengqing Zhang, Jia Luo, Jia Song, Jialu Cai, Jian Liang, Jiangting Zhou, Jieyu Yang, Jin Chen, Jingzi Zhou, Junmin Zheng, Leyi Xia, Linyan Zhu, Miaojuan Wang, Mingming Li, Minmin Han, Ning Wang, Panpan

附录

A. 作者列表与致谢

A.1. 作者列表

作者按名字首字母顺序排列。标有*的名字表示已离开我们团队的成员。

研究与工程：徐安怡、林邦才、薛冰、王冰轩*、徐炳轩、吴博超、张博威、林超凡、董晨、卢成达、赵成刚、邓成奇、徐晨浩、邵晨泽、阮冲*、孙康纳、戴达迈、郭大雅*、杨德建、陈德利、李东浩、李二航、林芳云、袁方舟、夏飞宇、戴富聪、郝广博、陈冠廷、曹国爱、孟国来、李国伟、余涵、张涵、徐汉威、李浩、梁浩奋、张浩玲、罗浩明、魏浩然*、袁昊天、张浩伟*、罗昊文、陈浩宇、纪浩哲、丁宏辉、唐宏轩、曹焕奇、高华卓、曲辉、曾慧、杨杰、朱建强、余佳、黄嘉亮、叶嘉盛、李家实、徐佳欣、胡杰文、严瑾、陈景昌、周景亮、向静婷、袁景阳、程景元、朱金华、余继平、孙约瑟、冉俊*、姜俊光、邱俊杰、李俊龙*、宋俊晓、董凯、高凯歌、管康、周克星、黄克钊*、于快、王乐聪、张乐聪、王磊、张力、赵亮、郭丽华、罗凌晓、马林旺、王立通、蔡立宇、张丽月、陈龙浩、迪明生、徐明远、梅麦克斯、张铭川、张铭华、唐铭辉、周铭旭、黄盼盼、丛培新、王培喜、王千乘、朱启豪、李清扬、陈钦宇、杜秋实、姜启伟、田瑞、徐瑞凡、卢瑞杰、徐瑞玲、葛瑞琪、张瑞松、潘瑞哲、王润基、陈润乾、尹润谦、徐润新、沈若萌、张若愚、刘尚浩、陆尚豪、周尚彦、陈善煌、蔡少飞、聂少恒、陈少元、胡胜鼎、刘胜宇、胡世强、马世荣、王世宇、余世平、周顺风、潘树婷、余淑英、周松阳、倪涛、云涛、金田、裴天、叶天、林天乐、季天然、崔天一、岳天元、余婷婷、王屯、张伟、曾王鼎、赵伟林、刘文、梁文峰、庞文杰、罗文静、姚文静、高文俊、杨文凯、黄文略、张文涛、马文婷、高熙、何翔、王向文、毕晓、刘晓东、王晓涵、陈晓康、张晓康、聂晓涛、程鑫、刘鑫、谢鑫、刘星辰、刘星潮、余星凯、李星友、杨新宇、陈旭、王轩宇、苏学成、林旭恒、傅旭伟、严永昌、王永强*、马永伟、罗燕峰、张阳、徐艳红、马艳茹、黄燕雯、李尧、李瑶、赵耀、孙耀峰、王耀辉、钱毅、余毅、张艺超、丁一凡、石一凡、吴宜佳、熊一超、贺英、周颖、罗英佳、钟寅敏、朴一石、王一松、张一翔、陈一潇、谭一轩、魏一萱、马一扬、刘一元、杨永伦、郭永强、吴永通、吴宇、程远、欧源、徐远帆、李远浩、王玉端、吴雨涵、孟宇航、邹宇恒、李宇坤、熊云帆、陈宇鹏、曹宇乾、王宇倩、张雨顺、林雨桐、谷雨仙、罗宇翔、游宇翔、刘宇轩、周宇翔、周宇阳、黄宇真、吴泽峰、王泽豪、赵泽华、任泽辉、沙张力、傅哲、徐哲安、谢振达、张正彦、郝哲文、苟志斌、马志诚、严志刚、邵志宏、黄志贤、陈志轩、吴志宇、任志洲、李卓书、张竹平、徐子安、王子豪、顾子慧、朱子佳、李子琳、张子鹏*、谢子威、高子怡、潘子庆、姚宗清。

业务与合规：陈辰玲、程雨侯、董洁吉、方伟、衡青张、贾洛、贾松、贾璐蔡、建梁、江婷周、杰宇杨、金陈、静子周、俊民郑、乐怡夏、林燕朱、妙君王、明明李、敏敏韩、宁王、盼盼

Wang, Peng Zhang, Ruyi Chen, Shangmian Sun, Shaoqing Wu, W.L. Xiao, Wei An, Wenqing Hou, Xianzu Wang, Xiaowen Sun, Xiaoxiang Wang, Xinyu Zhang, Xueyin Chen, Yao Xu, Yi Shao, Yiling Ma, Ying Tang, Yuehan Yang, Yuer Xu, Yukun Zha, Yuping Lin, Yuting Yan, Zekai Zhang, Zhe Ju, Zheren Gao, Zhongyu Wu, Zihua Qu, Ziyi Wan.

A.2. Acknowledgment

We would like to thank Dolly Deng and other testers for their valuable suggestions and feedback regarding the capabilities of DeepSeek-V4 series models.

B. Evaluation Details

Table 9 | Agentic Search vs. Retrieval Augmented Search for DeepSeek-V4-Pro.

Difficulty	Category	#	Agent Win	RAG Win	Tie	Agent%	RAG%	Tie%
Easy	Objective Q&A (客观问答)	196	110	43	43	56.1	21.9	21.9
	Subjective Q&A (主观问答)	321	198	56	67	61.7	17.4	20.9
Hard	Objective Q&A (客观问答)	168	102	33	33	60.7	19.6	19.6
	Subjective Q&A (主观问答)	184	126	27	31	68.5	14.7	16.8
Total (总计)		869	536	159	174	61.7	18.3	20.0

Table 10 | Cost Comparison:Agentic Search vs. Retrieval Augmented Search (Mean) for DeepSeek-V4-Pro. Most of the tool calls are parallel for Agentic Search.

Version	Tool Calls	Prefill (tokens)	Output (tokens)
V4 Agentic Search	16.2	13649	1526
V4 Retrieval Augmented Search	—	10453	1308

Table 11 | Comparative Evaluation of DeepSeek-V4-Pro and DeepSeek-V3.2 on Search Q&A Tasks.

Category	Subcategory	#	Internal Evaluation (内部综合评估)					
			V4 win	V3.2 win	tie	V4%	V3.2%	tie%
Objective Q&A (客观问答)	Single-value Search (单值信息查找)	95	36	10	49	37.9	10.5	51.6
	Entity Search (实体信息查找)	99	24	7	68	24.2	7.1	68.7
	Enumerative Search (枚举型信息查找)	95	19	8	68	20.0	8.4	71.6
	Subtotal (小计)	289	79	25	185	27.3	8.7	64.0
Subjective Q&A (主观问答)	Causal Analysis (原因分析)	100	28	5	67	28.0	5.0	67.0
	Comparison (对比)	96	28	20	48	29.2	20.8	50.0
	Advice Seeking (寻求建议)	92	23	8	61	25.0	8.7	66.3
	Recommendation (推荐)	95	26	19	50	27.4	20.0	52.6
	Planning & Strategy (攻略计划)	92	32	11	49	34.8	12.0	53.3
	Opinion & Evaluation (评价看法)	96	30	8	58	31.2	8.3	60.4
	Trend Analysis (趋势分析)	96	23	3	70	24.0	3.1	72.9
Subtotal (小计)		667	190	74	403	28.5	11.1	60.4
TOTAL (总计)		956	269	99	588	28.1	10.4	61.5

王、彭张、如意陈、上绵孙、少卿吴、W.L.肖、伟安、文清侯、贤祖王、晓文孙、晓翔王、新宇张、雪银陈、姚徐、毅邵、一灵马、英唐、月寒杨、悦尔徐、宇坤查、玉萍林、雨婷严、泽凯张、哲鞠、哲人高、忠宇吴、子华曲、子怡万。

A.2. 致谢

我们要感谢Dolly Deng及其他测试人员对DeepSeek-V4系列模型能力提出的宝贵建议与反馈。

B. 评估详情

表9 | DeepSeek-V4-Pro的代理搜索与检索增强搜索对比。

Difficulty	Category	#	Agent Win	RAG Win	Tie	Agent%	RAG%	Tie%
Easy	Objective Q&A (客观问答)	196	110	43	43	56.1	21.9	21.9
	Subjective Q&A (主观问答)	321	198	56	67	61.7	17.4	20.9
Hard	Objective Q&A (客观问答)	168	102	33	33	60.7	19.6	19.6
	Subjective Q&A (主观问答)	184	126	27	31	68.5	14.7	16.8
Total (总计)		869	536	159	174	61.7	18.3	20.0

表10 | 成本对比：DeepSeek-V4-Pro的代理式搜索与检索增强搜索（均值）。代理式搜索的大多数工具调用为并行处理。

Version	Tool Calls	Prefill (tokens)	Output (tokens)
V4 Agentic Search	16.2	13649	1526
V4 Retrieval Augmented Search	—	10453	1308

表11 | DeepSeek-V4-Pro与DeepSeek-V3.2在搜索问答任务上的对比评估。

Category	Subcategory	#	Internal Evaluation (内部综合评估)					
			V4 win	V3.2 win	tie	V4%	V3.2%	tie%
Objective Q&A (客观问答)	Single-value Search (单值信息查找)	95	36	10	49	37.9	10.5	51.6
	Entity Search (实体信息查找)	99	24	7	68	24.2	7.1	68.7
	Enumerative Search (枚举型信息查找)	95	19	8	68	20.0	8.4	71.6
	Subtotal (小计)	289	79	25	185	27.3	8.7	64.0
Subjective Q&A (主观问答)	Causal Analysis (原因分析)	100	28	5	67	28.0	5.0	67.0
	Comparison (对比)	96	28	20	48	29.2	20.8	50.0
	Advice Seeking (寻求建议)	92	23	8	61	25.0	8.7	66.3
	Recommendation (推荐)	95	26	19	50	27.4	20.0	52.6
	Planning & Strategy (攻略计划)	92	32	11	49	34.8	12.0	53.3
	Opinion & Evaluation (评价看法)	96	30	8	58	31.2	8.3	60.4
	Trend Analysis (趋势分析)	96	23	3	70	24.0	3.1	72.9
Subtotal (小计)		667	190	74	403	28.5	11.1	60.4
TOTAL (总计)		956	269	99	588	28.1	10.4	61.5



Figure 14 | Example output of a task that requires comparing two regular investment strategies for the NASDAQ.



Figure 15 | Example output of a task which requires researching 2020-2025 Nobel Science Prizes and generating an analytical PDF report.

Table 12 | Comparative Analysis of DeepSeek-V4-Pro and Gemini-3.1-Pro in Chinese Functional Writing.

Category	Subcategory	#	Internal Evaluation (内部综合评估)					
			DS win	Gem win	Tie	DS%	Gem%	Tie%
Business Writing (办公文本)	Report (报告)	527	350	162	15	66.41	30.74	2.85
	Proposal (方案策划)	291	181	103	7	62.20	35.40	2.41
	Education (教育培训)	159	100	56	3	62.89	35.22	1.89
	Email & Letter (邮件书信)	146	107	37	2	73.29	25.34	1.37
	Notice (通知公告)	72	43	24	5	59.72	33.33	6.94
	Professional (专业文本)	63	34	27	2	53.97	42.86	3.17
	Recruitment (招聘求职)	42	27	15	0	64.29	35.71	0.00
	Technical (技术文本)	29	22	7	0	75.86	24.14	0.00
	Review (介绍评价)	20	15	5	0	75.00	25.00	0.00
Subtotal (小计)		1349	879	436	34	65.16	32.32	2.52
Media Writing (媒体文本)	Social Media (社交媒体文案)	267	156	101	10	58.43	37.83	3.75
	Ad Copy (广告商品文案)	214	109	98	7	50.93	45.79	3.27
	Long-form Content (内容平台长文)	99	71	25	3	71.72	25.25	3.03
	News Report (新闻报道)	51	27	22	2	52.94	43.14	3.92
	Advertorial (营销软文)	17	12	4	1	70.59	23.53	5.88
	Headline (标题)	11	7	4	0	63.64	36.36	0.00
	Narration Script (口播文案)	4	2	1	1	50.00	25.00	25.00
	Comment (评论)	3	2	1	0	66.67	33.33	0.00
Subtotal (小计)		666	386	256	24	57.96	38.44	3.60
Everyday Writing (生活文本)	Congratulatory (祝贺文本)	101	54	41	6	53.47	40.59	5.94
	Communication (沟通回复)	100	71	26	3	71.00	26.00	3.00
	Reflection (心得感想)	90	68	17	5	75.56	18.89	5.56
	Review (介绍评价)	55	44	9	2	80.00	16.36	3.64
	Comment (评论)	44	34	8	2	77.27	18.18	4.55
Subtotal (小计)		390	271	101	18	69.49	25.90	4.62
Oral Writing (口头文本)	Speech (发言稿)	226	135	85	6	59.73	37.61	2.65
	Narration Script (口播文案)	51	25	23	3	49.02	45.10	5.88
	Sales Script (话术)	31	22	6	3	70.97	19.35	9.68
	Dialogue (对话文本)	10	4	6	0	40.00	60.00	0.00
	Congratulatory (祝贺文本)	1	1	0	0	100.00	0.00	0.00
Subtotal (小计)		319	187	120	12	58.62	37.62	3.76
Official Document (公文文本)	Administrative Doc (事务文书)	117	60	53	4	51.28	45.30	3.42
	Personal Doc (个人文书)	73	45	27	1	61.64	36.99	1.37
	Government Doc (行政公文)	34	19	14	1	55.88	41.18	2.94
	Speech (发言稿)	3	1	2	0	33.33	66.67	0.00
	Essay Writing (申论写作)	3	1	1	1	33.33	33.33	33.33
Subtotal (小计)		230	126	97	7	54.78	42.17	3.04
Academic Writing (学术文本)	Research Paper (学术论文)	104	67	32	5	64.42	30.77	4.81
	Coursework (课程作业)	90	53	35	2	58.89	38.89	2.22
	Academic Support (学术辅助)	15	11	3	1	73.33	20.00	6.67
	Science Outreach (专业科普)	7	6	1	0	85.71	14.29	0.00
Subtotal (小计)		216	137	71	8	63.43	32.87	3.70
Total (总计)		3170	1986	1081	103	62.65	34.10	3.25

表12 | DeepSeek-V4-Pro与Gemini-3.1-Pro在中文功能性写作中的对比分析

Category	Subcategory	#	Internal Evaluation (内部综合评估)					
			DS win	Gem win	Tie	DS%	Gem%	Tie%
Business Writing (办公文本)	Report (报告)	527	350	162	15	66.41	30.74	2.85
	Proposal (方案策划)	291	181	103	7	62.20	35.40	2.41
	Education (教育培训)	159	100	56	3	62.89	35.22	1.89
	Email & Letter (邮件书信)	146	107	37	2	73.29	25.34	1.37
	Notice (通知公告)	72	43	24	5	59.72	33.33	6.94
	Professional (专业文本)	63	34	27	2	53.97	42.86	3.17
	Recruitment (招聘求职)	42	27	15	0	64.29	35.71	0.00
	Technical (技术文本)	29	22	7	0	75.86	24.14	0.00
	Review (介绍评价)	20	15	5	0	75.00	25.00	0.00
Subtotal (小计)		1349	879	436	34	65.16	32.32	2.52
Media Writing (媒体文本)	Social Media (社交媒体文案)	267	156	101	10	58.43	37.83	3.75
	Ad Copy (广告商品文案)	214	109	98	7	50.93	45.79	3.27
	Long-form Content (内容平台长文)	99	71	25	3	71.72	25.25	3.03
	News Report (新闻报道)	51	27	22	2	52.94	43.14	3.92
	Advertorial (营销软文)	17	12	4	1	70.59	23.53	5.88
	Headline (标题)	11	7	4	0	63.64	36.36	0.00
	Narration Script (口播文案)	4	2	1	1	50.00	25.00	25.00
	Comment (评论)	3	2	1	0	66.67	33.33	0.00
Subtotal (小计)		666	386	256	24	57.96	38.44	3.60
Everyday Writing (生活文本)	Congratulatory (祝贺文本)	101	54	41	6	53.47	40.59	5.94
	Communication (沟通回复)	100	71	26	3	71.00	26.00	3.00
	Reflection (心得感想)	90	68	17	5	75.56	18.89	5.56
	Review (介绍评价)	55	44	9	2	80.00	16.36	3.64
	Comment (评论)	44	34	8	2	77.27	18.18	4.55
Subtotal (小计)		390	271	101	18	69.49	25.90	4.62
Oral Writing (口头文本)	Speech (发言稿)	226	135	85	6	59.73	37.61	2.65
	Narration Script (口播文案)	51	25	23	3	49.02	45.10	5.88
	Sales Script (话术)	31	22	6	3	70.97	19.35	9.68
	Dialogue (对话文本)	10	4	6	0	40.00	60.00	0.00
	Congratulatory (祝贺文本)	1	1	0	0	100.00	0.00	0.00
Subtotal (小计)		319	187	120	12	58.62	37.62	3.76
Official Document (公文文本)	Administrative Doc (事务文书)	117	60	53	4	51.28	45.30	3.42
	Personal Doc (个人文书)	73	45	27	1	61.64	36.99	1.37
	Government Doc (行政公文)	34	19	14	1	55.88	41.18	2.94
	Speech (发言稿)	3	1	2	0	33.33	66.67	0.00
	Essay Writing (申论写作)	3	1	1	1	33.33	33.33	33.33
Subtotal (小计)		230	126	97	7	54.78	42.17	3.04
Academic Writing (学术文本)	Research Paper (学术论文)	104	67	32	5	64.42	30.77	4.81
	Coursework (课程作业)	90	53	35	2	58.89	38.89	2.22
	Academic Support (学术辅助)	15	11	3	1	73.33	20.00	6.67
	Science Outreach (专业科普)	7	6	1	0	85.71	14.29	0.00
Subtotal (小计)		216	137	71	8	63.43	32.87	3.70
Total (总计)		3170	1986	1081	103	62.65	34.10	3.25

Table 13 | Comparative Analysis of DeepSeek-V4-Pro and Gemini-3.1-Pro in Chinese Creative Writing.

Subcategory (文体)	#	Instruction Following(指令遵循)						Writing Quality (写作质量)					
		DS	Gem	Tie	DS%	Gem%	Tie%	DS	Gem	Tie	DS%	Gem%	Tie%
Fiction (小说故事)	836	504	323	5	60.58	38.82	0.60	672	157	3	80.77	18.87	0.36
General Fiction (泛小说故事)	662	368	290	3	55.67	43.87	0.45	467	194	0	70.65	29.35	0.00
Fan Fiction (同人文)	410	253	150	3	62.32	36.95	0.74	338	67	1	83.25	16.50	0.25
General Fan Fic. (泛同人文)	202	111	90	1	54.95	44.55	0.50	161	40	1	79.70	19.80	0.50
Narrative (记叙文)	171	115	54	2	67.25	31.58	1.17	141	30	0	82.46	17.54	0.00
General Prose (泛散文)	124	83	40	1	66.94	32.26	0.81	88	36	0	70.97	29.03	0.00
Prose (散文)	112	74	38	0	66.07	33.93	0.00	92	20	0	82.14	17.86	0.00
Writing Style (文笔)	112	81	31	0	72.32	27.68	0.00	86	26	0	76.79	23.21	0.00
Classical Poetry (古诗文)	48	24	24	0	50.00	50.00	0.00	39	9	0	81.25	18.75	0.00
Modern Poetry (现代诗)	43	23	20	0	53.49	46.51	0.00	32	11	0	74.42	25.58	0.00
Lyrics (歌词)	30	8	22	0	26.67	73.33	0.00	16	14	0	53.33	46.67	0.00
Literary Appreciation (赏析)	27	20	7	0	74.07	25.93	0.00	18	9	0	66.67	33.33	0.00
General Argument. (泛议论文)	24	15	9	0	62.50	37.50	0.00	17	7	0	70.83	29.17	0.00
General Narrative (泛记叙文)	23	11	12	0	47.83	52.17	0.00	15	8	0	65.22	34.78	0.00
General Classical (泛古文诗歌)	9	5	4	0	55.56	44.44	0.00	5	4	0	55.56	44.44	0.00
Creative Writing (创意写作)	6	2	4	0	33.33	66.67	0.00	4	2	0	66.67	33.33	0.00
Argumentative (议论文)	5	5	0	0	100.00	0.00	0.00	5	0	0	100.00	0.00	0.00
General Mod. Poetry (泛现代诗)	2	1	1	0	50.00	50.00	0.00	2	0	0	100.00	0.00	0.00
Total (总计)	2837	1703	1119	15	60.03	39.44	0.53	2198	634	5	77.48	22.35	0.18

Table 14 | DeepSeek-V4-Pro vs. Claude-Opus-4.5 on Complex Instruction Following and Multi-Turn Writing.

		Internal Evaluation (内部综合评估)					
Category	#	DS	Opus	Tie	DS%	Opus%	Tie%
Complex Inst. Following (复杂指令跟随)	49	23	26	0	46.9%	53.1%	0.0%
Multi-Turn Writing (多轮写作)	147	67	76	4	45.6%	51.7%	2.7%
Total (总计)	196	90	102	4	45.9%	52.0%	2.0%

表13 | DeepSeek-V4-Pro与Gemini-3.1-Pro在中文创意写作上的对比分析。

Subcategory (文体)	#	Instruction Following(指令遵循)						Writing Quality (写作质量)					
		DS	Gem	Tie	DS%	Gem%	Tie%	DS	Gem	Tie	DS%	Gem%	Tie%
Fiction (小说故事)	836	504	323	5	60.58	38.82	0.60	672	157	3	80.77	18.87	0.36
General Fiction (泛小说故事)	662	368	290	3	55.67	43.87	0.45	467	194	0	70.65	29.35	0.00
Fan Fiction (同人文)	410	253	150	3	62.32	36.95	0.74	338	67	1	83.25	16.50	0.25
General Fan Fic. (泛同人文)	202	111	90	1	54.95	44.55	0.50	161	40	1	79.70	19.80	0.50
Narrative (记叙文)	171	115	54	2	67.25	31.58	1.17	141	30	0	82.46	17.54	0.00
General Prose (泛散文)	124	83	40	1	66.94	32.26	0.81	88	36	0	70.97	29.03	0.00
Prose (散文)	112	74	38	0	66.07	33.93	0.00	92	20	0	82.14	17.86	0.00
Writing Style (文笔)	112	81	31	0	72.32	27.68	0.00	86	26	0	76.79	23.21	0.00
Classical Poetry (古诗文)	48	24	24	0	50.00	50.00	0.00	39	9	0	81.25	18.75	0.00
Modern Poetry (现代诗)	43	23	20	0	53.49	46.51	0.00	32	11	0	74.42	25.58	0.00
Lyrics (歌词)	30	8	22	0	26.67	73.33	0.00	16	14	0	53.33	46.67	0.00
Literary Appreciation (赏析)	27	20	7	0	74.07	25.93	0.00	18	9	0	66.67	33.33	0.00
General Argument. (泛议论文)	24	15	9	0	62.50	37.50	0.00	17	7	0	70.83	29.17	0.00
General Narrative (泛记叙文)	23	11	12	0	47.83	52.17	0.00	15	8	0	65.22	34.78	0.00
General Classical (泛古文诗歌)	9	5	4	0	55.56	44.44	0.00	5	4	0	55.56	44.44	0.00
Creative Writing (创意写作)	6	2	4	0	33.33	66.67	0.00	4	2	0	66.67	33.33	0.00
Argumentative (议论文)	5	5	0	0	100.00	0.00	0.00	5	0	0	100.00	0.00	0.00
General Mod. Poetry (泛现代诗)	2	1	1	0	50.00	50.00	0.00	2	0	0	100.00	0.00	0.00
Total (总计)	2837	1703	1119	15	60.03	39.44	0.53	2198	634	5	77.48	22.35	0.18

表14 | DeepSeek-V4-Pro 对比 Claude-Opus-4.5 在复杂指令跟随与多轮对话写作中的表现。

		Internal Evaluation (内部综合评估)					
Category	#	DS	Opus	Tie	DS%	Opus%	Tie%
Complex Inst. Following (复杂指令跟随)	49	23	26	0	46.9%	53.1%	0.0%
Multi-Turn Writing (多轮写作)	147	67	76	4	45.6%	51.7%	2.7%
Total (总计)	196	90	102	4	45.9%	52.0%	2.0%